

Compositional Quantum Control Flow with Efficient Compilation in Qunity

MIKHAIL MINTS, California Institute of Technology, USA

FINN VOICHICK and LEONIDAS LAMPROPOULOS, University of Maryland, USA

ROBERT RAND, University of Chicago, USA

Abstract

Most existing quantum programming languages are based on the quantum circuit model of computation, as higher-level abstractions are particularly challenging to implement, especially ones relating to quantum control flow. The Qunity language, proposed by Voichick et al., offered such an abstraction in the form of a quantum control construct, with great care taken to ensure that the resulting language is still realizable. However, Qunity lacked a working implementation, and the originally proposed compilation procedure was very inefficient, with even simple quantum algorithms compiling to unreasonably large circuits.

In this work, we focus on the efficient compilation of high-level quantum control flow constructs, using Qunity as our starting point. We introduce a wider range of abstractions on top of Qunity’s core language that offer compelling trade-offs compared to its existing control construct. We create a complete implementation of a Qunity compiler, which converts high-level Qunity code into the quantum assembly language OpenQASM 3. We develop optimization techniques for multiple stages of the Qunity compilation procedure, including both low-level circuit optimizations as well as methods that consider the high-level structure of a Qunity program, greatly reducing the number of qubits and gates used by the compiler.

1 INTRODUCTION

In recent years, many *high-level quantum programming languages* have been proposed, aiming to allow algorithm implementers to work at a higher level of abstraction compared to the quantum circuit model of computation. Languages such as QML [1], Silq [2], Tower [22], and Qunity [20] walk a fine line between providing constructs familiar from classical computing and being realizable in quantum hardware, and all the physical restrictions associated with it. High-level quantum control flow constructs are particularly challenging since quantum programs must be compilable to fixed-length quantum circuits generated by classical computation [24]. As a result, proposed high-level quantum languages offer branching statements with significant restrictions on the expressive powers of their individual branches.

Among these languages, Qunity stands out for its unique handling of control flow that emphasizes compositionality. Qunity focuses on extending classical programming constructs into the domain of quantum computation in a natural way, and admits a *compositional denotational semantics* defined in terms of quantum operators (acting on state vectors) and superoperators (acting on density matrices), allowing for more flexible design than what is possible with unitary gate-based quantum circuits. Qunity’s `ctrl` construct offers a generalization of pattern matching that can coherently control on the output of an arbitrary, possibly irreversible, quantum or classical computation, relying on the BQP subroutine theorem [21]. The construct matches the scrutinee against a set of reversible classical patterns and outputs a superposition of the corresponding outcomes, provided that certain conditions (like the orthogonality of the patterns and the consistent erasure of quantum variables) hold. Consider, for example, Deutsch’s algorithm [7] as written in abstract Qunity syntax:

Authors’ addresses: Mikhail Mints, California Institute of Technology, Pasadena, USA, mmints@caltech.edu; Finn Voichick, finn@umd.edu; Leonidas Lampropoulos, leonidas@umd.edu, University of Maryland, College Park, USA; Robert Rand, University of Chicago, Chicago, USA, rand@uchicago.edu.

```

deutsch( $f$ ) :=
  let  $x =_{\text{Bit}}$  (had 0) in
     $\left( \text{ctrl } (f \ x) \left\{ \begin{array}{l} \emptyset \mapsto x \\ 1 \mapsto x \triangleright \text{gphase}_{\text{Bit}}(\pi) \end{array} \right\}_{\text{Bit}} \right) \triangleright \text{had}$ 

```

Given an *arbitrary* quantum oracle f that takes in and outputs a single qubit, `deutsch` will test whether f is constant or not using only a single evaluation (rather than two as in classical computing), which is then used in Qunity’s `ctrl` construct. Unlike other quantum programming languages, Qunity allows the programmer to write subroutines in a style similar to classical programming, without having to explicitly translate irreversible programs into unitary circuit forms. For instance, to input the constant-one oracle into the above example of Deutsch’s algorithm,

the programmer can just use $\lambda x \xrightarrow{\text{Bit}} 1$ instead of constructing a reversible map of the form $U_f |x, y\rangle = |x, y \oplus f(x)\rangle = |x, y \oplus 1\rangle$ as is typical in circuit-style programming. However, Qunity’s type system places several constraints on the expressions in the `ctrl` block to ensure its realizability: specifically, the right-hand-side patterns must *erase* the quantum variables in the scrutinee expression in a consistent way so that the compiler can perform automatic uncomputation.

To address these limitations, we introduce two new pattern matching constructs to Qunity, which trade off the subroutine capabilities of `ctrl` for greater flexibility in designing pure and mixed quantum computations. The new `match` construct corresponds to classical or mixed pattern matching and has minimal restrictions, allowing the programmer to easily write classical logic (which may then become a quantum subroutine). By contrast, `pmatch` (or pure match) allows for symmetric pattern matching between orthogonal sets of pure expressions, similarly to [15]. This allows us to easily express reversible quantum programs without the erasure restrictions of `ctrl` and simultaneously aids in compiling Qunity code to efficient circuits.

Compiling Qunity is a key contribution of this work. Qunity’s high degree of abstraction from the quantum circuit model of computation makes compilation particularly challenging. In addition to control flow and pattern matching, Qunity also generalizes the notion of error handling through `try/catch` statements (which can be viewed as projective measurements) and provides support for quantum sum types (whose semantics corresponds to direct sums of Hilbert spaces). To represent these abstractions, the compiler must encode Qunity types and contexts into quantum registers and allocate ancillary qubits to express Qunity’s non-unitary semantics in terms of unitary gates, a process which can lead to significant inefficiencies. While Voichick et al. describe an algorithm for converting Qunity into low-level qubit circuits, this procedure is more of a proof-of-realizability than a practical compiler: even simple quantum algorithms tend to be compiled into prohibitively large circuits. In this work, we develop a practical Qunity compiler, incorporating optimizations at various stages of the compilation process that significantly reduce the number of qubits and gates used in the final compiled circuits.

Main Contributions:

- We develop the first implementation of the Qunity language. We add a surface syntax to the core Qunity language, allowing the user to define parameterized types and subroutines that may involve recursion and higher-order operations (Section 2).
- We introduce new pattern-matching constructs into the core Qunity language, to avoid the restrictions imposed by the quantum control construct and improve the language’s expressiveness (Section 3).

- We implement the first working Qunity compiler that converts high-level Qunity code into low-level quantum circuits in OpenQASM 3, as well as a Qunity interpreter (Section 4).
- We design optimizations for the compilation procedure to reduce the number of qubits and gates used in the final compiled circuit, acting at several stages of the compilation process (Section 5 and Section 6).

2 QUNITY BY EXAMPLE

Qunity’s core language does not support higher-order functions and recursion, due to complications that arise when trying to generalize these ideas to the quantum setting [24]. Furthermore, Qunity’s core syntax does not allow subroutines to be associated with names to be called in a program multiple times. For these reasons, we augment Qunity with a *surface syntax* which provides a layer of *metaprogramming* on top of the core Qunity language. This system allows the user to create parameterized types, expressions, programs, and real number expressions that get evaluated at compile time during the *preprocessing* stage. User-defined types can be written as *variants* with named *constructors*, which are evaluated into the core language’s left and right injections during the preprocessing stage. The full grammar of the surface syntax, as used in Qunity source files, can be found in Appendix E. In this section, we showcase this syntax via a sequence of examples, taking the opportunity to introduce the new pattern matching constructs through small code snippets that will serve as running examples for the rest of the paper.

2.1 Deutsch’s Algorithm

Let us begin by revisiting the Deutsch example of the introduction and presenting it in terms of the new surface syntax:

```
def $deutsch{@f : Bit → Bit} : Bit :=
  let x = $plus in
  ctrl @f(x) [
    $0 → x;
    $1 → x ▷ gphase{pi}
  ]
  ▷ @had
end
```

Qunity Surface Syntax. In this syntax, the symbols \$, @, and # are used as sigils to write names for Qunity expressions, programs, and numbers, respectively. Qunity types have names starting with a capital letter, and quantum variables (which are part of the core syntax and remain after the preprocessing stage) start with a lowercase letter or underscore. The syntax ▷ is a shorthand for function application: $x ▷ @f$ is equivalent to $@f(x)$. Additionally, `let x = y in z` is syntactic sugar for `(lambda x → z)(y)`.

Qunity Datatypes. The type `Bit` is our first example of a user-defined datatype, which is provided by the Qunity standard library:

```
type Bit := $0 | $1 end
```

Here, `Bit` is defined as a variant type with two constructors that do not take any arguments. When the preprocessor evaluates this into the base Qunity syntax, it converts the variant types into sum types and the constructors into left and right injections. So, `$0` becomes `leftUnit@Unit()`, and `$1` becomes `rightUnit@Unit()`. Armed with `Bit`, we can define `$plus` and `@had`:

```

148 def $plus : Bit := @had($0) end
149 def @had : Bit → Bit := u3{pi/2, 0, pi} end

```

where `u3` is a Qunity primitive for general single-qubit gates with the type $\text{Unit} \oplus \text{Unit} \rightsquigarrow \text{Unit} \oplus \text{Unit}$ in the core Qunity language.

Qunity's Typing. We can also use this example to introduce Qunity's typing rules, which impose restrictions on ways quantum expressions and programs can be composed to ensure their physical realizability. Qunity has two distinct typing judgments: one *pure*, which corresponds to Kraus operator semantics, and one *mixed*, which corresponds to trace non-increasing superoperator semantics. These judgments and semantics are interrelated, allowing for a flexible combination of both reversible and irreversible programming. Here, we define `$deutsch` as a Qunity expression parameterized by an oracle `@f` that takes in and outputs a bit. According to the typing rules for the `ctrl` construct, `@f(x)` is typed using the *mixed expression* typing judgment: We write $\emptyset \parallel (x : \text{Bit}) \Vdash @f(x) : \text{Bit}$ to mean that `@f(x)` has mixed type (hence the notation $\Vdash$ as opposed to $\vdash$) `Bit` with classical context $\emptyset$ and quantum context $x : \text{Bit}$.¹ Thus, the semantics of `@f(x)` is a *superoperator* acting on the space of density matrices that correspond to a mixed state of a single qubit. Similarly, the Qunity typechecker types `@f` as a *quantum channel* $\vdash f : \text{Bit} \Rightarrow \text{Bit}$, and the program `@f` represents the same superoperator as the expression `@f(x)` – the difference being that the input space is associated with the *type* `Bit` rather than the *context* $x : \text{Bit}$.

When typing the `ctrl` expression, while the typing judgment accepts an arbitrary mixed expression as the scrutinee, it is much more restrictive regarding the kinds of expressions that can occur on both sides of the pattern-matching block. The typing judgment enforces that the left-hand-side (LHS) expressions must satisfy the *orthogonality judgment*, meaning that they represent orthogonal subspaces of the input type. For the typechecker to detect this, these patterns must satisfy a quite restrictive condition: the only program applications that they may use are the direct sum injections, corresponding to datatype constructors in the surface language. The right-hand-side (RHS) expressions must satisfy the *erasure judgment*, which enforces that all quantum variables in the scrutinee expression are consistently used in the same way on the RHS. This allows the Qunity compiler to correctly uncompute the garbage data generated when taking the purification of the scrutinee expression. Applying a global phase to one of the expressions as above is allowed, but a quantum transformation that prevents the compiler from automatically uncomputing the scrutinee would cause the typing judgment to fail.

2.2 Order Finding

The following example is a simplified demonstration of the order finding algorithm, which is the quantum part of Shor's algorithm for integer factorization [17]. Given relatively prime integers N and $a < N$, the goal is to find an integer r such that $a^r \equiv 1 \pmod{N}$. For the purposes of this demonstration, we will assume that N is a power of two – the resulting program is not useful for Shor's algorithm, but it is simpler to implement and illustrative of Qunity's features.

In our surface language, we can define a datatype of arrays of a given length, and then represent numbers `Num{#n}` in little-endian form using this datatype:

```

190 type Array{#n, 'a} := if #n <= 0 then Unit
191                       else 'a * Array{#n - 1, 'a} endif end
192 type Num{#n} := Array{#n, Bit} end

```

¹Note that the formalism governing classical and quantum contexts in this work differs from that in Voichick et al. For a detailed explanation of the differences and the Qunity typing rules see Appendix A.

Using this representation, we can write the following code to add a number $\#a$ fixed at compile time to a quantum $\#n$ -bit number (overflowing so that the addition is modulo $2^{\#n}$) in a reversible way:

```

197 def @add_const{#n, #a} : Num{#n} → Num{#n} :=
198   if #n <= 0 then
199     @id{Unit}
200   else
201     pmatch [
202       ($0, x) → (if #a % 2 = 0 then $0 else $1 endif,
203                 x ▷ @add_const{#n - 1, (#a - #a%2)/2});
204       ($1, x) → (if #a % 2 = 0 then $1 else $0 endif,
205                 x ▷ @add_const{#n - 1, (#a - #a%2)/2 + #a%2})
206     ]
207   endif
208 end

```

This showcases the use of the newly added `pmatch` construct. This form of pattern matching, which has *pure program* semantics, allows us to reversibly transform between two orthogonal sets of Qunity expressions. The type system imposes orthogonality requirements on *both* sides of the pattern-matching block - yet, the design of `pmatch` allows the orthogonality judgment to be more general than in `ctrl`: the expressions may involve applications of programs other than constructors provided that the type system is able to statically check that they correspond to *unitary* operators. For this particular example, `@add_const` is defined recursively, using versions of itself with $\#n - 1$ and different values of $\#a$ depending on whether there is a carry bit. It is clear that `@add_const{0, #a}` has unitary semantics since it is the identity map. Now, for any integer $\#n$, assuming that `@add_const{#n - 1, #a}` is unitary, the typechecker will correctly type the `pmatch` statement and conclude that `@add_const{#n, #a}` is also unitary because the two sets of orthogonal expressions *span* their corresponding spaces.

Since `pmatch` does not need to perform automatic uncomputation, it avoids the erasure judgment required by `ctrl`, so the use of variables on the right-hand of the pattern matching block is much less restricted. It would be difficult to define this operation using only the `ctrl` construct.

Now that we have addition of a constant, we can use it to define modular multiplication by an odd constant $\#a$ modulo $2^{\#n}$, using the approach in [9]:

```

231 def @mod_mult{#n, #a} : Num{#n} → Num{#n} :=
232   if #n = 1 then
233     @id{Num{#n}}
234   else
235     lambda (x0, x1) →
236       let (x0, x1) = (x0, x1 ▷ @mod_mult{#n - 1, #a}) in
237       ctrl x0 [
238         $0 → (x0, x1);
239         $1 → (x0, @add_const{#n - 1, (#a - 1) / 2}(x1))
240       ]
241     endif
242   end

```

This procedure works because if a is odd, then

$$\sum_{k=0}^{n-1} x_k 2^k a \equiv \sum_{k=0}^{n-1} x_k \left(2^{k+1} \frac{a-1}{2} + 1 \right),$$

where all addition is modulo 2^n . Now, we can define a modular exponentiation program that implements the operation $|x, y\rangle \mapsto |x, y \cdot a^x\rangle$:

```

246
247
248
249
250
251
252
253 def @mod_exp{#m, #n, #a} : Num{#m}*Num{#n} → Num{#m}*Num{#n} :=
254   if #m = 0 then
255     @id{Num{#m} * Num{#n}}
256   else
257     lambda ((x0, x1), y) →
258       let ((x0, x1), y) = ctrl x0 [
259         $0 → ((x0, x1), y);
260         $1 → ((x0, x1), @mod_mult{#n, #a}(y))
261       ]
262       in let (x0, (x1, y)) = (x0, @mod_exp{#m - 1, #n,
263         #a * #a}(x1, y))
264       in ((x0, x1), y)
265   endif
266 end
267

```

Here, we use `ctrl` to coherently condition on the bits of x to apply a modular multiplication by the current power of $\#a$. Now, we can define the main order finding procedure:

```

270
271 def $order_finding{#n, #a} : Num{#n} :=
272   ($repeated{#n, Bit, $plus}, $num_to_state{#n, 1})
273   > @mod_exp{#n, #n, #a}
274   > @fst{Num{#n}, Num{#n}}
275   > @adjoint {Num{#n}, Num{#n}, @qft{#n}}
276   > @reverse {#n, Bit}
277 end
278

```

The order finding procedure starts with a uniform superposition of the values of x and $y = 1$, applies the modular exponentiation, then discards the second register and applies the inverse quantum Fourier transform, reversing the result to display it in big-endian. Measuring the output will result in a random number of the form Nj/r for some integer j . The procedure relies on the reversibility of the operations and the pure typing judgment to maintain quantum coherence before applying the inverse quantum Fourier transform.

2.3 Grover's Algorithm

An interesting example of the use of `ctrl` is Grover's search algorithm [11]: Given any quantum oracle that takes in a value of some type and outputs a bit, we can start from a uniform superposition of the type and amplify the amplitude of states for which the oracle outputs 1. The generalized code for Grover's algorithm in Qunity is as follows:

```

291 def @grover_iter{'a, $equal_superpos : 'a,
292   @f : 'a → Bit} : 'a → 'a :=
293   lambda x → ctrl @f(x) [
294

```

```

295         $0 → x;
296         $1 → x ▷ gphase{pi}
297     ] ▷ @reflect{'a', $equal_superpos}
298 end
299
300 def $grover{'a', $equal_superpos : 'a',
301         @f : 'a → Bit, #n_iter} : 'a :=
302     if #n_iter = 0 then
303         $equal_superpos
304     else
305         $grover{'a', $equal_superpos, @f, #n_iter - 1} ▷
306         @grover_iter {'a', $equal_superpos, @f}
307     endif
308 end
309 end

```

Observe the use of the `ctrl` construct in this example: this code will work for any, potentially irreversible, function `@f`. This allows us to define quantum oracles for Grover's algorithm in a completely classical way, without ever needing to worry about the reversibility of the computations. The construction of such subroutines is greatly facilitated by the newly added `match` construct - a form of mixed or irreversible pattern matching that has minimal restrictions, functioning like matching constructs in classical programming languages. Consider, for instance, the following example of a function testing whether a list of bits has an odd number of ones:

```

318 def @is_odd_sum{#n} : List{#n, Bit} → Bit :=
319     if #n = 0 then
320         lambda l → $0
321     else
322         lambda l → match l [
323             $ListEmpty {#n, Bit} → $0;
324             @ListCons {#n, Bit}($0, l') → @is_odd_sum{#n - 1}(l');
325             @ListCons {#n, Bit}($1, l') →
326                 @not(@is_odd_sum{#n - 1}(l'))
327         ]
328     endif
329 end
330 end

```

Here, the `List` datatype is a list of variable but bounded length, defined as:

```

333 type List{#n, 'a} :=
334     | $ListEmpty
335     | @ListCons of (if #n <= 0 then Void
336                     else 'a * List{#n - 1, 'a} endif)
337 end
338

```

The definition of `@is_odd_sum` is typed as a mixed program, with the `match` block typed as a mixed expression. It is defined recursively, using `@is_odd_sum{#n - 1}` in the RHS of the pattern-matching block. This means that it would be difficult to write this in terms of just the `ctrl` construct, which types its RHS expressions as pure. Since `match` has mixed semantics, it is not

directly responsible for performing any uncomputation and is able to discard quantum data: hence, it can avoid the erasure requirements present in `ctrl`.

Now, we can use this as an oracle in Grover’s algorithm simply by writing

```
def #n := 2 end
$grover{List{#n, Bit}, $equal_superpos_list{#n},
  @is_odd_sum{#n}, 3}
```

where `$equal_superpos_list` is an expression generating an equal superposition of all possible lists of bits with lengths bounded by `#n`.

3 EXTENDING THE LANGUAGE: NEW CONTROL FLOW CONSTRUCTS

Qunity’s `ctrl` construct is very powerful, as it allows an arbitrary quantum computation, potentially involving decoherence, to be used as a subroutine in a pure, reversible computation [20]. However, this feature is accompanied by some notable restrictions that can make this construct more difficult to use than necessary. The first significant restriction is that the left-hand-side (LHS) expressions of the `ctrl`-block must be classical—that is, they cannot include any invocations of `u3` or `rphase` (in fact, any programs other than the direct sum injections), and so their semantics correspond to classical basis states. The second restriction arises from the requirement that the right-hand-side (RHS) expressions must satisfy the *erasure judgment*, which stipulates that all variables in the quantum context Δ of the scrutinee expression must be present in the same way in each of the RHS expressions. If, for instance, Δ involves the variable x , this essentially forces the programmer to make all the RHS expressions have the form $(x, \dots)$ if they want to output something other than just x itself. See Appendix B for the definition of the erasure inference rules.

The erasure requirement is essential to ensure that it is possible to implement `ctrl`: without it, the semantics of `ctrl` will not be reversible. At the circuit level, the context Δ needs to be shared to produce the RHS expressions, but applying the adjoint of the purified scrutinee expression inevitably creates another copy of $\mathcal{H}(\Delta)$ alongside the output $\mathcal{H}(T')$, and the erasure judgment is needed to coherently delete this. The classical requirement in the orthogonality judgment does not seem as obvious; indeed, applying a unitary transformation to an orthogonal set of states maintains their orthogonality. However, a problem appears when trying to modify the compilation circuit for `ctrl` to allow this. In the general case, there is no well-defined basis along which to share the quantum context Δ given some basis for $\mathcal{H}(T)$, since an arbitrary unitary transformation from the computational basis to the given one does not necessarily decompose as a tensor product. Additionally, inserting an arbitrary unitary transformation into the semantics of `ctrl` might make it non-compositional.

Regardless, it is useful to have a construct that defines a reversible transformation between two orthogonal sets of expressions, which do not necessarily need to be in the classical basis, while avoiding the erasure requirement. Also, if the programmer is implementing classical logic (which may be part of a larger quantum algorithm), it may be useful to have a construct that defines *irreversible* control flow in a manner most similar to a `match` statement in classical programming languages, again without the restrictions of the erasure judgment. These ideas give rise to two new Qunity primitives: `pmatch` and `match`.

3.1 The `pmatch` Construct

The `pmatch` construct (the “p” stands for “pure”) has a design that is very similar to that of the symmetric pattern-matching language (SPM) described by Sabry et al. [15]. We can write a typing judgment for `pmatch` as follows:

Feature	ctrl	match	pmatch
Purely typed	✓	✗	✓
Mixed scrutinee allowed	✓	✓	✗
Mixed expressions allowed on RHS	✗	✓	✗
Non-classical expressions allowed on LHS	✗	✗	✓
Avoids erasure requirements	✗	✓	✓
Can be used in the RHS of a ctrl	✓	✗	✓
Can be used in the RHS of a match	✓	✓	✓
Can be used in the LHS or RHS of a pmatch	✓*	✗	✓*

*if the unitary judgment holds

Table 1. Comparison of Qunity’s three pattern-matching constructs (extended version in Appendix F).

$$\begin{array}{c}
\text{ortho}_T^*(e_1, \dots, e_n) \quad \emptyset \parallel \Delta_j \vdash e_j : T \quad \forall j \\
\text{ortho}_{T'}^*(e'_1, \dots, e'_n) \quad \emptyset \parallel \Delta_j \vdash e'_j : T' \quad \forall j \\
\hline
\vdash \text{pmatch} \left\{ \begin{array}{c} e_1 \mapsto e'_1 \\ \dots \\ e_n \mapsto e'_n \end{array} \right\}_{T'} : T \rightsquigarrow T' \quad \text{T-PMATCH}
\end{array}$$

Each pattern and branch must be appropriately typed, and they must also satisfy a relaxed version of orthogonality, allowing the expressions to be in a basis other than the classical one (the full formal definition of ortho^* can be found in Appendix C). If the classical requirement from the regular ortho judgment was still in place, then **pmatch** could be described as syntactic sugar over the **ctrl** construct, using a technique known as *specialized erasure* [20].² Instead, we generalize **pmatch** allowing expressions in an arbitrary basis on both sides. This, however, is done at the expense of losing some of the capabilities of **ctrl**: since the semantics of **pmatch** is that of a pure program rather than a pure expression, it does not include a scrutinee directly and thus does not have the power to uncompute arbitrary mixed expressions via purification. It also has a requirement dictating that the same contexts must be used symmetrically in the LHS and RHS, making it possible to simply flip the two sides to invert the program.

We can describe the semantics of **pmatch** as:

$$\llbracket \vdash \text{pmatch} \left\{ \begin{array}{c} e_1 \mapsto e'_1 \\ \dots \\ e_n \mapsto e'_n \end{array} \right\}_{T'} : T \rightsquigarrow T' \rrbracket |v\rangle = \sum_{j=1}^n \llbracket \emptyset : \emptyset \parallel \Delta_j \vdash e'_j : T' \rrbracket \llbracket \emptyset : \emptyset \parallel \Delta_j \vdash e_j : T \rrbracket^\dagger |v\rangle$$

Each term of the above sum is a projection of the input state onto one of the LHS expressions, combined with an application of the corresponding RHS expression. If there are no free variables in the context Δ_j , then $\llbracket \emptyset : \emptyset \parallel \Delta_j \vdash e_j : T \rrbracket^\dagger |v\rangle$ will just be a scalar, but in general, this will be a quantum state encoding the values of the quantum variables in the context Δ_j , which are then mapped into the type T' by $\llbracket \emptyset : \emptyset \parallel \Delta_j \vdash e'_j : T' \rrbracket$.

For an example of the benefit provided by **pmatch**, consider the following Qunity code which prepares an equal superposition of **Maybe{Bit}**, which is isomorphic to a qutrit:

²This is discussed by Voichick et al. by the name “match”, but we change the terminology as **match** will be a different construct.

```

442 $0
443 > u3{2 * arccos(sqrt(1 / 3)), 0, 0}
444 > pmatch [
445   $0 → $Nothing {Bit};
446   $1 → @Just {Bit}($plus)
447 ]
448

```

Here, we are easily able to transform between two orthogonal bases of expressions, one in the type `Bit` and the other in the type `Maybe{Bit}`. The semantics of this can be written as:

$$\begin{aligned}
 & (| \$Nothing\{Bit\} \rangle \langle \$0| + | \$Just\{Bit\}(\$plus) \rangle \langle \$1|) \left(u_3 \left(2 \arccos \sqrt{\frac{1}{3}}, 0, 0 \right) |0\rangle \right) \\
 &= (| \$Nothing\{Bit\} \rangle \langle \$0| + | \$Just\{Bit\}(\$plus) \rangle \langle \$1|) \left(\frac{1}{\sqrt{3}} | \$0 \rangle + \frac{\sqrt{2}}{\sqrt{3}} | \$1 \rangle \right) \\
 &= \frac{1}{\sqrt{3}} | \$Nothing\{Bit\} \rangle + \frac{\sqrt{2}}{\sqrt{3}} | \$Just\{Bit\}(\$plus) \rangle \\
 &= \frac{1}{\sqrt{3}} (| \$Nothing\{Bit\} \rangle + | \$Just\{Bit\}(\$0) \rangle + | \$Just\{Bit\}(\$1) \rangle)
 \end{aligned}$$

If we were to implement this only using `ctrl`, we would need to write something like the following:

```

464 $0
465 > u3 {2 * arccos(sqrt(1 / 3)), 0, 0}
466 > lambda x → ctrl x [
467   $0 → (x, $Nothing {Bit});
468   $1 → (x, @Just {Bit}($plus))
469 ]
470
471 > lambda (
472   ctrl x' [
473     $Nothing {Bit} → ($0, x');
474     @Just {Bit}(_) → ($1, x')
475   ]
476 ) → x'
477

```

This is an instance of the specialized erasure pattern: in order to respect the erasure judgment, we must explicitly uncompute the variable `x` through an inverted `ctrl` expression.

Moreover, unitaries can be combined with `pmatch` in more complicated ways, such as the following piece of code which allows us to view the rest of the tuple in the standard or Fourier basis depending on the value of the first element:

```

483 $repeated{4, Bit, $minus} > pmatch [
484   ($0, (x, y)) → (@qft{2}($plus, (x, ())), @qft{2}(y));
485   ($1, @qft{3}(x, y)) →
486     (@qft{2}($minus, (x, ())), @add_const{2, 1}(y));
487 ]
488
489
490

```

Note that we can use the Quantum Fourier Transform (`@qft` - see its implementation in Appendix J.2) and `@add_const` (from Section 2.2) on both sides of the pattern-matching block since the type system can recognize that both of them have unitary semantics.

3.2 The `match` Construct

In a sense, the `match` is a step in the opposite direction from `pmatch`: while `pmatch` extends the “pure” capabilities of `ctrl`, `match` extends its “mixed” capabilities. The goal of the `match` construct is to have as few restrictions as possible, creating a semantics that most closely corresponds to a `match` statement in a classical programming language like OCaml. We can create a typing judgment for `match`, ensuring that the types of the scrutinee, patterns, and branches align in the current context, while also requiring orthogonality:

$$\frac{\begin{array}{c} \Gamma \parallel \Delta, \Delta_0 \Vdash e : T \quad \text{ortho}_T(e_1, \dots, e_n) \\ \emptyset \parallel \Gamma_j \vdash e_j : T \text{ for all } j \quad \Gamma, \Gamma_j \parallel \Delta, \Delta_1 \Vdash e'_j : T' \text{ for all } j \end{array}}{\Gamma \parallel \Delta, \Delta_0, \Delta_1 \Vdash \text{match } e \left\{ \begin{array}{c} e_1 \mapsto e'_1 \\ \dots \\ e_n \mapsto e'_n \end{array} \right\}_T : T'} \quad \text{T-MATCH}$$

and we can define its semantics as:

$$\begin{aligned} \llbracket \sigma : \Gamma \parallel \Delta, \Delta_0, \Delta_1 \Vdash \text{match } e \left\{ \begin{array}{c} e_1 \mapsto e'_1 \\ \dots \\ e_n \mapsto e'_n \end{array} \right\}_T : T' \rrbracket (|\tau, \tau_0, \tau_1\rangle \langle \tau', \tau'_0, \tau'_1|) \\ = \sum_{v \in \mathbb{V}(T)} \langle v | \llbracket \sigma : \Gamma \parallel \Delta, \Delta_0 \Vdash e : T \rrbracket (|\tau, \tau_0\rangle \langle \tau', \tau'_0|) | v \rangle \cdot \\ \cdot \sum_{j=1}^n \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket \emptyset : \emptyset \parallel \Gamma_j \vdash e_j : T \rrbracket^\dagger | v \rangle \cdot \\ \cdot \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel \Delta, \Delta_1 \Vdash e'_j : T' \rrbracket (|\tau, \tau_1\rangle \langle \tau', \tau'_1|) \end{aligned}$$

This is very similar to the semantics of `ctrl`, described in Appendix A. Here,

$$\langle v | \llbracket \sigma : \Gamma \parallel \Delta, \Delta_0 \Vdash e : T \rrbracket (|\tau, \tau_0\rangle \langle \tau', \tau'_0|) | v \rangle$$

gives the *probability* of the expression e being measured as $|v\rangle$ in the classical basis, with the given classical and quantum context variables. The expression e is thus treated as a “classical black box”: thus, there is no way to distinguish the pure state $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ from the mixed state $\frac{1}{2}(|0\rangle\langle 0| + |1\rangle\langle 1|)$, since their probability distributions when measured in the classical basis are identical. This behavior is exactly the same as in `ctrl`. However, unlike `ctrl`, which has pure semantics, `match` has mixed semantics: it uses the superoperator $\llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel \Delta, \Delta_1 \Vdash e'_j : T' \rrbracket$ formed from the RHS expression e'_j .

The `match` construct is designed to be used in circumstances where we do not care if our computation is reversible or not. For instance, it is convenient to implement a logical-AND operation using `match` as follows:

```
def @and : Bit * Bit → Bit :=
  lambda x → match x [
    ($1, $1) → $1;
    else → $0;
```

```

540   ]
541 end

```

Note that the `else` keyword here is a syntactic construct that gets converted to the remaining expressions spanning `Bit * Bit` (that is, `($1, $0)` and `($0, _)`) during the preprocessing stage. The `match` construct no longer has the erasure requirement from `ctrl`, simply because it does not need to be reversible and can always discard any extra data it has, without the need to uncompute it. Like for `pmatch`, there exists a version of `match` that can be built as syntactic sugar over `ctrl`. This version can be constructed by simply combining all the variables in Δ into a tuple and pairing it with each of the RHS expressions, and then feeding the result into `snd`. However, the additional feature of `match` when it is implemented as a primitive is the fact that the RHS expressions, unlike in `ctrl`, do not need to be pure. See the example in Section 2.3 for an instance where this becomes useful for quantum algorithm implementation. The `match` construct is designed to be maximally similar to pattern matching in classical languages, avoiding the restrictions placed on `ctrl` other than the orthogonality requirement for the LHS. Using the `match` construct, Qunity programmers may write long and complex classical algorithms without worrying about reversibility, and then use them as subroutines in larger quantum algorithms.

4 COMPILER DESIGN

Figure 1 shows an outline of the main components of the Qunity interpreter and compiler, which are implemented in OCaml.

The preprocessor first evaluates the surface syntax definitions and produces a single Qunity expression, with the same abstract syntax as in Voichick et al. The typechecker then uses Qunity’s extended typing judgments to check if the expression is well-typed and outputs a data structure representing a proof of typing, effectively elaborating the Qunity expression with helpful information that can later be used by the interpreter and the compiler, such as the contexts associated with its subexpressions. We can then either interpret or compile this elaborated expression: On the left side of Figure 1, we depict a simulator explicitly calculating the matrices corresponding to its semantics. This is an exponentially slow process, however, as it is equivalent to simulating a quantum computer on a classical computer. The main focus of this work is the compilation process shown on the right side, transforming a high-level elaborated Qunity expression into OpenQASM 3. This process can be performed efficiently on a classical computer, and the final output is a quantum circuit in a form that can be run on quantum hardware.

4.1 Formalism

Before making improvements to Qunity’s original proof-of-concept compilation scheme, we found it necessary to revise some of the mathematical formalism that governs how the low-level circuits should be constructed in order to establish stronger correctness invariants throughout the process.

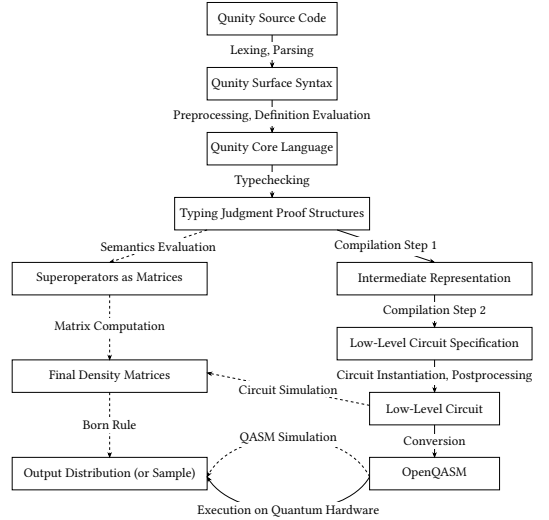


Fig. 1. Diagram showing the procedures used in the Qunity interpreter (left path), and the Qunity compiler (right path). Solid and dashed arrows indicate polynomial and exponential time processes respectively.

In particular, many Qunity values are stored in a quantum register, represented as bitstring using a type-based *encoding*. However, the space of all possible encodings does not necessarily span the space of all possible register states—for instance, a qutrit (a 3-level quantum system) can be stored in two bits, but only 3 out of 4 possible bit strings will be valid encodings. The compilation procedure in Voichick et al. includes some low-level circuits (namely, the direct sum) that can potentially output invalid encodings due to the way flag and output registers in the circuit’s components are combined. The compilation lemmas do not address this possibility, which makes some parts of the compilation procedure incorrect.

We considered two methods of resolving the problem caused by the possibility of gates outputting invalid encodings when formalizing the compilation lemmas. The first possibility was to create an error-checking circuit component, which would ensure that any state in the invalid encoding subspace is sent to the error subspace (with nonzero values for the flag qubits). However, the issue with this approach is that such an error-checking component would need to be inserted after every circuit component in the compilation procedure, making it much more complex and inefficient. The alternative approach was to ensure that no states enter the invalid encoding subspace under any conditions. We can create a mathematical definition of this as follows:

Definition 4.1. For a type T , define the space of valid encodings as

$$\mathbb{W}(T) = \text{span} \{ |\text{enc}(v)\rangle \mid v \in \mathbb{V}(T) \},$$

where enc is the encoding function that maps values to bitstrings:

$$\text{enc}(\text{()}) = ""$$

$$\text{enc}(\text{left}_{T_0 \oplus T_1} v) = "0" ++ \text{enc}(v)$$

$$\text{enc}(\text{right}_{T_0 \oplus T_1} v) = "1" ++ \text{enc}(v)$$

$$\text{enc}((v_0, v_1)) = \text{enc}(v_0) ++ \text{enc}(v_1)$$

Definition 4.2. We say that it is possible to implement a norm-non-increasing operator $E : \mathcal{H}(T) \rightarrow \mathcal{H}(T')$ if there is a low-level circuit implementing a unitary operator

$$U : \mathbb{C}^{2^{\text{size}(T)+n_p}} \rightarrow \mathbb{C}^{2^{\text{size}(T')+n_f}} \text{ for some } n_p, n_f \in \mathbb{N} \text{ such that for all } v \in \mathbb{V}(T), v' \in \mathbb{V}(T'),$$

$$\langle \text{enc}(v'), 0^{\otimes n_f} | U | \text{enc}(v), 0^{\otimes n_p} \rangle = \langle v' | E | v \rangle,$$

and, for all $v \in \mathbb{V}(T)$,

$$U | \text{enc}(v), 0^{\otimes n_p} \rangle \in \mathbb{W}(T') \otimes \mathbb{C}^{2^{n_f}},$$

and, for all $v' \in \mathbb{V}(T')$,

$$U^\dagger | \text{enc}(v'), 0^{\otimes n_f} \rangle \in \mathbb{W}(T) \otimes \mathbb{C}^{2^{n_p}}.$$

This definition is formed by augmenting Definition 6.2 from Voichick et al. with conditions that guarantee encoding validity is preserved. With this new definition, any optimizations to the low-level circuit components need to respect the encoding-validity-preserving property.

4.2 Gates, Circuits, and Circuit Specifications

The final output of the compiler takes the form of a low-level qubit circuit, consisting of a series of gates, which can be:

- The identity
- A single-qubit $U_3(\theta, \phi, \lambda)$ gate
- A global phase gate
- A reset gate, which measures and resets a qubit to the $|0\rangle$ state

- An error-measurement gate, which measures a qubit and signals an error if it is in the $|1\rangle$ state
- A swap gate
- A controlled gate, with a list of control qubits
- Special zero-state labels and potential deletion labels, whose uses are detailed in Section 5.3.

These gates can very straightforwardly be converted into OpenQASM 3 code. However, each gate must already contain information that specifies the indices of the qubits it is applied to, which makes this data structure lack modularity. When compiling the intermediate representation into low-level circuits, we often want to separately construct two circuit components representing different operators, and then feed the output of one into the input of the other, or maybe reuse the same input and prep qubits for controlled versions of two different operators (see Section 5.1), which is difficult to do with just the gates. The Qunity compiler uses a system of *circuits* and *circuit specifications* to allow for the construction of low-level circuits out of smaller components. This system is constructed in a way that facilitates the tracking of qubits in various registers to ensure that the components fit together properly.

A circuit consists of a series of gates combined with additional data that specifies the roles of the qubits used by it - specifically, which qubit indices constitute the input registers, the output registers, the prep register (additional inputs prepared in the $|0\rangle$ state), the flag register (output qubits expected to be in the $|0\rangle$ state in the absence of error), and the garbage register (output qubits that are measured and discarded, corresponding to a partial trace operation).

A circuit specification is a wrapper around a function, `circ_fun`, that can construct a circuit given a list of input registers, a set of used wires (qubits that are in use elsewhere and cannot be used as new prep qubits), and additional instantiation settings that can dictate certain aspects of how the circuit is built. In addition to the circuit, this function also outputs the updated set of used wires. The circuit specification also includes data on the required input and output register dimensions. Circuit specifications are more modular than gates or circuits, because they are not tied to specific qubit indices - these are not realized until their `circ_fun` is called. Thus, it is possible to easily construct circuit specifications from other circuit specifications through various circuit construction functions. For instance, one such function could take in two circuit specifications, and define its `circ_fun` to instantiate the first specification into a circuit, and then use its output registers and used wires set to instantiate the second specification into a circuit, and then output a combined circuit whose gate is a sequence of the two original circuits' gates. The possible ways in which circuit specifications can be combined together correspond closely to the low-level circuits defined in Appendix H.1 of Voichick et al.

4.3 The Intermediate Representation

The intermediate representation used in the Qunity compiler describes high-level circuit diagrams in which the wires correspond to Hilbert spaces associated with Qunity types and contexts. This representation is built from *operators* that take in some list of input registers and output some registers. This level abstracts away the “flag” and “garbage” utility registers and can thus describe operators with non-unitary semantics. Operators in the intermediate representation can be either built from smaller components using primitive constructors (corresponding to specific low-level circuit components), or they can be constructed from a series of commands in a fashion similar to a simple imperative language. These custom operators can associate registers with variable names and apply other operators to them, enforcing the condition that every variable must be used exactly once. This design makes it convenient to describe high-level circuits corresponding to each Qunity typing rule.

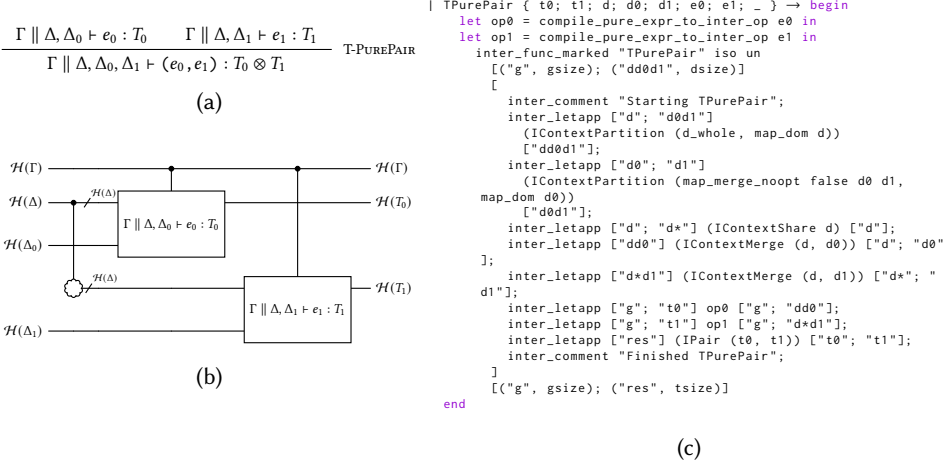


Fig. 2. (a) The typing judgment T-PUREPAIR for typing pairs of Qunity expressions with a product type. (b) The compilation circuit for T-PUREPAIR as presented in Appendix H.2 of Voichick et al. (c) Excerpt from the `compile_pure_expr_to_inter_op` function in the Qunity compiler, written in OCaml. This shows the compilation of the T-PUREPAIR typing judgment into the intermediate representation, corresponding to the circuit in (b).

In the first step of the compilation procedure, the typing judgment proofs obtained from Qunity expressions by the typechecker are converted into custom operators in the intermediate representation. This can be done easily because of the close correspondence between the intermediate representation and the circuit diagrams described in Appendix H.2 of Voichick et al. Figure 2c shows the code for compiling T-PUREPAIR into the intermediate representation. We can see that this describes an operator that is a map of the form $\mathcal{H}(\Gamma) \otimes \mathcal{H}(\Delta, \Delta_0, \Delta_1) \rightarrow \mathcal{H}(\Gamma) \otimes \mathcal{H}(T)$. First, the typing judgment proofs of e_0 and e_1 are compiled into the intermediate representation. Then, they are used in the `inter_func_marked`, which is a wrapper around the `InterFunc` (user-defined operator) constructor, adding an extra operator around it using isometry and unitary judgment information. The `inter_comment` is just used to create annotations for debugging purposes. Next, the input register corresponding to the space $\mathcal{H}(\Delta, \Delta_0, \Delta_1)$ is partitioned into separate registers corresponding to $\mathcal{H}(\Delta)$, $\mathcal{H}(\Delta_0)$, and $\mathcal{H}(\Delta_1)$. Note that the compiler maintains an invariant that in a register corresponding to a context, the values of all variables must be stored in lexicographic order. Thus, the partitioning circuit is not necessarily the identity and may involve some rearrangement of wires, which is done on the low level through a special circuit construction function. Then, the share gate (whose low-level implementation involves a new prep register of the same size as the input and CNOT gates) is applied, followed by the previously compiled operator circuits, and the result is combined into a single register. While it can be somewhat verbose, this code is essentially equivalent to the circuit diagram shown in Figure 2b.

5 LOW-LEVEL OPTIMIZATIONS

This section describes optimizations that modify the second step of the Qunity compilation procedure (where the intermediate representation is instantiated into low-level circuits that keep track of qubit indices) and additional post-processing steps that are applied to the resulting circuits. Each time a circuit component is instantiated, the compiler needs to recursively instantiate its constituent parts, and decide how to connect the inputs and outputs of the parts to each other. This

process may also require allocation of additional prep qubits, which increases the complexity of the generated circuit. The goal of the low-level optimizations is to improve the process of constructing these low-level circuits, avoiding unnecessarily using many qubits at once.

5.1 Improving the Direct Sum Circuits

An important component of the second step of the compilation procedure is being able to implement the direct sum of two operators. Given circuits U_0 implementing $E_0 : \mathcal{H}(T_0) \rightarrow \mathcal{H}(T'_0)$ (with input size s_0 , output size s'_0 , prep size p_0 , and flag size f_0), and U_1 implementing $E_1 : \mathcal{H}(T_1) \rightarrow \mathcal{H}(T'_1)$ (with corresponding sizes s_1, s'_1, p_1, f_1), the goal is to implement the direct sum circuit $E_0 \oplus E_1 : \mathcal{H}(T_0 \oplus T_1) \rightarrow \mathcal{H}(T'_0 \oplus T'_1)$. The implementation must be in accordance with Definition 4.2, recalling that the encoding of the direct sum injections is defined as

$$\begin{aligned} \text{enc}(\text{left}_{T_0 \oplus T_1} v) &= "0" \mathbin{++} \text{enc}(v) \mathbin{++} "0"^{(\max(\text{size}(T_0), \text{size}(T_1)) - \text{size}(T_0))} \\ \text{enc}(\text{right}_{T_0 \oplus T_1} v) &= "1" \mathbin{++} \text{enc}(v) \mathbin{++} "0"^{(\max(\text{size}(T_0), \text{size}(T_1)) - \text{size}(T_1))} \end{aligned}$$

An initial naïve implementation that still respects the encoding-validity-preserving property specified in Definition 4.2 uses $\min\{s_0, s_1\} + p_0 + p_1$ prep wires. Because the number of prep wires scales with the size of the input register, this can cause the number of qubits used by the circuit to explode when an operator is summed with itself multiple times, which is a common pattern in the compilation of quantum control.

We have created an improved version of the direct sum circuit, which still ensures encoding validity but reuses input and prep qubits as much as possible. Table 2 shows the number of prep qubits needed for this improved circuit in different cases. The full circuits and a sample proof of correctness for one of them are shown in Appendix G.

$s_0 \geq s_1, s'_0 \geq s'_1, p_0 \geq p_1$	$p_0 + \max\{0, f_1 - f_0\}$
$s_0 \geq s_1, s'_0 \geq s'_1, p_0 \leq p_1$	$p_1 + \max\{0, f_1 - f_0 + p_0 - p_1\}$
$s_0 \geq s_1, s'_0 \leq s'_1, p_0 \geq p_1$	$p_0 + s'_1 - s'_0$
$s_0 \geq s_1, s'_0 \leq s'_1, p_0 \leq p_1$	$p_1 + \max\{0, s'_1 - s'_0 + p_0 - p_1\}$

Table 2. Number of prep qubits needed for the improved direct sum circuit in 4 different cases. The 4 other cases can be obtained from these ones by applying a commutativity isomorphism (which is just an X gate).

5.2 Recycling of Garbage and Flag Qubits

An important optimization to the Qunity compiler comes from the possibility of measuring and resetting garbage and flag qubits in the middle of a circuit and reusing them as new prep qubits for subsequent circuit components - that is, *recycling* the garbage and flag qubits. This has to be done with some care, because there are situations in which we want to avoid resetting these qubits. Specifically, we may be taking the purification of a circuit, which feeds its garbage qubits into the output register, or we may be using an error-handling circuit, which uses the flag register to coherently toggle a new prep qubit, outputting values in the space $\mathcal{H} \oplus \mathbb{C}$, where $\mathcal{H}$ is the output space of the original circuit. This means that a circuit specification has to “know”, at the time of its instantiation into a circuit, whether or not it is allowed to recycle its garbage or flag register. This is done by using the instantiation settings, which are passed to a circuit specification at the time of instantiation, and are used to decide whether the qubits can be recycled. Purification is implemented as a function that takes in a circuit specification and outputs another one, whose instantiation function passes in a value of false for the instantiation settings’ reset_garb field

when calling the wrapped circuit specification's instantiation function. The same thing happens with the error-handling circuits and the `reset_flag` field. This optimization reduces the qubit count of the circuits output by the compiler, since now, if there are many circuit components in series that each can throw an error and are not part of a larger error-handling component, the compiler does not have to allocate new qubits every time.

5.3 Post-Processing Optimizations

We have introduced a post-processing step that optimizes the quantum circuits generated by the Qunity compiler. The overall procedure for low-level optimization converts the input circuit into a list of gates and then continually applies *optimization passes* to this list, until changes stop taking place. In a single optimization pass, the gate list is processed from left to right, detecting possible patterns that can be optimized. This process is guaranteed to eventually terminate, since each change made to the circuit has to decrease the number of qubits or gates. The currently implemented pass includes the following:

- Canceling a gate that is immediately followed by its adjoint.
- Combining adjacent controlled- U and anti-controlled- U gates into just U .
- Removing physical swap gates without controls and relabeling the subsequent wires.
- Performing a commutation pass: taking a gate and commuting it to the right until either it cannot commute past something (in which case it is reverted to its original position) or it cancels with something.
- Classical propagation - moving through the circuit and keeping track of qubits we know to be in a classical state, modifying any gates that are controlled on them by removing controls or deleting the gates.
- Detecting regions where a CNOT gate is applied, sharing one qubit to another one initially in the $|0\rangle$ state, then any other gates are the Pauli X gate or only use either qubit as a control, and then an identical CNOT is applied. In this case, it is possible to transfer the controls on the second qubit to the first one, which can make it possible to delete the second qubit with the procedure described below.
- Deleting qubits (or gates on a qubit between measurements) if certain conditions are met.

We will now discuss this last point in more detail. This mechanism relies on high-level information from the *isometry judgment* in the Qunity typechecker (see Appendix D) to identify candidate qubits for potential deletion. These candidates are either garbage qubits or flag qubits that we are certain will always be in the $|0\rangle$ state when measured.

The candidate qubits are identified during the procedure of instantiating a circuit specification into a concrete circuit. The instantiation settings, which include Boolean fields `reset_garb` and `reset_flag` for whether or not the garbage and flag registers should be reset, as described in Section 5.2, also contain a field `iso` for whether or not this circuit can be interpreted as an isometry and thus whether we can expect the flag qubits to be in the $|0\rangle$ state. This variable is set to true by a special wrapper circuit specification that acts very similarly to the purification and error-handling circuits, and is added to the circuit based on the results of the isometry judgment in the first step of the compilation process. Now, during instantiation, if `reset_garb` is true, then a special gate called a *potential deletion label* is added to the qubits in the garbage register. Similarly, if both `reset_flag` and `iso` are true, the potential deletion labels are also added to the flag register. Additionally, instead of measuring flag qubits known to be in the $|0\rangle$ state, the compiler instead adds a special *zero-state label* onto them.

However, not all candidate qubits are safe to delete, because they may be entangled with the output qubits at some point during the computation, possibly being used as ancillas. In the post-processing stage, all the deletion labels are first shifted to the left as far as possible until they hit a measurement gate or a zero-state label to ensure that they are selecting an entire region of interest. Then, during an optimization pass, when seeing a potential deletion label, the segment between it and the next measurement is evaluated to determine whether or not it is safe to delete. The procedure determines that all gates in the region involving the selected qubit are safe to delete if all of them are Pauli X gates, controlled X gates with the qubit as a target, or uncontrolled global phase gates. This is because if the qubit starts in the $|0\rangle$ state, then applying only these gates will not have any effect on the reduced density matrix of the system obtained by removing the qubit. At the end of the optimization procedure, if a qubit has no more gates remaining on it, it is completely removed from the circuit.

6 HIGH-LEVEL OPTIMIZATIONS

6.1 Simplifying the Spanning Circuit

A significant part of the circuit for compiling the T-CTRL judgment in the first step of the compilation process is the following circuit component:

$$\mathcal{H}(T) \longrightarrow \boxed{\llbracket \text{ortho}_T(e_1, \dots, e_n) \rrbracket} \xrightarrow{\mathcal{H}(T)^{\oplus n}} \boxed{\bigoplus_j \llbracket e_j \rrbracket^\dagger} \longrightarrow \bigoplus_j \mathcal{H}(\Gamma_j)$$

This circuit is quite inefficient: it uses $O(n)$ extra qubits to store the register corresponding to $\mathcal{H}(T)^{\oplus n}$, and it involves many expensive associativity isomorphisms to transform the direct sum structure of the given space to something of the form $\mathcal{H}(T) \oplus (\mathcal{H}(T) \oplus (\mathcal{H}(T) \oplus \dots))$. In this work, we introduce two major changes to this: allowing direct sums of operators to be taken *along a binary tree*, and replacing the two above circuit components with a single one that goes directly from $\mathcal{H}(T)$ to $\bigoplus_j \mathcal{H}(\Gamma_j)$, without the intermediate step of $\mathcal{H}(T)^{\oplus n}$. Refer to Appendix H for the definitions and notations used for describing sums of operators along a binary tree and leveling a tree to a given height.

We can derive a binary tree from the structure of the spanning judgment (or the spanning* judgment, see Appendix C), as follows:

Definition 6.1 (Tree derived from the spanning judgment).

- The trees derived from S-VOID, S-UNIT, and S-VAR are all Leaf.
- The S-SUM rule states that if $e_1, \dots, e_m$ span T and $e'_1, \dots, e'_m$ span T' , then the left injections of the e_j and the right injections of the e'_j together span $T \oplus T'$. The tree derived from S-SUM is a root whose left subtree is the tree derived from the left expressions and whose right subtree is the tree derived from the right expressions.
- The S-PAIR rule states that if $e_1, \dots, e_m$ span T , and for each $1 \leq j \leq m$, the expressions $e'_{j,1}, \dots, e'_{j,n_j}$ span T' , then the list of pairs $(e_j, e'_{j,k})$ span $T \otimes T'$. The tree derived from S-PAIR is constructed by taking the tree $\mathcal{R}_0$ derived from $\text{spanning}_T(e_1, \dots, e_m)$, and replacing each j th leaf with the tree $\mathcal{R}_1^j$ derived from $\text{spanning}_{T'}(e'_{j,1}, \dots, e'_{j,n_j})$. This essentially decomposes a tensor product into a direct sum structure.
- The same constructions apply for the starred versions of the rules, with the tree derived from S*-UNAPP being the same as that derived from $\text{spanning}_T^*(e_1, \dots, e_n)$.

Now, we want to define $\llbracket \text{spanning}_T(e_1, \dots, e_n) \rrbracket$ in such a way that when $\text{spanning}_T^*(e_1, \dots, e_n)$ holds with tree structure $\mathcal{R}$, then for all j , for all $\sigma_j \in \mathbb{V}(\Gamma_j)$, we have that

$$\llbracket \text{spanning}_T(e_1, \dots, e_n) \rrbracket \llbracket e_j \rrbracket |\sigma_j\rangle = \text{inj}_j^{\mathcal{R}} |\sigma_j\rangle.$$

That is, we are constructing an operator

$$\llbracket \text{spanning}_T(e_1, \dots, e_n) \rrbracket : \mathcal{H}(T) \rightarrow \bigoplus_{j:\mathcal{R}} \mathcal{H}(\Gamma_j).$$

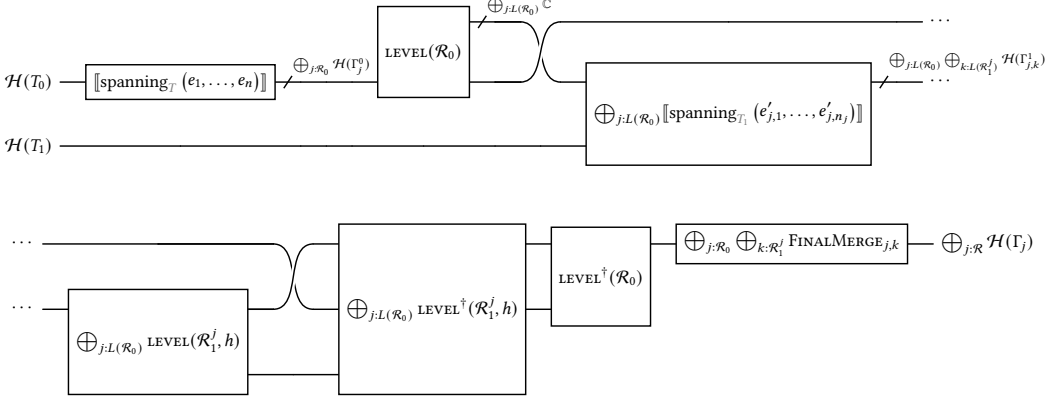
For S-VOID, S-UNIT, and S-VAR, this operator is simply the identity.

For S-SUM, we define

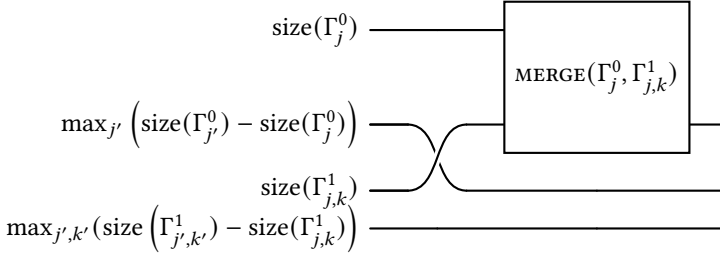
$$\llbracket \text{spanning}_{T_0 \oplus T_1} \left(\text{left}_{T_0 \oplus T_1} e_1, \dots, \text{left}_{T_0 \oplus T_1} e_n, \text{right}_{T_0 \oplus T_1} e'_1, \dots, \text{right}_{T_0 \oplus T_1} e'_n \right) \rrbracket = \llbracket \text{spanning}_{T_0}(e_1, \dots, e_n) \rrbracket \oplus \llbracket \text{spanning}_{T_1}(e'_1, \dots, e'_n) \rrbracket.$$

This is significantly better than the existing compilation circuit for S-SUM, which takes the direct sum of the left and right injections, unnecessarily duplicating the data already stored in the first qubit of the register corresponding to the direct sum.

Now, for S-PAIR, the operator is defined by the following circuit:



where we define $h = \max_j \{\text{height}(\mathcal{R}_1^j)\}$, we define $\text{FINALMERGE}_{j,k}$ (with a diagram drawn in the style of low-level circuits) as:



where the bottom two wires are marked as flag registers, since they contain zero-padding regions from direct sum encodings.

The LEVEL operator is designed to separate a potentially uneven direct sum encoding structure into a separate *index register* and *data register*. This allows us to concatenate the tree path from $\mathcal{R}_0$ and $\mathcal{R}_1^j$ into a single path in the combined tree, then merge the pieces of data from the two sides of the pair and put them in the correct place in $\mathcal{R}$.

While this circuit appears complicated, it is more efficient than the existing solution in many cases. In fact, if the expressions contain no variables in the first elements of the pairs, this entire circuit actually simplifies to the identity. This is because in that case, we do not need to do anything to combine the indices of the two trees, as the encodings already correspond to the desired structure.

Now, all of these can be extended in the obvious way to the spanning^* judgment, and the $\text{S}^*\text{-UNAPP}$ circuit can be implemented by simply applying $\llbracket f \rrbracket^\dagger \llbracket \text{spanning}_T^*(e_1, \dots, e_n) \rrbracket$.

With this change, we can now modify the circuit for the compilation of T-CTRL to use a single circuit component that converts the input $\mathcal{H}(T)$ directly into $\bigoplus_{j:\mathcal{R}} \mathcal{H}(\Gamma_j)$, where $\mathcal{R}$ is the tree derived from the spanning judgment. Then, that can be simply converted into a sum over the tree derived from the orthogonality judgment by removing certain subtrees of $\mathcal{R}$ (and on the circuit level, applying adjoints of direct sum injections). Then, for the rest of the circuit, all direct sums can be taken along this tree instead of a standardized associativity structure.

6.2 Compilation of **pmatch**

The same construction as above can be used to construct a high-level circuit for compiling T-PMATCH into the intermediate representation. This can be done as follows:

$$\mathcal{H}(T) \xrightarrow{\llbracket \text{ortho}_I^*(e_1, \dots, e_n) \rrbracket} \bigoplus_{j:\mathcal{R}_0} \mathcal{H}(\Delta_j) \xrightarrow{\text{TREEREARANGE}(\mathcal{R}_0, \mathcal{R}_1)} \bigoplus_{j:\mathcal{R}_1} \mathcal{H}(\Delta_j) \xrightarrow{\llbracket \text{ortho}_I^*(e_1, \dots, e_n) \rrbracket^\dagger} \mathcal{H}(T')$$

where $\mathcal{R}_0$ and $\mathcal{R}_1$ are binary trees derived from the corresponding orthogonality judgments, except the leaves in them are also labeled with the indices of the corresponding expressions. Then, TREEREARANGE is an algorithm for transforming one binary tree into another using only operations from a certain set, encoded as a quantum circuit.

The possible operations are:

- A right tree rotation, which can be implemented as the associativity isomorphism $(T_0 \oplus T_1) \oplus T_2 \rightarrow T_0 \oplus (T_1 \oplus T_2)$, for which a low-level circuit implementation exists.
- A left tree rotation, implementable as the adjoint of the associativity isomorphism.
- Switching the left and right subtree, which can be implemented with a single Pauli X gate.
- Other transformations applied to the subtrees of a given tree, which can be implemented using the direct sum circuit.
- A sequence of two transformations, which can be implemented as a sequence of two circuits.
- A special *conditional commutation* transformation, shown in Figure 3. It can be implemented with a multi-controlled X gate, where the target is the qubit corresponding to the root node, and the controls have states corresponding to the path taken.

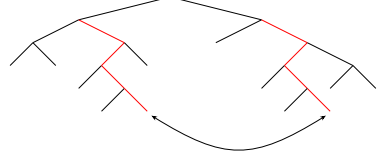


Fig. 3. An example of the conditional commutation tree transformation, which swaps the two vertices shown in this diagram. Their paths must be identical except for the very first step from the root.

The algorithm for transforming trees, expressed in terms of these allowed operations, is:

- (1) Transform the input tree so it has the same shape as the goal, ignoring the values at the leaves. This can be done by transferring nodes from one subtree to another making sure they have the correct numbers, and then making recursive calls on each subtree. The transferring of nodes can be accomplished by a sequence of tree rotations.
- (2) For each leaf value that needs to be transferred to a different location, do the following to switch the leaves at the two locations:
 - (a) Bring the leaf at the larger depth (the one farther from the root) to the same depth as the other one. This can be accomplished by a sequence of tree rotations and commutations.
 - (b) Make the trees have identical paths from the root except for the first place in which their paths differ. This can be done by a sequence of commutation operations applied to the appropriate subtrees.
 - (c) Apply the special conditional commutation operation to switch the two leaves.

- (d) Apply the operations in the first two steps in reverse order so that the only effect of the entire transformation is swapping the two leaves of interest.

In the OCaml implementation of the compiler, running this algorithm creates a record of all the tree transformations that need to be applied to perform the operation. Then, the resulting tree transformations are converted into the intermediate representation in the manner described above, thus completing the compilation of `pmatch`. Beside the fact that this allows `pmatch` to work in an arbitrary basis, it is also much more efficient than implementing a classical-basis version of it as syntactic sugar over `ctrl`: the tree rearrangement algorithm attempts to avoid significantly increasing the height of the tree at any intermediate step, which means that it does not unnecessarily use additional qubits.

7 EVALUATION

In this section we detail our evaluation of the Qunity compiler. We primarily focus on the effectiveness of the compiler in reducing compiled circuit size, but we also report on our differential unit testing that we used to gain confidence in the correctness of our implementation.

Compilation Efficiency. We evaluate the Qunity compiler on several benchmark programs, shown in Table 3. We compare the most recent version of the compiler with the initially implemented version, which followed directly the compilation procedure of Voichick et al. (with the direct sum circuit corrected to preserve encoding validity but not optimized). This baseline did not support the new pattern-matching constructs, so where possible, the benchmark code was rewritten to equivalent expressions using only `ctrl`. We also compare the performance of our compiler to low-level circuits constructed directly in Qiskit [14], using circuits from `qiskit.circuit.library` when relevant. The gate counts reported in this table are after the circuit has been transpiled by Qiskit to only use single-qubit gates and CNOT.

Benchmark	Qunity (unoptimized)		Qunity (optimized)		Qiskit	
	Qubits	Gates	Qubits	Gates	Qubits	Gates
Phase conditioned on AND of 5 qubits	769	147,871	9	121	5	101
Quantum Fourier Transform (5 qubits)	15	282	6	143	5	54
Phase estimation example (5 qubits)	75	1465	6	275	5	60
Order finding (#n=5, #a=13)	–	–	10	460	–	–
Reversible CDKM Adder [6] (5 bits)	1031	160,321	11	152	11	159
Grover (5-bit match oracle, 1 iteration)	1131	??	7	760	6	389
Grover (List sum oracle, #n=2, 1 iteration)	–	–	19	215,104	5	116

Table 3. Benchmark tests comparing the performance of the optimized and unoptimized Qunity compilers with an implementation in Qiskit. The code used for these benchmarks is shown in Appendix J. The missing entries in the unoptimized Qunity compiler result from the naive compiler not supporting the new pattern-matching constructs. We also do not have a suitable reference for the order-finding circuit. The entry marked with ?? has such a large circuit that it was infeasible to transpile it to obtain a gate count.

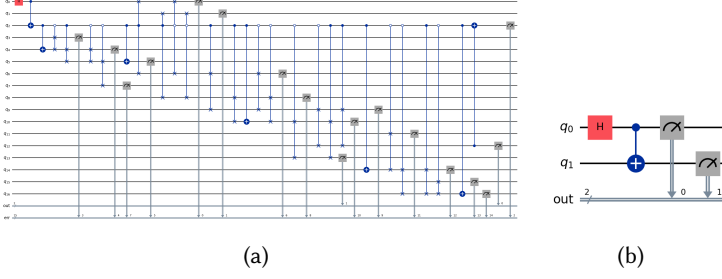


Fig. 4. (a) The circuit produced by compiling a Qunity program for preparing the state $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ without compiler optimizations. (b) The circuit produced using the the optimized compiler for the same program.³

The results show a very significant improvement of the optimized Qunity compiler compared to the unoptimized one. In many instances, the optimized Qunity compiler reaches a gate count that is comparable to the handcrafted Qiskit implementation, whereas the baseline Qunity compiler resulted in orders of magnitude larger circuits. Still there is room for improvement: to uncover the limits of our compiler implementation, we included a benchmark that implements Grover’s algorithm using the list sum oracle, which compiles into 215,104 low-level gates, while it is possible to manually construct a small circuit that accomplishes the same task. The inefficiencies arise due to isomorphisms and transformations between types that are introduced during the compilation procedure: while the optimizations introduced in Section 6 help to simplify them to a large extent, there are still many inefficiencies present that are nontrivial to eliminate.

Still, for most programs, including smaller ones that we used for testing, our Qunity compiler results in practical circuits. For one example, Figure 4 demonstrates the improvement in the efficiency of the compiler resulting from the optimizations implemented in this work when preparing a Bell state using the quantum control construct. Note that the simplest way to prepare this state in Qunity is using entanglement through variable reuse: `$plus > lambda x → (x, x)`, which compiles to the circuit in Figure 4b even with the unoptimized compiler. However, for this example, the entanglement is achieved by defining a CNOT gate in terms of the `ctrl` construct, which has a much more complicated compilation procedure. (This can be used to write circuit-style programs in Qunity: see Appendix I). The circuit produced by the unoptimized compiler, in Figure 4a, needs 17 qubits and 27 CNOT and controlled-SWAP gates just to represent an operation equivalent to a single CNOT gate. This is due to a combination of several factors, including the unoptimized direct sum circuits, the unnecessary associativity isomorphisms, and the redundancy in the original orthogonality circuit. It is difficult to simplify this circuit by post-processing alone: the higher-level optimizations described in this work are necessary. Using the combination of these optimization techniques, the compiler is able to reduce this circuit to a form that can be more easily handled by the post-processing optimizations and converted to the circuit in Figure 4b.

Differential Unit Testing. Finally, we also created a test suite to verify the correctness of the compilation procedure, checking it against the output of the interpreter. The test proceeds as follows: given a Qunity expression e of type T , we evaluate its semantics as a density matrix $\rho = \llbracket \emptyset : \emptyset \parallel \emptyset \Vdash e : T \rrbracket$ using the Qunity interpreter. We run the compiler to generate a low-level qubit circuit, and simulate this circuit. Taking a projection onto the subspace where the flag register

³These circuits have measurements because the output register is always measured at the end when running the Qunity compiler.

is in the $|0\rangle$ state and taking a partial trace over the garbage registers, we obtain a state $\tilde{\rho}$. Then, for each $v \in \mathbb{V}(T)$, we check that $\langle v | \rho | v \rangle = \langle \text{enc}(v) | \tilde{\rho} | \text{enc}(v) \rangle$.

Most programs in this suite are small, and many are just simple demonstrations of various patterns (such as pattern matching constructs nested in different ways, operations on datatypes, and error handling). We exclude programs from Table 3 that are too expensive to simulate.

8 RELATED WORK

Many existing quantum programming languages are based on the paradigm of “quantum data, classical control,” which was popularized by Selinger [16]. In this model, quantum circuits can be constructed by a classical computer, using classical data which may be obtained by performing measurements on qubits. Most of the widely-used quantum languages rely on this paradigm from low-level languages like OpenQASM [5], Cirq [8], and Qiskit [14] to higher-level languages like Quipper [10], Q# [18] and Qualtran [12].

However, we often want to express programs with quantum control flow, such as the quantum walk algorithm by Childs et al. [4]. This runs into problems, as fully general quantum control flow is physically infeasible, as argued by Bădescu and Panangaden [3] and later Yuan et al. [24]. A number of languages have attempted to address this issue through restricting control flow, including QML [1], Symmetric Pattern Matching (SPM) [15], Silq [2], and Tower [22]. Of these, Silq and Tower compile to circuits.

Venev’s [19] compiler for Silq programs is focused on the problem of *uncomputation*, specifically in compiling and optimizing away the results of the common compute-copy-uncompute pattern used in the generation of quantum oracles. Silq’s uncomputation, however, is restricted to applications of qfree (strictly classical) functions - Qunity’s `ctrl` construct, on the other hand, enables uncomputation of an arbitrary mixed program (although doing this on a program that introduces superposition will lead to norm-decreasing semantics). While none of our examples make use of this feature, it may be useful in more advanced quantum algorithms such as the triangle finding algorithm [13], which uses a quantum subroutine that is itself a probabilistic quantum algorithm.

Tower’s compiler, Spire [23] is closer in its aims to this work. Like Qunity, Tower is a high-level language with sophisticated (QRAM-based) control flow, and Spire aims to optimize it at a high level. Spire uses two key optimizations: conditional flattening (which consists of combining nested if statements, reducing the controls on internal unitaries) and conditional narrowing, which pulls statements out of quantum ifs (again, reducing the number of controls). These two optimizations prove surprisingly valuable in practice, enabling a significant reduction in the circuits’ T -complexity, and showing that high-level optimizations of quantum programs could be useful in practice.

Qunity is set apart from the previous work listed above by its focus on compositionality and its denotational semantics. It takes the unique and unusual approach of defining both a pure and mixed typing judgment, allowing for a richer interplay of reversible and irreversible programming. Qunity is similar to classical functional languages, while languages like Silq and Tower follow an imperative paradigm. The metaprogramming layer allows for higher-order functions and recursion without running into the fundamental limitations of implementing them as quantum constructs [3]. Qunity’s pattern-matching constructs offer a novel and expressive way of expressing quantum control flow.

9 CONCLUSION AND FUTURE WORK

In this work, we show how to efficiently compile Qunity, a high-level quantum programming language with compositional semantics, by combining low-level optimizations that simplify the construction of qubit circuit components with higher-level optimizations that simplify manipulations of Hilbert spaces.

Still, there are more optimizations that we would like to implement in the future. For example, in instances where no purification is performed, the `try/catch` statement could be implemented by measuring the flag register, and applying some gates depending on the measurement outcome. Similarly, the `match` construct could be implemented by measuring the scrutinee expression and using the result to evaluate one of the RHS expressions. Including this capability can simplify many Qunity programs with classical computation outside of quantum subroutines. Other avenues for potential improvements include more sophisticated post-processing algorithms for low-level circuits. With improvements like these, we believe Qunity can truly become a practical quantum programming language, offering a promising alternative to circuit-based quantum languages and advancing the development of tools for high-level quantum programming and algorithm design.

DATA AVAILABILITY STATEMENT

All our work will be made publicly available. We intend to submit an artifact for artifact evaluation that includes the compiler and interpreter for Qunity, the example programs contained in this paper, and scripts to replicate the results of the evaluation section.

REFERENCES

- [1] T. Altenkirch and J. Grattage. 2005. A Functional Quantum Programming Language. In *20th Annual IEEE Symposium on Logic in Computer Science (LICS'05)*. IEEE, 249–258. <https://doi.org/10.1109/lics.2005.1>
- [2] Benjamin Bichsel, Maximilian Baader, Timon Gehr, and Martin Vechev. 2020. Silq: a high-level quantum language with safe uncomputation and intuitive semantics. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation* (London, UK) (PLDI 2020). Association for Computing Machinery, New York, NY, USA, 286–300. <https://doi.org/10.1145/3385412.3386007>
- [3] Costin Bădescu and Prakash Panangaden. 2015. Quantum Alternation: Prospects and Problems. *Electronic Proceedings in Theoretical Computer Science* 195 (Nov. 2015), 33–42. <https://doi.org/10.4204/eptcs.195.3>
- [4] Andrew M. Childs, Ben W. Reichardt, Robert Spalek, and Shengyu Zhang. 2007. Every NAND formula of size N can be evaluated in time $N^{1/2+o(1)}$ on a quantum computer. arXiv:quant-ph/0703015 [quant-ph] <https://arxiv.org/abs/quant-ph/0703015>
- [5] Andrew Cross, Ali Javadi-Abhari, Thomas Alexander, Niel De Beaudrap, Lev S. Bishop, Steven Heidel, Colm A. Ryan, Prasahnt Sivarajah, John Smolin, Jay M. Gambetta, and Blake R. Johnson. 2022. OpenQASM 3: A Broader and Deeper Quantum Assembly Language. *ACM Transactions on Quantum Computing* 3, 3 (Sept. 2022), 1–50. <https://doi.org/10.1145/3505636>
- [6] Steven A. Cuccaro, Thomas G. Draper, Samuel A. Kutin, and David Petrie Moulton. 2004. A new quantum ripple-carry addition circuit. arXiv:quant-ph/0410184 [quant-ph] <https://arxiv.org/abs/quant-ph/0410184>
- [7] David Deutsch and Roger Penrose. 1985. Quantum theory, the Church–Turing principle and the universal quantum computer. *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences* 400, 1818 (1985), 97–117. <https://doi.org/10.1098/rspa.1985.0070> arXiv:https://royalsocietypublishing.org/doi/pdf/10.1098/rspa.1985.0070
- [8] Cirq Developers. 2024. Cirq. Zenodo. <https://doi.org/10.5281/ZENODO.4062499>
- [9] Craig Gidney. 2017. Simple Algorithm for Multiplicative Inverses mod 2^n . <https://algassert.com/post/1709>
- [10] Alexander S. Green, Peter LeFanu Lumsdaine, Neil J. Ross, Peter Selinger, and Benoît Valiron. 2013. Quipper: A Scalable Quantum Programming Language. *CoRR* abs/1304.3390 (2013). arXiv:1304.3390 <http://arxiv.org/abs/1304.3390>
- [11] Lov K Grover. 1996. A fast quantum mechanical algorithm for database search. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*. 212–219.
- [12] Matthew P. Harrigan, Tanuj Khattar, Charles Yuan, Anurudh Peduri, Noureldin Yosri, Fionn D. Malone, Ryan Babbush, and Nicholas C. Rubin. 2024. Expressing and Analyzing Quantum Algorithms with Qualtran. <https://doi.org/10.48550/arXiv.2409.04643> arXiv:2409.04643
- [13] Frédéric Magniez, Miklos Santha, and Mario Szegedy. 2007. Quantum Algorithms for the Triangle Problem. *SIAM J. Comput.* 37, 2 (2007), 413–424. <https://doi.org/10.1137/050643684> arXiv:quant-ph/0310134
- [14] Qiskit contributors. 2023. Qiskit: An Open-source Framework for Quantum Computing. <https://doi.org/10.5281/zenodo.2573505>
- [15] Amr Sabry, Benoît Valiron, and Juliana Kaizer Vizzotto. 2018. From Symmetric Pattern-Matching to Quantum Control (Extended Version). arXiv:1804.00952 [cs.LO] <https://arxiv.org/abs/1804.00952>
- [16] Peter Selinger. 2004. Towards a Quantum Programming Language. *Mathematical Structures in Computer Science* 14, 4 (August 2004), 527–586. <https://doi.org/10.1017/S0960129504004256>

[17] Peter W. Shor. 1997. Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer. *SIAM J. Comput.* 26, 5 (Oct. 1997), 1484–1509. <https://doi.org/10.1137/s0097539795293172>

[18] Krysta Svore, Alan Geller, Matthias Troyer, John Azariah, Christopher Granade, Bettina Heim, Vadym Kliuchnikov, Mariia Mykhailova, Andres Paz, and Martin Roetteler. 2018. Q#: Enabling Scalable Quantum Computing and Development with a High-level DSL. In *Proceedings of the Real World Domain Specific Languages Workshop 2018* (Vienna, Austria) (*RWDSL2018*). Association for Computing Machinery, New York, NY, USA, Article 7, 10 pages. <https://doi.org/10.1145/3183895.3183901>

[19] Hristo Venev, Timon Gehr, Dimitar Dimitrov, and Martin Vechev. 2024. Modular Synthesis of Efficient Quantum Uncomputation. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (2024), 2097–2124.

[20] Finn Voichick, Liyi Li, Robert Rand, and Michael Hicks. 2023. Qunity: A Unified Language for Quantum and Classical Computing. *Proceedings of the ACM on Programming Languages* 7, POPL (Jan. 2023), 921–951. <https://doi.org/10.1145/3571225>

[21] John Watrous. 2009. *Quantum Computational Complexity*. Springer New York, New York, NY, 7174–7201. https://doi.org/10.1007/978-0-387-30440-3_428

[22] Charles Yuan and Michael Carbin. 2022. Tower: data structures in Quantum superposition. *Proceedings of the ACM on Programming Languages* 6, OOPSLA2 (2022), 259–288.

[23] Charles Yuan and Michael Carbin. 2024. The T-Complexity Costs of Error Correction for Control Flow in Quantum Computation. *Proceedings of the ACM on Programming Languages* 8, PLDI (2024), 492–517.

[24] Charles Yuan, Agnes Villanyi, and Michael Carbin. 2024. Quantum Control Machine: The Limits of Control Flow in Quantum Programming. *Proc. ACM Program. Lang.* 8, OOPSLA1, Article 94 (April 2024), 28 pages. <https://doi.org/10.1145/3649811>

APPENDICES

A FORMAL DEFINITIONS OF TYPING JUDGMENTS AND SEMANTICS

A.1 Syntax

$T ::=$	(data type)	$e ::=$	(expression)
Void	(bottom)	()	(unit)
Unit	(unit)	x	(variable)
$T \oplus T$	(sum)	(e, e)	(pair)
$T \otimes T$	(product)		
$F ::=$	(program type)	$\text{ctrl } e \left\{ \begin{array}{c} e \mapsto e \\ \dots \\ e \mapsto e \end{array} \right\}_T$	(coherent control)
$T \rightsquigarrow T$	(coherent map)		
$T \Rightarrow T$	(quantum channel)	$\text{match } e \left\{ \begin{array}{c} e \mapsto e \\ \dots \\ e \mapsto e \end{array} \right\}_T$	(decoherent match)

Fig. 5. Qunity types.

$\Gamma ::=$	(context)	$f ::=$	(program)
$\emptyset$	(empty)	$u_3(r, r, r)$	(qubit gate)
$\Gamma, x : T$	(binding)	$\text{left}_{T \oplus T}$	(left tag)
$\text{dom}(\emptyset) := \emptyset$	(dom-none)	$\text{right}_{T \oplus T}$	(right tag)
$\text{dom}(\Gamma, x : T) := \text{dom}(\Gamma) \cup \{x\}$	(dom-bind)	$\lambda e \overset{T}{\mapsto} e$	(abstraction)
		$\text{rphase}_T \left\{ \begin{array}{c} e \mapsto r \\ \text{else} \mapsto r \end{array} \right\}$	(relative phase)
		$\text{pmatch} \left\{ \begin{array}{c} e \mapsto e \\ \dots \\ e \mapsto e \end{array} \right\}_{T'}$	(symmetric matching)

Fig. 6. Typing contexts.

Fig. 7. Base Qunity syntax.

Figure 7 shows the base abstract syntax of the Qunity language. Expressions e are assigned types T and programs f are assigned program types F . Here, x ranges over some set of variable names and r ranges over a representation of real numbers. This work adds two new primitive syntactic constructs to the original Qunity language: **match** and **pmatch**, discussed in Section 3.

A.2 Type System

Qunity has two distinct typing judgments for expressions: *pure* expression typing and *mixed* expression typing. These correspond to two distinct but interrelated semantics: a pure semantics that is defined in terms of Kraus operators acting on state vectors and a mixed semantics that is defined in terms of trace non-increasing superoperators acting on density matrices. Typing an expression as pure is useful when the expression is used as a pattern in a lambda or a **ctrl**-block,

because Kraus operators are reversible. The pure expression typing judgment enforces that variables in the quantum context Δ are always *relevant*, so that the expression may be inverted to obtain its context. Furthermore, applying a global phase only makes a semantic difference if the semantics is defined in terms of state vectors, so pure semantics is needed if global phase is used inside of a quantum control block to coherently apply a relative phase. On the other hand, sometimes maintaining reversibility is unnecessary and requires a large amount of tedious bookkeeping. Mixed typing and semantics allow the Qunity programmer to create decoherence by irreversibly discarding quantum information when necessary. Quantum algorithm designers often work with both the statevector and density matrix formalisms, and the design of the Qunity language allows both to be implemented in a convenient way.

$$\begin{array}{c}
\frac{}{\Gamma \parallel \emptyset \vdash \text{Unit} : \text{Unit}} \text{T-UNIT} \quad \frac{}{\Gamma, x : T, \Gamma' \parallel \emptyset \vdash x : T} \text{T-CVAR} \quad \frac{x \notin \text{dom}(\Gamma)}{\Gamma \parallel x : T \vdash x : T} \text{T-QVAR} \\
\\
\frac{\Gamma \parallel \Delta, \Delta_0 \vdash e_0 : T_0 \quad \Gamma \parallel \Delta, \Delta_1 \vdash e_1 : T_1}{\Gamma \parallel \Delta, \Delta_0, \Delta_1 \vdash (e_0, e_1) : T_0 \otimes T_1} \text{T-PUREPAIR} \quad \frac{\vdash f : T \rightsquigarrow T' \quad \Gamma \parallel \Delta \vdash e : T}{\Gamma \parallel \Delta \vdash f e : T'} \text{T-PUREAPP} \\
\\
\frac{\begin{array}{c} \Gamma \parallel \Delta \Vdash e : T \quad \text{ortho}_T(e_1, \dots, e_n) \quad \emptyset \parallel \Gamma_j \vdash e_j : T \text{ for all } j \\ \text{erases}_{T'}(x; e'_1, \dots, e'_n) \text{ for all } x \in \text{dom}(\Delta) \quad \Gamma, \Gamma_j \parallel \Delta, \Delta' \vdash e'_j : T' \text{ for all } j \end{array}}{\Gamma \parallel \Delta, \Delta' \vdash \text{ctrl } e \left\{ \begin{array}{c} e_1 \mapsto e'_1 \\ \vdots \\ e_n \mapsto e'_n \end{array} \right\}_T : T'} \text{T-CTRL}
\end{array}$$

Fig. 8. Pure expression typing rules.

We write $\Gamma \parallel \Delta \vdash e : T$ to indicate that expression e has pure type T with respect to classical context Γ and quantum context Δ . The semantics of such an expression corresponds to a *pure state* in the Hilbert space $\mathcal{H}(T)$ associated with the type T . Figure 8 shows the typing rules for the pure expression typing judgment. While the separation into “classical” and “quantum” contexts may seem unexpected for a language like Qunity that aims to unify classical and quantum computation, the classical contexts do in fact play an important role in the type system. When a variable x is in a classical context Γ , it does not mean that it is literally in a classical register - it means that from the perspective of the current expression, we view it as being in a classical basis state and interact with it by sharing it relative to the classical basis. This also means that the type system does not place the same relevance restrictions on variables in classical contexts as it does on quantum variables: we may assume that the compiler automatically uncomputes the variable when it goes out of scope. The primary use case for these classical contexts is the `ctrl` construct: as seen in the T-CTRL rule, the quantum contexts of the left-hand-side (LHS) pattern expressions become classical contexts when typing the right-hand-side (RHS) expressions. This allows the programmer to safely ignore these variables in the RHS expressions, knowing that when compiling T-CTRL, their uncomputation will be handled automatically.

We write $\Gamma \parallel \Delta \Vdash e : T$ to indicate that expression e has mixed type T with respect to classical context Γ and quantum context Δ . The semantics of such an expression corresponds to a *mixed state* in the Hilbert space $\mathcal{H}(T)$ associated with the type T . Observe that the rule T-MIX allows pure expressions to be typed as mixed.

$$\begin{array}{c}
\frac{\Gamma \parallel \Delta \vdash e : T}{\Gamma \parallel \Delta \Vdash e : T} \text{ T-MIX} \quad \frac{\Gamma \parallel \Delta \vdash e : T}{\Gamma \parallel \Delta, \Delta_0 \Vdash e : T} \text{ T-DISCARD} \\
\\
\frac{\Gamma \parallel \Delta, \Delta_0 \Vdash e_0 : T_0 \quad \Gamma \parallel \Delta, \Delta_1 \Vdash e_1 : T_1}{\Gamma \parallel \Delta, \Delta_0, \Delta_1 \Vdash (e_0, e_1) : T_0 \otimes T_1} \text{ T-MIXEDPAIR} \\
\\
\frac{\Gamma \parallel \Delta_0 \Vdash e_0 : T \quad \Gamma \parallel \Delta_1 \Vdash e_1 : T}{\Gamma \parallel \Delta_0, \Delta_1 \Vdash \text{try } e_0 \text{ catch } e_1 : T} \text{ T-TRY} \quad \frac{\vdash f : T \Rightarrow T' \quad \Gamma \parallel \Delta \Vdash e : T}{\Gamma \parallel \Delta \Vdash f e : T'} \text{ T-MIXEDAPP} \\
\\
\frac{\begin{array}{c} \Gamma \parallel \Delta, \Delta_0 \Vdash e : T \quad \text{ortho}_T(e_1, \dots, e_n) \\ \emptyset \parallel \Gamma_j \vdash e_j : T \text{ for all } j \quad \Gamma, \Gamma_j \parallel \Delta, \Delta_1 \Vdash e'_j : T' \text{ for all } j \end{array}}{\Gamma \parallel \Delta, \Delta_0, \Delta_1 \Vdash \text{match } e \left\{ \begin{array}{c} e_1 \mapsto e'_1 \\ \dots \\ e_n \mapsto e'_n \end{array} \right\}_{T'} : T'} \text{ T-MATCH}
\end{array}$$

Fig. 9. Mixed expression typing rules.

Note that this presentation differs from the original mixed expression typing rules in Voichick et al., which did not include classical contexts since the mixed expression typing judgment has no relevance restrictions on the quantum variables. The primary reason this change was made is for consistency between `ctrl` and the newly introduced `match`: for instance, it allows expressions of the following form to be accepted by the typechecker:

$$\text{match } x \left\{ \begin{array}{c} y \mapsto \text{ctrl } y \text{ }_T \{(\text{expressions that do not erase } y)\}_{T'} \\ \dots \end{array} \right\}_{T'}$$

Here, y is passed into the *classical* context of the `ctrl`, which is initially typed as mixed by the T-MATCH rule, and then as pure through T-MIX rule. Because y is in the classical context of the `ctrl` expression, it is not required to satisfy the erasure judgment in `ctrl` since it only applies to quantum context variables.

The T-DISCARD rule is also a new addition, as Voichick et al. used the T-MIXEDABS program typing rule to make variable discarding possible. This new change is made to more easily deal with discarded variables in the `match` construct.

These rules rely on several additional judgments, namely `ortho`, `ortho*`, and `erases`. These are listed in Appendix B and Appendix C.

A.3 Semantics

Qunity's semantics is defined in terms of operators and superoperators mapping between Hilbert spaces associated with Qunity types and contexts. These Hilbert spaces are defined as follows:

$$\begin{aligned}
\mathcal{H}(\text{Void}) &:= \{0\} \\
\mathcal{H}(\text{Unit}) &:= \mathbb{C} \\
\mathcal{H}(T_0 \oplus T_1) &:= \mathcal{H}(T_0) \oplus \mathcal{H}(T_1) \\
\mathcal{H}(T_0 \otimes T_1) &:= \mathcal{H}(T_0) \otimes \mathcal{H}(T_1) \\
\mathcal{H}(x_1 : T_1, \dots, x_n : T_n) &:= \mathcal{H}(T_1) \otimes \dots \otimes \mathcal{H}(T_n)
\end{aligned}$$

$$\begin{array}{c}
\text{1373} \\
\text{1374} \\
\text{1375} \\
\text{1376} \\
\text{1377} \\
\text{1378} \\
\text{1379} \\
\text{1380} \\
\text{1381} \\
\text{1382} \\
\text{1383} \\
\text{1384} \\
\text{1385} \\
\text{1386} \\
\text{1387} \\
\text{1388} \\
\text{1389} \\
\text{1390} \\
\text{1391} \\
\text{1392} \\
\text{1393} \\
\text{1394} \\
\text{1395} \\
\text{1396} \\
\text{1397} \\
\text{1398} \\
\text{1399} \\
\text{1400} \\
\text{1401} \\
\text{1402} \\
\text{1403} \\
\text{1404} \\
\text{1405} \\
\text{1406} \\
\text{1407} \\
\text{1408} \\
\text{1409} \\
\text{1410} \\
\text{1411} \\
\text{1412} \\
\text{1413} \\
\text{1414} \\
\text{1415} \\
\text{1416} \\
\text{1417} \\
\text{1418} \\
\text{1419} \\
\text{1420} \\
\text{1421}
\end{array}$$

$$\begin{array}{c}
\frac{}{\vdash u_3(r_\theta, r_\phi, r_\lambda) : \text{Bit} \rightsquigarrow \text{Bit}} \text{ T-GATE} \\
\\
\frac{}{\vdash \text{left}_{T_0 \oplus T_1} : T_0 \rightsquigarrow T_0 \oplus T_1} \text{ T-LEFT} \quad \frac{}{\vdash \text{right}_{T_0 \oplus T_1} : T_1 \rightsquigarrow T_0 \oplus T_1} \text{ T-RIGHT} \\
\\
\frac{\emptyset \parallel \Delta \vdash e : T \quad \emptyset \parallel \Delta \vdash e' : T'}{\vdash \lambda e \overset{T}{\mapsto} e' : T \rightsquigarrow T'} \text{ T-PUREABS} \quad \frac{\emptyset \parallel \Delta \vdash e : T}{\vdash \text{rphase}_T \left\{ \begin{array}{l} e \mapsto r \\ \text{else} \mapsto r' \end{array} \right\} : T \rightsquigarrow T} \text{ T-RPHASE} \\
\\
\frac{\vdash f : T \rightsquigarrow T'}{\vdash f : T \Rightarrow T'} \text{ T-CHANNEL} \quad \frac{\emptyset \parallel \Delta \vdash e : T \quad \emptyset \parallel \Delta \Vdash e' : T'}{\vdash \lambda e \overset{T}{\mapsto} e' : T \Rightarrow T'} \text{ T-MIXEDABS} \\
\\
\frac{\text{ortho}_T^*(e_1, \dots, e_n) \quad \text{ortho}_{T'}^*(e'_1, \dots, e'_n) \quad \emptyset \parallel \Delta_j \vdash e_j : T \forall j \quad \emptyset \parallel \Delta_j \vdash e'_j : T' \forall j}{\vdash \text{pmatch} \left\{ \begin{array}{l} e_1 \mapsto e'_1 \\ \dots \\ e_n \mapsto e'_n \end{array} \right\}_T : T \rightsquigarrow T'} \text{ T-PMATCH}
\end{array}$$

Fig. 10. Program typing rules.

For a type T , we define the set of classical expressions of that type as follows:

$$\mathbb{V}(\text{Void}) := \emptyset$$

$$\mathbb{V}(\text{Unit}) := \{\text{Unit}\}$$

$$\mathbb{V}(T_0 \oplus T_1) := \{\text{left}_{T_0 \oplus T_1} v_0 \mid v_0 \in \mathbb{V}(T_0)\} \cup \{\text{right}_{T_0 \oplus T_1} v_1 \mid v_1 \in \mathbb{V}(T_1)\}$$

$$\mathbb{V}(T_0 \otimes T_1) := \{(v_0, v_1) \mid v_0 \in \mathbb{V}(T_0), v_1 \in \mathbb{V}(T_1)\}$$

For a context Δ , we have a set of *valuations*, defined as

$$\mathbb{V}(x_1 : T_1, \dots, x_n : T_n) := \{x_1 \mapsto v_1, \dots, x_n \mapsto v_n \mid v_1 \in \mathbb{V}(T_1), \dots, v_n \in \mathbb{V}(T_n)\}$$

The space $\mathcal{H}(T)$ is spanned by an orthonormal basis $\{|v\rangle : v \in \mathbb{V}(T)\}$, where we define

$$|()\rangle := 1$$

$$|\text{left}_{T_0 \oplus T_1} v_0\rangle := |v_0\rangle \oplus 0$$

$$|\text{right}_{T_0 \oplus T_1} v_1\rangle := 0 \oplus |v_1\rangle$$

$$|(v_0, v_1)\rangle := |v_0\rangle \otimes |v_1\rangle$$

The basis states for contexts Δ are defined by $|\tau\rangle$ for valuations $\tau \in \mathbb{V}(\Delta)$.

$$|x_1 \mapsto v_1, \dots, x_n \mapsto v_n\rangle := |v_1\rangle \otimes \dots \otimes |v_n\rangle$$

Now, we can define the denotational semantics of Qunity. For pure expression semantics, we say that if $\Gamma \parallel \Delta \vdash e : T$ and $\sigma \in \mathbb{V}(\Gamma)$, then $\llbracket \sigma : \Gamma \parallel \Delta \vdash e : T \rrbracket \in \mathcal{L}(\mathcal{H}(\Delta), \mathcal{H}(\Gamma))$ defines the pure semantics of e . Here σ is a valuation of Γ , representing “classical data”. For mixed expression semantics, we have that if $\Gamma \parallel \Delta \Vdash e : T$ and $\sigma \in \mathbb{V}(\Gamma)$, then $\llbracket \sigma : \Gamma \parallel \Delta \Vdash e : T \rrbracket \in \mathcal{L}(\mathcal{L}(\mathcal{H}(\Delta)), \mathcal{L}(\mathcal{H}(\Gamma)))$, defining a superoperator acting on the space of density matrices. For program semantics, we have that $\llbracket \vdash f : T \rightsquigarrow T' \rrbracket \in \mathcal{L}(\mathcal{H}(T), \mathcal{H}(T'))$, and $\llbracket \vdash f : T \Rightarrow T' \rrbracket \in \mathcal{L}(\mathcal{L}(\mathcal{H}(T)), \mathcal{L}(\mathcal{H}(T')))$.

$$\begin{aligned}
& \llbracket \sigma : \Gamma \parallel \emptyset \vdash () : \text{Unit} \rrbracket |\emptyset\rangle := |\text{Unit}\rangle \\
& \llbracket \sigma : \Gamma \parallel \emptyset \vdash x : T \rrbracket |\emptyset\rangle := |\sigma(x)\rangle \\
& \llbracket \sigma : \Gamma \parallel x : T \vdash x : T \rrbracket |x \mapsto v\rangle := |v\rangle \\
& \llbracket \sigma : \Gamma \parallel \Delta, \Delta_0, \Delta_1 \vdash (e_0, e_1) : T_0 \otimes T_1 \rrbracket |\tau, \tau_0, \tau_1\rangle := \llbracket \sigma : \Gamma \parallel \Delta, \Delta_0 \vdash e_0 : T_0 \rrbracket |\tau, \tau_0\rangle \\
& \quad \otimes \llbracket \sigma : \Gamma \parallel \Delta, \Delta_1 \vdash e_1 : T_1 \rrbracket |\tau, \tau_1\rangle \\
& \llbracket \sigma, \sigma' : \Gamma, \Gamma' \parallel \Delta, \Delta' \vdash \text{ctrl } e \left\{ \begin{array}{c} e_1 \mapsto e'_1 \\ \cdots \\ e_n \mapsto e'_n \end{array} \right\}_{T'} : T' \rrbracket |\tau, \tau'\rangle := \sum_{v \in \mathbb{V}(T)} \langle v | \llbracket \Gamma, \Delta \vdash e : T \rrbracket (|\sigma, \tau\rangle \langle \sigma, \tau|) |v\rangle \\
& \quad \cdot \sum_{j=1}^n \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket \emptyset : \emptyset \parallel \Gamma_j \vdash e_j : T \rrbracket^\dagger |v\rangle \\
& \quad \cdot \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel \Delta, \Delta' \vdash e'_j : T' \rrbracket |\tau, \tau'\rangle \\
& \llbracket \sigma : \Gamma \parallel \Delta \vdash f e : T \rrbracket |\tau\rangle := \llbracket \vdash f : T \rightsquigarrow T' \rrbracket \llbracket \sigma : \Gamma \parallel \Delta \vdash e : T \rrbracket |\tau\rangle
\end{aligned}$$

Fig. 11. Pure expression semantics.

$$\begin{aligned}
& \llbracket \vdash f : T \Rightarrow T' \rrbracket (|v\rangle \langle v'|) := \llbracket \vdash f : T \rightsquigarrow T' \rrbracket |v\rangle \langle v'| \llbracket \vdash f : T \rightsquigarrow T' \rrbracket^\dagger \\
& \llbracket \vdash \lambda e \overset{T}{\mapsto} e' : T \Rightarrow T' \rrbracket (|v\rangle \langle v'|) := \\
& \quad \llbracket \emptyset : \emptyset \parallel \Delta \vdash e' : T' \rrbracket \left(\llbracket \emptyset : \emptyset \parallel \Delta \vdash e : T \rrbracket^\dagger |v\rangle \langle v'| \llbracket \emptyset : \emptyset \parallel \Delta \vdash e : T \rrbracket \right)
\end{aligned}$$

Fig. 12. Mixed program semantics.

$$\begin{aligned}
& \llbracket \sigma : \Gamma \parallel \Delta \vdash e : T \rrbracket (|\tau\rangle\langle\tau'|) := \llbracket \sigma : \Gamma \parallel \Delta \vdash e : T \rrbracket (|\tau\rangle\langle\tau'|) \\
& \llbracket \sigma : \Gamma \parallel \Delta, \Delta_0 \vdash e : T \rrbracket (|\tau, \tau_0\rangle\langle\tau', \tau'_0|) := \llbracket \sigma : \Gamma \parallel \Delta \vdash e : T \rrbracket |\tau\rangle\langle\tau'| \llbracket \sigma : \Gamma \parallel \Delta \vdash e : T \rrbracket^\dagger \\
& \llbracket \sigma : \Gamma \parallel \Delta, \Delta_0, \Delta_1 \vdash (e_0, e_1) : T_0 \otimes T_1 \rrbracket (|\tau, \tau_0, \tau_1\rangle\langle\tau', \tau'_0, \tau'_1|) := \llbracket \sigma : \Gamma \parallel \Delta, \Delta_0 \vdash e_0 : T_0 \rrbracket (|\tau, \tau_0\rangle\langle\tau', \tau'_0|) \\
& \quad \otimes \llbracket \sigma : \Gamma \parallel \Delta, \Delta_1 \vdash e_1 : T_1 \rrbracket (|\tau, \tau_1\rangle\langle\tau', \tau'_1|) \\
& \llbracket \sigma : \Gamma \parallel \Delta_0, \Delta_1 \vdash \text{try } e_0 \text{ catch } e_1 : T \rrbracket (|\tau_0, \tau_1\rangle\langle\tau_0, \tau'_1|) := \llbracket \sigma : \Gamma \parallel \Delta_0 \vdash e_0 : T \rrbracket (|\tau_0\rangle\langle\tau_0|) \\
& \quad + (1 - \text{tr}(\llbracket \sigma : \Gamma \parallel \Delta_0 \vdash e_0 : T \rrbracket (|\tau_0\rangle\langle\tau_0|))) \\
& \quad \cdot \llbracket \sigma : \Gamma \parallel \Delta_1 \vdash e_1 : T \rrbracket (|\tau_1\rangle\langle\tau'_1|) \\
& \llbracket \sigma : \Gamma \parallel \Delta_0, \Delta_1 \vdash \text{try } e_0 \text{ catch } e_1 : T \rrbracket (|\tau_0, \tau_1\rangle\langle\tau_0, \tau'_1|) := \llbracket \sigma : \Gamma \parallel \Delta_0 \vdash e_0 : T \rrbracket (|\tau_0\rangle\langle\tau'_0|) \text{ if } \tau_0 \neq \tau'_0 \\
& \llbracket \sigma : \Gamma \parallel \Delta \vdash f e : T' \rrbracket (|\tau\rangle\langle\tau'|) := \llbracket f : T \Rightarrow T' \rrbracket (\llbracket \sigma : \Gamma \parallel \Delta \vdash e : T \rrbracket (|\tau\rangle\langle\tau'|)) \\
& \llbracket \sigma : \Gamma \parallel \Delta, \Delta_0, \Delta_1 \vdash \text{match } e \left\{ \begin{array}{l} e_1 \mapsto e'_1 \\ \cdots \\ e_n \mapsto e'_n \end{array} \right\}_{T'} : T' \rrbracket (|\tau, \tau_0, \tau_1\rangle\langle\tau', \tau'_0, \tau'_1|) := \\
& \quad = \sum_{v \in \mathbb{V}(T)} \langle v | \llbracket \sigma : \Gamma \parallel \Delta, \Delta_0 \vdash e : T \rrbracket (|\tau, \tau_0\rangle\langle\tau', \tau'_0|) | v \rangle \cdot \\
& \quad \cdot \sum_{j=1}^n \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket \emptyset : \emptyset \parallel \Gamma_j \vdash e_j : T \rrbracket^\dagger | v \rangle \cdot \\
& \quad \cdot \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel \Delta, \Delta_1 \vdash e'_j : T' \rrbracket (|\tau, \tau_1\rangle\langle\tau', \tau'_1|)
\end{aligned}$$

Fig. 13. Mixed expression semantics.

$$\begin{aligned}
& \llbracket \vdash u_3(r_\theta, r_\phi, r_\lambda) : \text{Bit} \rightsquigarrow \text{Bit} \rrbracket |\emptyset\rangle := \cos(r_\theta/2) |\emptyset\rangle + e^{ir_\phi} \sin(r_\theta/2) |1\rangle \\
& \llbracket \vdash u_3(r_\theta, r_\phi, r_\lambda) : \text{Bit} \rightsquigarrow \text{Bit} \rrbracket |1\rangle := -e^{ir_\lambda} \sin(r_\theta/2) |\emptyset\rangle + e^{i(r_\phi+r_\lambda)} \cos(r_\theta/2) |1\rangle \\
& \llbracket \vdash \text{left}_{T_0 \oplus T_1} : T_0 \rightsquigarrow T_0 \oplus T_1 \rrbracket |v\rangle := |\text{left}_{T_0 \oplus T_1} v\rangle \\
& \llbracket \vdash \text{right}_{T_0 \oplus T_1} : T_1 \rightsquigarrow T_0 \oplus T_1 \rrbracket |v\rangle := |\text{right}_{T_0 \oplus T_1} v\rangle \\
& \llbracket \vdash \lambda e \xrightarrow{T} e' : T \rightsquigarrow T' \rrbracket |v\rangle := \llbracket \emptyset : \emptyset \parallel \Delta \vdash e' : T' \rrbracket \llbracket \emptyset : \emptyset \parallel \Delta \vdash e : T \rrbracket^\dagger |v\rangle \\
& \llbracket \vdash \text{rphase}_T \left\{ \begin{array}{l} e \mapsto r \\ \text{else} \mapsto r' \end{array} \right\} : T \rightsquigarrow T \rrbracket |v\rangle := e^{ir} \llbracket \emptyset : \emptyset \parallel \Delta \vdash e : T \rrbracket \llbracket \emptyset : \emptyset \parallel \Delta \vdash e : T \rrbracket^\dagger |v\rangle \\
& \quad + e^{ir'} \left(\mathbb{I} - \llbracket \emptyset : \emptyset \parallel \Delta \vdash e : T \rrbracket \llbracket \emptyset : \emptyset \parallel \Delta \vdash e : T \rrbracket^\dagger \right) |v\rangle \\
& \llbracket \vdash \text{pmatch} \left\{ \begin{array}{l} e_1 \mapsto e'_1 \\ \cdots \\ e_n \mapsto e'_n \end{array} \right\}_T : T \rightsquigarrow T' \rrbracket |v\rangle := \sum_{j=1}^n \llbracket \emptyset : \emptyset \parallel \Delta_j \vdash e'_j : T' \rrbracket \llbracket \emptyset : \emptyset \parallel \Delta_j \vdash e_j : T \rrbracket^\dagger |v\rangle
\end{aligned}$$

Fig. 14. Pure program semantics.

B THE ERASURE JUDGMENT

$$\begin{array}{c}
\frac{\text{erases}_T(x; e_1, \dots, e_n)}{\text{erases}_T(x; e_1, \dots, e_{j-1}, e_j \triangleright \text{gphase}_T(r), e_{j+1}, \dots, e_n)} \quad \text{E-GPHASE} \\
\\
\frac{\text{erases}_T(x; e_1, \dots, e_{j-1}, e_{j,1}, \dots, e_{j,m}, e_{j+1}, \dots, e_n)}{\text{erases}_T \left(x; e_1, \dots, e_{j-1}, \text{ctrl } e \begin{array}{c} \left\{ \begin{array}{l} e'_1 \mapsto e_{j,1} \\ \dots \\ e'_m \mapsto e_{j,m} \end{array} \right\}_{T'} \end{array}, e_{j+1}, \dots, e_n \right)} \quad \text{E-CTRL} \\
\\
\frac{}{\text{erases}_T(x; x, x, \dots, x)} \quad \text{E-VAR} \quad \frac{\text{erases}_{T_0}(x; e_{0,1}, \dots, e_{0,n})}{\text{erases}_{T_0 \otimes T_1}(x; (e_{0,1}, e_{1,1}), \dots, (e_{0,n}, e_{1,n}))} \quad \text{E-PAIR0} \\
\\
\frac{\text{erases}_{T_1}(x; e_{1,1}, \dots, e_{1,n})}{\text{erases}_{T_0 \otimes T_1}(x; (e_{0,1}, e_{1,1}), \dots, (e_{0,n}, e_{1,n}))} \quad \text{E-PAIR1}
\end{array}$$

Fig. 15. Erasure inference rules.

C EXTENDING THE SPANNING AND ORTHOGONALITY JUDGMENTS

We will define the ortho^* judgment mentioned in Section 3.1, and the spanning^* judgment it depends on.

$$\begin{array}{c}
\frac{}{\text{spanning}_{\text{Void}}^*()} \quad \text{S}^*\text{-VOID} \quad \frac{}{\text{spanning}_{\text{Unit}}^*(\text{Unit})} \quad \text{S}^*\text{-UNIT} \quad \frac{}{\text{spanning}_T^*(x)} \quad \text{S}^*\text{-VAR} \\
\\
\frac{\text{spanning}_T^*(e_1, \dots, e_n) \quad \text{un}(f)}{\text{spanning}_T^*(f e_1, \dots, f e_n)} \quad \text{S}^*\text{-UNAPP} \\
\\
\frac{\text{spanning}_{T_0}^*(e_1, \dots, e_n) \quad \text{spanning}_{T_1}^*(e'_1, \dots, e'_{n'})}{\text{spanning}_{T_0 \oplus T_1}^* \left(\begin{array}{c} \text{left}_{T_0 \oplus T_1} e_1, \dots, \text{left}_{T_0 \oplus T_1} e_n, \\ \text{right}_{T_0 \oplus T_1} e'_1, \dots, \text{right}_{T_0 \oplus T_1} e'_{n'} \end{array} \right)} \quad \text{S}^*\text{-SUM} \\
\\
\frac{\text{spanning}_{T_0}^*(e_1, \dots, e_m) \quad \text{spanning}_{T_1}^*(e'_{j,1}, \dots, e'_{j,n_j}) \text{ for all } j}{\text{FV}(e_j) \cap \bigcup_{k=1}^{n_m} \text{FV}(e'_{j,k}) = \emptyset \text{ for all } j} \quad \text{S}^*\text{-PAIR} \\
\\
\text{spanning}_{T_0 \otimes T_1}^* \left(\begin{array}{c} (e_1, e'_{1,1}), \dots, (e_1, e'_{1,n_1}), \\ \dots, \\ (e_m, e'_{m,1}), \dots, (e_m, e'_{m,n_m}) \end{array} \right)
\end{array}$$

Fig. 16. Spanning inference rules, expanded using the unitary judgment to create the spanning^* judgment.

This is an extension of the spanning judgment, which differs only in that it does not have the $\text{S}^*\text{-UNAPP}$ rule. This rule depends on the *unitary judgment*, which is defined as follows:

$$\frac{\text{iso}(f) \quad \dim(T) = \dim(T')}{\text{un}(\vdash f : T \rightsquigarrow T')} \quad \text{U-PROG}$$

The ortho^* judgment can now simply be defined as

$$\frac{\text{spanning}_T^*(e'_1, \dots, e'_m) \quad [e_1, \dots, e_n] \text{ is a subsequence of } [e'_1, \dots, e'_m]}{\text{ortho}_T^*(e_1, \dots, e_n)}$$

D THE ISOMETRY JUDGMENT

$$\begin{array}{c}
\frac{}{\text{iso}(())} \quad \text{I-UNIT} \quad \frac{}{\text{iso}(x)} \quad \text{I-VAR} \quad \frac{\text{iso}(e_0) \quad \text{iso}(e_1)}{\text{iso}((e_0, e_1))} \quad \text{I-PAIR} \\
\\
\frac{\text{classical}(e) \quad \text{iso}(e) \quad \text{spanning}_T(e_1, \dots, e_n) \quad \text{iso}(e'_1) \quad \dots \quad \text{iso}(e'_n)}{\text{iso}\left(\text{ctrl } e \left\{ \begin{array}{c} e_1 \mapsto e'_1 \\ \dots \\ e_n \mapsto e'_n \end{array} \right\}_T\right)} \quad \text{I-CTRL} \\
\\
\frac{\text{classical}(e) \quad \text{iso}(e) \quad \text{spanning}_T(e_1, \dots, e_n) \quad \text{iso}(e'_1) \quad \dots \quad \text{iso}(e'_n)}{\text{iso}\left(\text{match } e \left\{ \begin{array}{c} e_1 \mapsto e'_1 \\ \dots \\ e_n \mapsto e'_n \end{array} \right\}_T\right)} \quad \text{I-MATCH} \\
\\
\frac{\text{iso}(e)}{\text{iso}(\text{try } e \text{ catch } e')} \quad \text{I-TRY} \quad \frac{\text{iso}(e')}{\text{iso}(\text{try } e \text{ catch } e')} \quad \text{I-CATCH} \quad \frac{\text{iso}(f) \quad \text{iso}(e)}{\text{iso}(f e)} \quad \text{I-APP} \\
\\
\frac{}{\text{iso}(u_3(r_\theta, r_\phi, r_\lambda))} \quad \text{I-GATE} \quad \frac{}{\text{iso}(\text{left}_{T_0 \oplus T_1})} \quad \text{I-LEFT} \quad \frac{}{\text{iso}(\text{right}_{T_0 \oplus T_1})} \quad \text{I-RIGHT} \\
\\
\frac{\text{spanning}_T(e) \quad \text{iso}(e')}{\text{iso}(\lambda e \stackrel{T}{\mapsto} e')} \quad \text{I-ABS} \quad \frac{}{\text{iso}\left(\text{rphase}_T \left\{ \begin{array}{c} e \mapsto r \\ \text{else} \mapsto r' \end{array} \right\}\right)} \quad \text{I-RPHASE} \\
\\
\frac{\text{spanning}_T^*(e_1, \dots, e_n) \quad \text{spanning}_T^*(e'_1, \dots, e'_n)}{\text{iso}\left(\text{pmatch} \left\{ \begin{array}{c} e_1 \mapsto e'_1 \\ \dots \\ e_n \mapsto e'_n \end{array} \right\}_T\right)} \quad \text{I-PMATCH}
\end{array}$$

Fig. 17. Isometry inference rules.

E GRAMMAR OF THE SURFACE SYNTAX

```

1618 E GRAMMAR OF THE SURFACE SYNTAX
1619  $\langle qfile \rangle ::= \langle def \rangle^* \langle e \rangle$ 
1620
1621  $\langle def \rangle ::=$  type  $\langle tname \rangle \langle sig \rangle := \langle t \rangle$  end
1622 | type  $\langle tname \rangle \langle sig \rangle := (| \langle tname \rangle | \langle fname \rangle \text{ of } \langle t \rangle)^+ \text{ end}$ 
1623 | def  $\langle tname \rangle \langle sig \rangle : \langle t \rangle := \langle e \rangle$  end
1624 | def  $\langle fname \rangle \langle sig \rangle : \langle t \rangle \rightarrow \langle t \rangle := \langle f \rangle$  end
1625 | def  $\langle rname \rangle \langle sig \rangle := \langle r \rangle$  end
1626
1627  $\langle sig \rangle ::= \{ \langle param \rangle (, \langle param \rangle)^* \}$ 
1628
1629  $\langle param \rangle ::= \langle tvar \rangle$ 
1630 |  $\langle tname \rangle : \langle t \rangle$ 
1631 |  $\langle fname \rangle : \langle t \rangle \rightarrow \langle t \rangle$ 
1632 |  $\langle rname \rangle$ 
1633
1634  $\langle ge \rangle ::= \langle t \rangle | \langle e \rangle | \langle f \rangle | \langle r \rangle$ 
1635
1636  $\langle t \rangle ::=$  Void
1637 | Unit
1638 |  $\langle t \rangle^* \langle t \rangle$ 
1639 |  $\langle tvar \rangle$ 
1640 |  $\langle tname \rangle (\{ \langle ge \rangle, (, \langle ge \rangle)^* \})?$ 
1641 | if  $\langle be \rangle$  then  $\langle t \rangle$  else  $\langle t \rangle$  endif
1642
1643  $\langle e \rangle ::= ()$ 
1644 |  $\langle qvar \rangle$ 
1645 |  $(\langle e \rangle, \langle e \rangle)$ 
1646 | ctrl  $\langle e \rangle [ (\langle e \rangle \rightarrow \langle e \rangle)^* (\text{else} \rightarrow \langle e \rangle)? ]$ 
1647 | match  $\langle e \rangle [ (\langle e \rangle \rightarrow \langle e \rangle)^* (\text{else} \rightarrow \langle e \rangle)? ]$ 
1648 | try  $\langle e \rangle$  catch  $\langle e \rangle$ 
1649 |  $\langle f \rangle (\langle e \rangle)$ 
1650 |  $\langle tname \rangle (\{ \langle ge \rangle, (, \langle ge \rangle)^* \})?$ 
1651 | if  $\langle be \rangle$  then  $\langle e \rangle$  else  $\langle e \rangle$  endif
1652
1653  $\langle f \rangle ::=$  u3  $\{ \langle r \rangle, \langle r \rangle, \langle r \rangle \}$ 
1654 | lambda  $\langle e \rangle \rightarrow \langle e \rangle$ 
1655 | gphase  $\{ \langle r \rangle \}$ 
1656 | rphase  $\{ \langle e \rangle, \langle r \rangle, \langle r \rangle \}$ 
1657 | pmatch  $[ (\langle e \rangle \rightarrow \langle e \rangle)^* ]$ 
1658 |  $\langle fname \rangle (\{ \langle ge \rangle, (, \langle ge \rangle)^* \})?$ 
1659 | if  $\langle be \rangle$  then  $\langle f \rangle$  else  $\langle f \rangle$  endif
1660
1661  $\langle r \rangle ::=$  pi
1662 | euler
1663 |  $\langle const \rangle$ 
1664 |  $- \langle r \rangle$ 
1665 |  $\langle r \rangle + \langle r \rangle$ 
1666 | ... (other mathematical functions)
1667 |  $\langle rname \rangle (\{ \langle ge \rangle, (, \langle ge \rangle)^* \})?$ 
1668 | if  $\langle be \rangle$  then  $\langle r \rangle$  else  $\langle r \rangle$  endif

```

F COMPARISON OF CONTROL FLOW CONSTRUCTS

	ctrl	match	pmatch
What kind of semantics?	Pure expression	Mixed expression	Pure program
Restrictions	• LHS must satisfy orthogonality • LHS must be pure • RHS must be pure • RHS must satisfy erasure judgment for scrutinee quantum context • LHS must be classical	• LHS must satisfy orthogonality • LHS must be pure • LHS must be classical	• LHS must satisfy orthogonality • RHS must satisfy orthogonality • LHS must be pure • RHS must be pure • Each pair of matching expressions must share the same (quantum) context • Since it is a program, it is typed without context and does not contain a scrutinee directly
Benefits	• Scrutinee can be mixed while the entire expression is still pure • Can use classical context variables originating from an outer ctrl or match in an unrestricted way • Can be used to apply relative phases (conditional global phases)	• RHS can be mixed • Erasure judgment is not required • Can use classical context variables originating from an outer ctrl or match in an unrestricted way • No “unavoidable source of error” when scrutinee is not classical	• LHS and RHS can be in an arbitrary basis • Erasure judgment is not required • Can be used to apply relative phases (conditional global phases)
Can be used in a pattern / has an adjoint	Yes	No	Yes
Can be used in the LHS of a ctrl or match	No	No	No
Can be used in the RHS of a ctrl	Yes	No	Yes

Can be used the RHS of a match	Yes	Yes	Yes
Can be used in the LHS or RHS of a pmatch	Yes, if the unitary judgment holds	No	Yes, if the unitary judgment holds
Can contain a mixed expression in the scrutinee	Yes	Yes	N/A
Can contain a mixed expression on the LHS	No	No	No
Can contain a mixed expression on the RHS	No	Yes	No
Applying a global phase to a RHS expression makes a semantic difference	Yes	No	Yes

When to use?	• You want to control something on a subroutine which may involve measurement/de-coherence, but you want your entire expression to be reversible • You want to conditionally apply a global phase depending on the result of a mixed expression • You are able to keep the scrutinee variables on the RHS in satisfaction of the erasure judgment	• You do not care whether your expression is reversible • You want to conditionally apply some expressions that involve measurement/de-coherence • You want something that most closely corresponds to a <code>match</code> in classical programming languages • What you want to do can be accomplished by measuring the outcome of the input expression and then evaluating the appropriate RHS expression • You want to avoid the erasure requirement	• You want to reversibly map between two different bases • You want to avoid the erasure requirement • You do not need any outside variables • You want to create a unitary map to use inside other <code>pmatch</code> constructs
--------------	---	--	--

G DIRECT SUM CIRCUIT CORRECTNESS PROOF

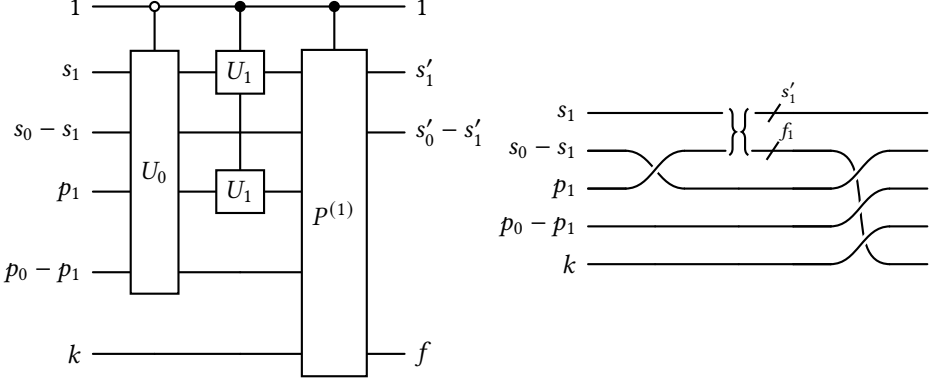


Fig. 18. Improved direct sum circuit for Case 1: $s_0 \geq s_1, s'_0 \geq s'_1, p_0 \geq p_1$. For this circuit, let $k = \max\{0, f_1 - f_0\}$, and let $f = \max\{f_0, f_1\} = f_0 + k$. The circuit on the left implements the direct sum, while the one on the right is the implementation of the component $P^{(1)}$. Note that where U_1 is drawn as two separate boxes, these are not two separate gates, but rather a single gate that only acts on the s_1 and p_1 registers and not on the one in between them.

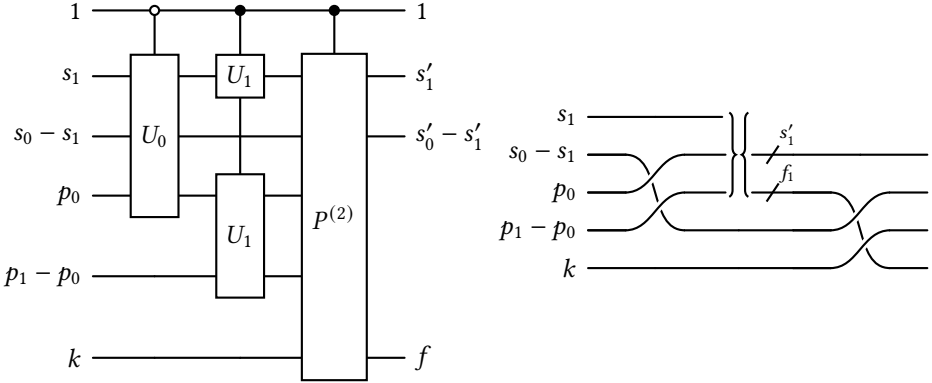


Fig. 19. Improved direct sum circuit for Case 2: $s_0 \geq s_1, s'_0 \geq s'_1, p_0 \leq p_1$. For this circuit, let $k = \max\{0, f_1 - f_0 + p_0 - p_1\}$, and let $f = \max\{f_0 + p_1 - p_0, f_1\}$. The circuit on the left implements the direct sum, while the one on the right is the implementation of the component $P^{(2)}$.

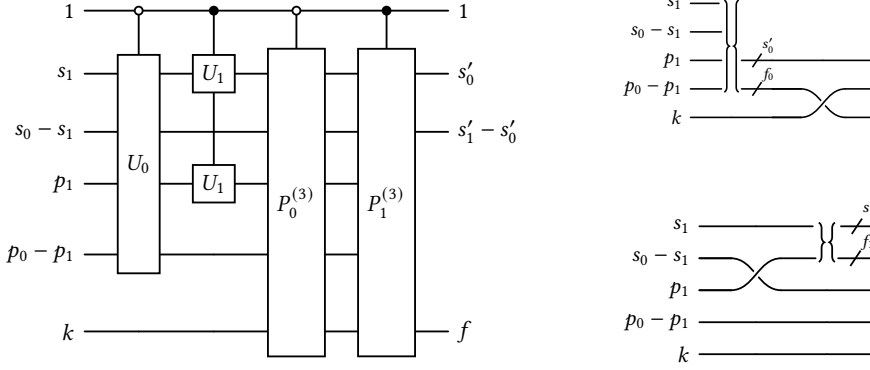


Fig. 20. Improved direct sum circuit for Case 3: $s_0 \geq s_1, s'_0 \leq s'_1, p_0 \geq p_1$. For this circuit, let $k = s'_1 - s'_0$, and let $f = f_0$. The circuit on the left implements the direct sum, while the ones on the right are the implementation of the components $P_0^{(3)}$ and $P_1^{(3)}$.

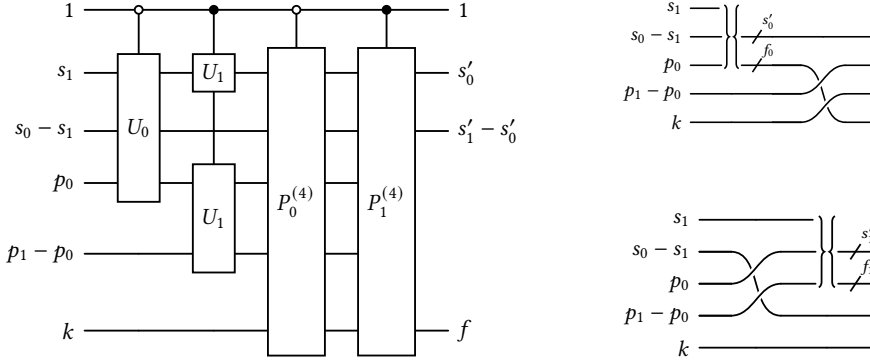


Fig. 21. Improved direct sum circuit for Case 4: $s_0 \geq s_1, s'_0 \leq s'_1, p_0 \leq p_1$. For this circuit, let $k = \max\{0, s'_1 - s'_0 + p_0 - p_1\}$, and let $f = s_0 + p_1 - s'_1 + k$. The circuit on the left implements the direct sum, while the ones on the right are the implementation of the components $P_0^{(4)}$ and $P_1^{(4)}$.

LEMMA G.1. Suppose it is possible to implement (as specified in Definition 4.2) the operators $E_0 : \mathcal{H}(T_0) \rightarrow \mathcal{H}(T'_0)$ and $E_1 : \mathcal{H}(T_1) \rightarrow \mathcal{H}(T'_1)$. Then it is possible to implement the operator $E_0 \oplus E_1 : \mathcal{H}(T_0 \oplus T_1) \rightarrow \mathcal{H}(T'_0 \oplus T'_1)$, using the circuits shown in Figures 18, 19, 20, and 21.

PROOF. This is a partial proof of the above lemma, only demonstrating the argument for Case 1 (Figure 18). The analysis of the other cases is similar.

Let $s_0 = \text{size}(T_0)$, $s'_0 = \text{size}(T'_0)$, $s_1 = \text{size}(T_1)$, $s'_1 = \text{size}(T'_1)$. Suppose that E_0 is implemented by a unitary U_0 using p_0 prep qubits and p_1 flag qubits, and E_1 is implemented by a unitary U_1 using p_1 prep qubits and f_1 flag qubits. We have that

$$s_0 + p_0 = s'_0 + f_0$$

$$s_1 + p_1 = s'_1 + f_1$$

Define $\Pi_{s_0} \in \mathcal{L}(\mathbb{C}^{2^{s_0+p_0}}, \mathbb{C}^{2^{s_0}})$, $\Pi_{p_0} \in \mathcal{L}(\mathbb{C}^{2^{s_0+p_0}}, \mathbb{C}^{2^{p_0}})$, $\Pi_{s'_0} \in \mathcal{L}(\mathbb{C}^{2^{s'_0+f_0}}, \mathbb{C}^{2^{s'_0}})$, $\Pi_{f_0} \in \mathcal{L}(\mathbb{C}^{2^{s'_0+f_0}}, \mathbb{C}^{2^{f_0}})$ to be projectors onto the input, prep, output, and flag subspaces, of U_0 respectively, and similarly for $\Pi_{s_1}, \Pi_{p_1}, \Pi_{s'_1}, \Pi_{f_1}$.

Now, without loss of generality, we can assume that $s_0 \geq s_1$, because if this were not the case, then we can implement $E_1 \oplus E_0$ and apply the direct sum's commutativity isomorphism by simply surrounding the circuit with Pauli X gates on the first "signal" qubit. Now, we have 4 cases to analyze. Here, will only demonstrate the analysis of Case 1, where $s_0 \geq s_1, s'_0 \geq s'_1, p_0 \geq p_1$.

For this case: let $k = \max\{0, f_1 - f_0\}$. Let $f = \max\{f_0, f_1\} = f_0 + k$. Then, consider the circuit U shown in Figure 18.

First, to verify that the circuit is valid and the wire counts are correct:

$$1 + s_0 + p_0 + \max\{0, f_1 - f_0\} = 1 + s'_0 + f_0 + \max\{0, f_1 - f_0\} = 1 + s'_0 + \max\{f_0, f_1\}$$

Now, to verify the correctness of the circuit:

$$\langle 0, \text{enc}(v'_0), 0 | U | 0, \text{enc}(v_0), 0, 0 \rangle = \langle \text{enc}(v'_0), 0 | U_0 | \text{enc}(v_0), 0 \rangle = \langle v'_0 | E_0 | v_0 \rangle = (\langle v'_0 | \oplus 0)(E_0 \oplus E_1)(|v_0 \rangle \oplus 0)$$

$$\langle 1, \text{enc}(v'_1), 0, 0 | U | 0, \text{enc}(v_0), 0, 0 \rangle = 0 = (0 \oplus \langle v'_1 |)(E_0 \oplus E_1)(|v_0 \rangle \oplus 0)$$

$$\langle 0, \text{enc}(v'_0), 0 | U | 1, \text{enc}(v_1), 0, 0, 0, 0 \rangle = 0 = (\langle v'_0 | \oplus 0)(E_0 \oplus E_1)(0 \oplus |v_1 \rangle)$$

$$\begin{aligned} & \langle 1, \text{enc}(v'_1), 0, 0 | U | 1, \text{enc}(v_1), 0, 0, 0, 0 \rangle = \\ &= \langle \text{enc}(v'_1), 0, 0 | P^{(1)} (\Pi_{s_1} U_1 | \text{enc}(v_1), 0 \rangle \otimes |0 \rangle \otimes \Pi_{p_1} U_1 | \text{enc}(v_1), 0 \rangle \otimes |0, 0 \rangle) = \\ &= \langle \text{enc}(v'_1), 0, 0 | (\Pi_{s'_1} U_1 | \text{enc}(v_1), 0 \rangle \otimes |0, 0, 0 \rangle \otimes \Pi_{f_1} U_1 | \text{enc}(v_1), 0 \rangle) = \langle \text{enc}(v'_1), 0 | U_1 | \text{enc}(v_1), 0 \rangle = \\ &= \langle v'_1 | E_1 | v_1 \rangle = (0 \oplus \langle v'_1 |)(E_0 \oplus E_1)(0 \oplus |v_1 \rangle) \end{aligned}$$

And now, to verify the preservation of encoding validity:

$$U | 0, \text{enc}(v), 0^{\otimes p_0}, 0^{\otimes k} \rangle = |0 \rangle \otimes U_0 | \text{enc}(v), 0^{\otimes p_0} \rangle \otimes |0^{\otimes k} \rangle.$$

By the hypothesis, we have that

$$U_0 | \text{enc}(v), 0^{\otimes p_0} \rangle \in \mathbb{W}(T'_0) \otimes \mathbb{C}^{2^{f_0}},$$

which means that

$$U | 0, \text{enc}(v), 0^{\otimes p_0}, 0^{\otimes k} \rangle \in \mathbb{W}(T'_0 \oplus T'_1) \otimes \mathbb{C}^{2^f}.$$

And for the adjoint,

$$U^\dagger \left| 0, \text{enc}(v'), 0^{\otimes f} \right\rangle = |0\rangle \otimes U_0^\dagger \left| \text{enc}(v'), 0^{\otimes f_0} \right\rangle \otimes \left| 0^{\otimes k} \right\rangle.$$

By the hypothesis, we have that

$$U_0^\dagger \left| \text{enc}(v'), 0^{\otimes f_0} \right\rangle \in \mathbb{W}(T_0) \otimes \mathbb{C}^{2^{p_0}},$$

which means that

$$U^\dagger \left| 0, \text{enc}(v'), 0^{\otimes f} \right\rangle \in \mathbb{W}(T_0 \oplus T_1) \otimes \mathbb{C}^{2^{p_0+k}}.$$

And, for the T_1 side:

$$\begin{aligned} U \left| 1, \text{enc}(v), 0^{\otimes(s_0-s_1)}, 0^{\otimes p_1}, 0^{\otimes(p_0-p_1)}, 0^{\otimes k} \right\rangle &= \\ &= P^{(1)} \left(|1\rangle \otimes \Pi_{s'_1} U_1 \left| \text{enc}(v), 0^{\otimes p_1} \right\rangle \otimes \left| 0^{\otimes(s_0-s_1)} \right\rangle \otimes \Pi_{f_1} U_1 \left| \text{enc}(v), 0^{\otimes p_1} \right\rangle \otimes \left| 0^{\otimes(p_0-p_1)+k} \right\rangle \right) = \\ &= |1\rangle \otimes \Pi_{s'_1} U_1 \left| \text{enc}(v), 0^{\otimes p_1} \right\rangle \otimes \left| 0^{\otimes(s_0-s_1+p_0-p_1+k)} \right\rangle \otimes \Pi_{f_1} U_1 \left| \text{enc}(v), 0^{\otimes p_1} \right\rangle. \end{aligned}$$

Since by the hypothesis, we have that

$$U_1 \left| \text{enc}(v), 0^{\otimes p_1} \right\rangle \in \mathbb{W}(T'_1) \otimes \mathbb{C}^{2^{f_1}},$$

meaning that

$$\Pi_{s'_1} U_1 \left| \text{enc}(v), 0^{\otimes p_1} \right\rangle \in \mathbb{W}(T'_1),$$

and we also have

$$s_0 - s_1 + p_0 - p_1 + k + f_1 = s'_0 + f_0 - s'_1 - f_1 + k + f_1 = s'_0 - s'_1 + f,$$

This means that

$$U \left| 1, \text{enc}(v), 0^{\otimes(s_0-s_1)}, 0^{\otimes p_1}, 0^{\otimes(p_0-p_1)}, 0^{\otimes k} \right\rangle \in \mathbb{W}(T'_0 \oplus T'_1) \otimes \mathbb{C}^{2^f}.$$

And for the adjoint,

$$\begin{aligned} U^\dagger \left| 1, \text{enc}(v'), 0^{\otimes(s'_0-s'_1)}, 0^{\otimes f} \right\rangle &= |1\rangle \otimes (U_1 \otimes \mathbb{I})^\dagger (P^{(1)})^\dagger \left| \text{enc}(v'), 0^{\otimes(s'_0-s'_1)}, 0^{\otimes f} \right\rangle = \\ &= |1\rangle \otimes (U_1 \otimes \mathbb{I})^\dagger \left(\Pi_{s_1} \left| \text{enc}(v'), 0^{\otimes f_1} \right\rangle \otimes \left| 0^{\otimes(s_0-s_1)} \right\rangle \otimes \Pi_{p_1} \left| \text{enc}(v'), 0^{\otimes f_1} \right\rangle \otimes \left| 0^{\otimes(p_0-p_1)+k} \right\rangle \right). \end{aligned}$$

Since by the hypothesis, we have that

$$U_1^\dagger \left| \text{enc}(v'), 0^{\otimes f_1} \right\rangle \in \mathbb{W}(T_1) \otimes \mathbb{C}^{2^{p_1}},$$

meaning that

$$\Pi_{s_1} U_1^\dagger \left| \text{enc}(v'), 0^{\otimes f_1} \right\rangle \in \mathbb{W}(T_1),$$

and so

$$U^\dagger \left| 1, \text{enc}(v'), 0^{\otimes(s'_0-s'_1)}, 0^{\otimes f} \right\rangle \in \mathbb{W}(T_0 \oplus T_1) \otimes \mathbb{C}^{2^{p_0+k}},$$

which completes the proof for Case 1.

□

H NOTATION AND DEFINITIONS FOR BINARY TREES

First, we can define a binary tree $\mathcal{R}$ as either Leaf or $(\mathcal{R}_0, \mathcal{R}_1)$, where $\mathcal{R}_0$ and $\mathcal{R}_1$ are binary trees, the left and right children of $\mathcal{R}$.

We can define size and height functions as:

Definition H.1.

$$\begin{aligned} \text{size}(\text{Leaf}) &:= 1 \\ \text{size}((\mathcal{R}_0, \mathcal{R}_1)) &:= \text{size}(\mathcal{R}_0) + \text{size}(\mathcal{R}_1) \\ \text{height}(\text{Leaf}) &:= 0 \\ \text{height}((\mathcal{R}_0, \mathcal{R}_1)) &:= 1 + \max(\text{height}(\mathcal{R}_0), \text{height}(\mathcal{R}_1)) \end{aligned}$$

Then, we can define the following:

Definition H.2 (Sum of operators along a binary tree).

$$\begin{aligned} \bigoplus_{j:\text{Leaf}} E_j &:= E_1 \\ \bigoplus_{j:(\mathcal{R}_0, \mathcal{R}_1)} E_j &:= \left(\bigoplus_{j:\mathcal{R}_0} E_j \right) \oplus \left(\bigoplus_{j:\mathcal{R}_1} E_{j+\text{size}(\mathcal{R}_0)} \right) \end{aligned}$$

Definition H.3 (Indexed injection into a tree).

$$\begin{aligned} \text{inj}_1^{\text{Leaf}} |v\rangle &= |v\rangle \\ \text{inj}_k^{(\mathcal{R}_0, \mathcal{R}_1)} |v\rangle &= \begin{cases} \text{inj}_k^{\mathcal{R}_0} |v\rangle \oplus 0^{\text{size}(\mathcal{R}_1)} & \text{if } 1 \leq k \leq \text{size}(\mathcal{R}_1) \\ 0^{\text{size}(\mathcal{R}_1)} \oplus \text{inj}_{k-\text{size}(\mathcal{R}_0)}^{\mathcal{R}_1} |v\rangle & \text{otherwise} \end{cases} \end{aligned}$$

Definition H.4 (Sum of operators along a binary tree, leveled to height h).

$$\begin{aligned} \bigoplus_{j:L(\text{Leaf}, 0)} E_j &:= E_1 \\ \bigoplus_{j:L(\text{Leaf}, h)} E_j &:= \left(\bigoplus_{j:L(\text{Leaf}, h-1)} E_j \right) \oplus \mathbb{I} \quad (\text{where } h \in \mathbb{N}, h > 0) \\ \bigoplus_{j:L((\mathcal{R}_0, \mathcal{R}_1), h)} E_j &:= \left(\bigoplus_{j:L(\mathcal{R}_0, h-1)} E_j \right) \oplus \left(\bigoplus_{j:L(\mathcal{R}_1, h-1)} E_{\text{size}(\mathcal{R}_0)} \right) \end{aligned}$$

As a shorthand, we can write

$$\bigoplus_{j:L(\mathcal{R})} E_j := \bigoplus_{j:L(\mathcal{R}, \text{height}(\mathcal{R}))} E_j.$$

Summing along a leveled tree, as in Definition H.4, can be thought of as adding left children to all nodes in the tree, until they reach a specified height (which must be at least the height of the tree). This construction will be useful when constructing the new spanning circuit.

I CIRCUIT-STYLE PROGRAMMING

While Qunity allows for programming on a much higher level of abstraction than the quantum circuit model, it is possible to work with Qunity as if it was a low-level circuit language, by defining the following constructs:

```

2059
2060
2061
2062
2063
2064
2065 def @gate_1q{#n, #i, @f : Bit → Bit} : Array{#n, Bit} → Array{
2066   #n, Bit} :=
2067   if #i <= 0 then
2068     lambda (x, y) → (@f(x), y)
2069   else
2070     lambda (x, y) → (x, @gate_1q{#n - 1, #i - 1, @f}(y))
2071   endif
2072 end
2073
2074
2075 def @controlled_1q{#n, #i, #j, @f : Bit → Bit} : Array{#n, Bit}
2076   → Array{#n, Bit} :=
2077   if #i > #j then
2078     lambda x → x
2079     ▷ @reverse{#n, Bit}
2080     ▷ @controlled_1q{#n, #n - 1 - #i, #n - 1 - #j, @f}
2081     ▷ @reverse{#n, Bit}
2082   else
2083     if #i <= 0 then
2084       lambda (x, y) → ctrl x [
2085         $0 → (x, y);
2086         $1 → (x, @gate_1q{#n - 1, #j - 1, @f}(y))
2087       ]
2088     else
2089       lambda (x, y) → (x, @cnot{#n - 1, #i - 1, #j - 1}(y))
2090     )
2091   endif
2092 endif
2093 end
2094
2095
2096
2097 def @cnot{#n, #i, #j} : Array{#n, Bit} → Array{#n, Bit} :=
2098   @controlled_1q{#n, #i, #j, @not}
2099 end
2100
2101
2102
2103
2104
2105
2106
2107

```

This allows us to use a Qunity array as a quantum register and apply gates to it, accessing qubits by index. Since we have defined single-qubit gates and CNOT in this way, this can in principle be used to represent any quantum computation. If performance is critical, a Qunity programmer may switch to writing in circuit style for some subroutines, since these constructs compile efficiently to the corresponding low-level gates.

J BENCHMARK TEST PROGRAMS

For these examples, the quantum registers in all circuits were initialized to a superposition state to prevent classical propagation optimizations from taking place.

J.1 Phase Conditioned on AND of 5 Qubits

Qunity implementation:

```
def @multi_and{#n} : Array{#n, Bit} → Bit :=
  if #n = 0 then
    lambda () → $1
  else
    lambda (x0, x1) → @and(x0, @multi_and{#n - 1}(x1))
  endif
end

$repeated{5, Bit, $plus} ▷ lambda x → ctrl @multi_and{5}(x) [
  $0 → x;
  $1 → x ▷ gphase{pi}
]
```

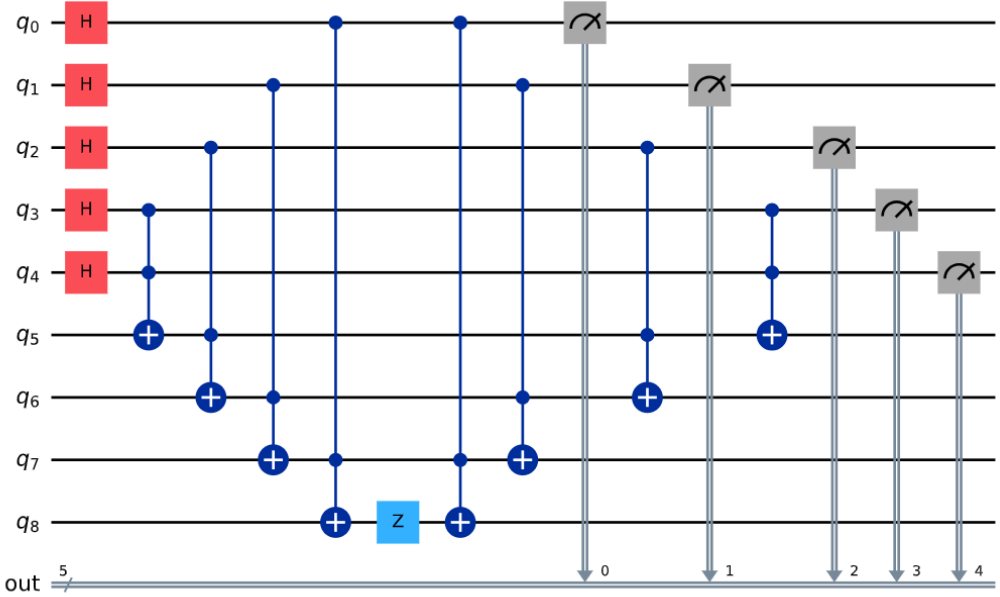


Fig. 22. Compiled circuit for phase conditioned on AND of 5 qubits.

Qiskit implementation:

```
circ_and5 = QuantumCircuit(5, 5)
circ_and5.h(range(5))
```



```

2157 circ_and5.mcp(np.pi, [0,1,2,3], [4])
2158 circ_and5.measure(range(5), range(5))
2159

```

2160 J.2 Quantum Fourier Transform (5 Qubits)

2161 Qunity implementation:

```

2162
2163 def @couple{#k} : Bit * Bit → Bit * Bit :=
2164     lambda (x0, x1) → (x1, x0) ▷ rphase{($1, $1), 2 * pi / (2 ^
2165         #k), 0}
2166 end
2167
2168 def @rotations{#n} : Array{#n, Bit} → Array{#n, Bit} :=
2169     if #n <= 0 then
2170         @id{Unit}
2171     else if #n = 1 then
2172         lambda (x, ()) → (@had(x), ())
2173     else
2174         lambda (x0, x) →
2175             let (x0, (y0', y)) = (x0, @rotations{#n - 1}(x)) in
2176             let ((y0, y1), y) = (@couple{#n}(x0, y0'), y) in
2177             (y0, (y1, y))
2178     endif
2179 endif
2180
2181 end
2182
2183 def @qft{#n} : Array{#n, Bit} → Array{#n, Bit} :=
2184     if #n <= 0 then
2185         @id{Unit}
2186     else
2187         lambda x →
2188             let (x0, x') = @rotations{#n}(x) in
2189             (x0, @qft{#n - 1}(x'))
2190     endif
2191
2192 end
2193

```

```

2194 @qft{5}($0, ($plus, ($plus, ($0, ($0, ()))))) ▷ @reverse{5, Bit}
2195

```

2196 Qiskit implementation:

```

2197 from qiskit.circuit.library import QFT
2198
2199 circ_qft = QuantumCircuit(5, 5)
2200 circ_qft.h([1, 2])
2201 circ_qft.append(QFT(5), [0, 1, 2, 3, 4])
2202 circ_qft.measure([0, 1, 2, 3, 4], [4, 3, 2, 1, 0])
2203

```

2204

2205

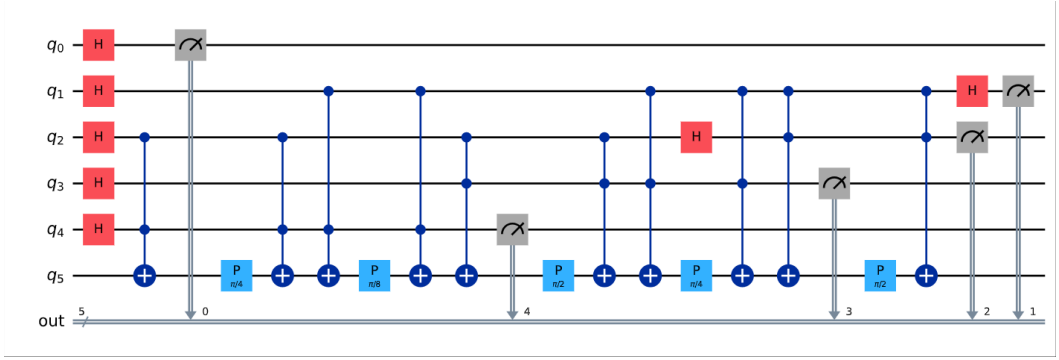


Fig. 23. Compiled circuit for the Quantum Fourier Transform.

J.3 Phase Estimation Example (5 Qubits)

Qunity implementation:

```

2223 def @apply_phase{#n, #p} : Num{#n} → Num{#n} :=
2224   if #n = 0 then
2225     @id{Unit}
2226   else
2227     lambda (x0, x') →
2228       (ctrl x0 [
2229         $0 → x0;
2230         $1 → x0 ▷ gphase{2 * pi * #p}
2231       ], @apply_phase{#n - 1, 2 * #p}(x'))
2232   endif
2233 end
2234
2235
2236 def $phase_estimation{#n, #p} : Num{#n} :=
2237   $repeated{#n, Bit, $plus}
2238   ▷ @apply_phase{#n, #p}
2239   ▷ @adjoint{Num{#n}, Num{#n}, @qft{#n}}
2240   ▷ @reverse{#n, Bit}
2241 end

```

\$phase_estimation{5, 1/3}

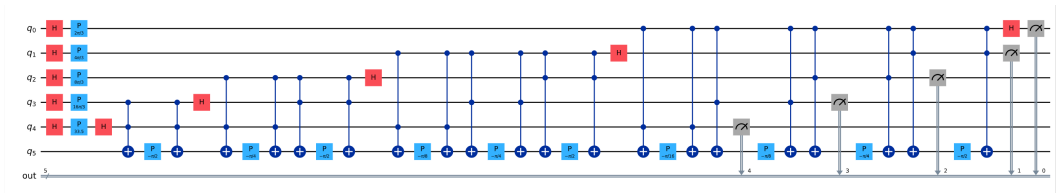


Fig. 24. Compiled circuit for phase estimation.

Qiskit implementation:

```
from qiskit.circuit.library import PhaseEstimation,
    GlobalPhaseGate

pe_circ = QuantumCircuit(5, 5)
pe_circ.h(range(5))
pe_circ.append(PhaseEstimation(5, GlobalPhaseGate(1/3)), range(5))
pe_circ.measure(range(5), range(5))
```

J.4 Order Finding

For the Qunity implementation, see Section 2.2.

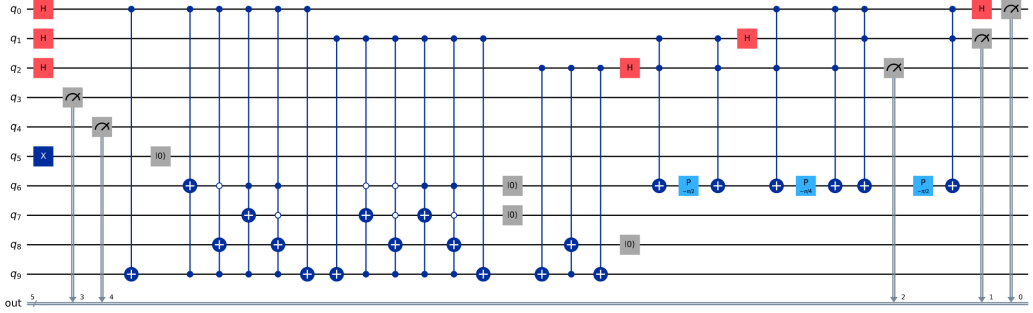


Fig. 25. Compiled circuit for order finding.

J.5 Reversible CDKM Adder (5 Bits)

Qunity implementation:

```
def @cdkm_maj : Bit * Bit * Bit → Bit * Bit * Bit :=
  lambda ((a, b), c) → ctrl c [
    $0 → (c, (a, b));
    $1 → (c, (@not(a), @not(b)))
  ] ▷ lambda (c, (a, b)) → ctrl (a, b) [
    ($1, $1) → ((a, b), @not(c));
    else → ((a, b), c);
  ]
end

def @cdkm_uma : Bit * Bit * Bit → Bit * Bit * Bit :=
  lambda ((a, b), c) → ctrl (a, b) [
    ($1, $1) → ((a, b), @not(c));
    else → ((a, b), c);
  ] ▷ lambda ((a, b), c) →
    (a, (b, (c, ()))) ▷ @cnot{3, 2, 0} ▷ @cnot{3, 0, 1} ▷
```

```

2304     lambda (a, (b, (c, ()))) → ((a, b), c)
2305 end
2306
2307 def @rev_adder_helper{#n} : Num{#n} * Num{#n} * Bit → Num{#n} *
2308     Num{#n} * Bit :=
2309     if #n <= 0 then
2310         @id{Num{#n} * Num{#n} * Bit}
2311     else
2312         lambda (((a0, a1), (b0, b1)), c) →
2313             let (((ca, ba), c'), (a1, b1)) = (@cdkm_maj((c, b0), a0),
2314                 (a1, b1)) in
2315             let ((ca, ba), ((a1, s1), c'')) = ((ca, ba), @
2316                 rev_adder_helper{#n - 1}((a1, b1), c')) in
2317             let ((a1, s1), ((c, s0), a0)) = ((a1, s1), @cdkm_uma((ca,
2318                 ba), c'')) in
2319                 (((a0, a1), (s0, s1)), c)
2320     endif
2321 end
2322
2323 def @rev_adder{#n} : Num{#n} * Num{#n} → Num{#n} * Num{#n} :=
2324     lambda (a, b) → ((a, b), $0) ▷ @rev_adder_helper{#n} ▷
2325     lambda (x, $0) → x
2326 end
2327
2328 @rev_adder{5}($repeated{5, Bit, $plus}, $repeated{5, Bit, $plus})

```

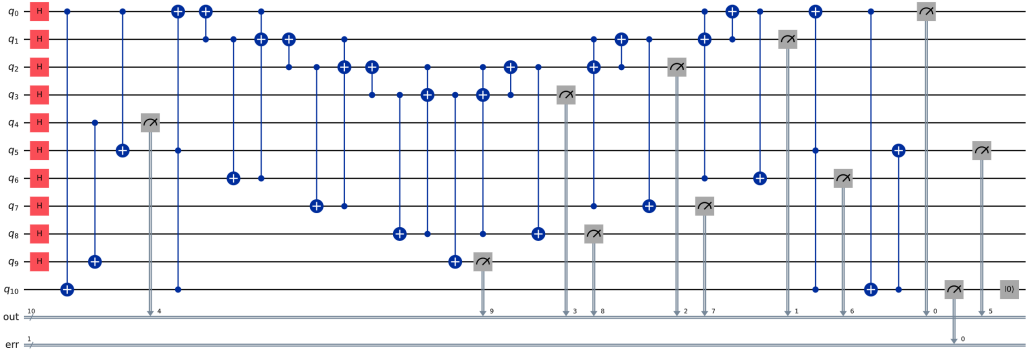


Fig. 26. Compiled circuit for the reversible CDKM adder.

Qiskit implementation:

```

2347 from qiskit.circuit.library import CDKM RippleCarryAdder
2348
2349 add_circ = QuantumCircuit(11, 10)

```

```

2353 add_circ.h(range(11))
2354 add_circ.append(CDKMRippleCarryAdder(5, kind="fixed"), range(11))
2355 add_circ.measure(range(10), range(10))
2356

```

2357 J.6 Grover (5-Bit Match Oracle, 1 Iteration)

2358 Qunity implementation (for the definition of `$grover`, see Section 2.3):

```

2359 def #n := 5 end
2360 def $answer : Num{#n} := ($0, ($1, ($1, ($0, ($0, ()))))) end
2361
2362 def @f : Num{#n} → Bit :=
2363   lambda x → ctrl x [
2364     @answer → (x, $1);
2365     else → (x, $0)
2366   ] ▷ @snd{Num{#n}, Bit}
2367 end
2368
2369 $grover{Num{#n}, $repeated{#n, Bit, $plus}, @f, 1}
2370
2371
2372
2373

```

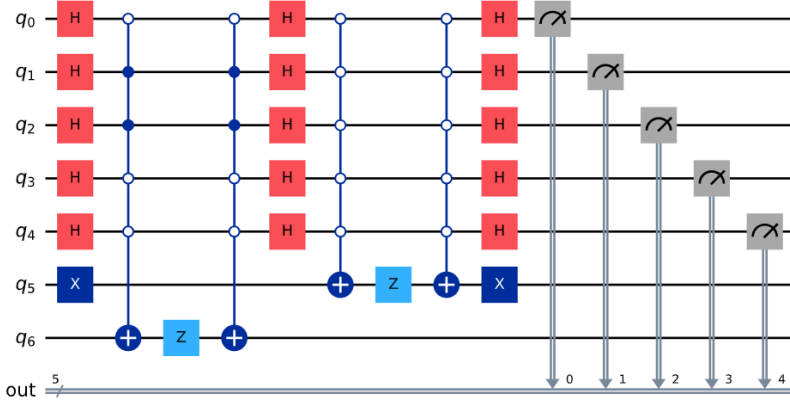


Fig. 27. Compiled circuit for Grover's algorithm with a simple `match` oracle.

Qiskit implementation:

```

2391 circ_grover = QuantumCircuit(6, 5)
2392 circ_grover.x(5)
2393 circ_grover.h(range(6))
2394 circ_grover.mcx([0,1,2,3,4], 5)
2395 circ_grover.h(range(5))
2396 circ_grover.mcx([0,1,2,3,4], 5, ctrl_state="00000")
2397 circ_grover.h(range(5))
2398 circ_grover.measure(range(5), range(5))
2399
2400
2401

```

K PROOFS OF CORRECTNESS FOR THE COMPILATION PROCEDURE

In this section, we justify the correctness of the high-level stage of the compilation procedure, reproducing the original proofs from Voichick et al., with modifications from the addition of the new constructs and the changes described in Section 6. We do not provide proofs of correctness for the low-level stage of compilation (beyond the proof for the direct sum circuit in Appendix G), as mathematically formalizing the circuit specification data structures, the process of circuit instantiation, and qubit allocation and recycling in registers is a complex task beyond the scope of this work.

Here we give the compiled circuits for all of the typing judgment cases and demonstrate algebraically that they are correct.

$$\begin{array}{l} \mathcal{H}(\Gamma) \text{ --- } \mathcal{H}(\Gamma) \\ T\text{-UNIT. } \mathcal{H}(\emptyset) \text{ --- } \mathcal{H}(\text{Unit}) \end{array}$$

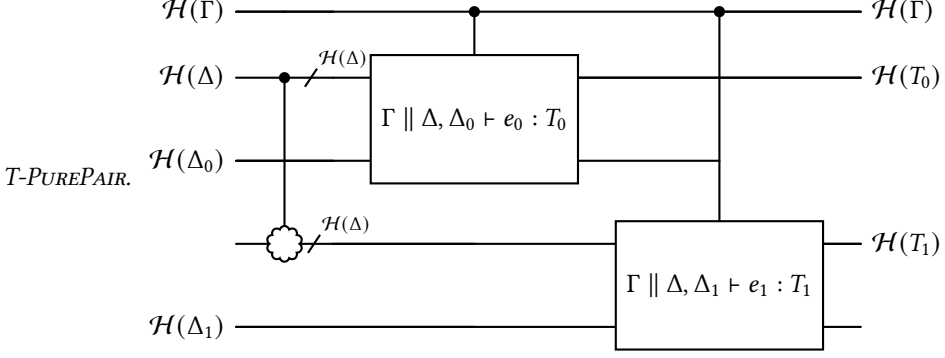
$$\begin{aligned} |\sigma, \emptyset\rangle &\in \mathcal{H}(\Gamma) \otimes \mathcal{H}(\emptyset) \\ &= |\sigma, ()\rangle \in \mathcal{H}(\Gamma) \otimes \mathcal{H}(\text{Unit}) \end{aligned}$$

$$\begin{array}{l} \mathcal{H}(\Gamma) \text{ --- } \mathcal{H}(\Gamma) \\ T\text{-C}_{\text{VAR.}} \quad \mathcal{H}(x : T) \text{ --- } \mathcal{H}(x : T) \\ \mathcal{H}(\Gamma') \text{ --- } \mathcal{H}(\Gamma') \\ \mathcal{H}(\emptyset) \text{ --- } \mathcal{H}(T) \end{array}$$

$$\begin{aligned} |\sigma, x \mapsto v, \sigma'\rangle &\in \mathcal{H}(\Gamma, x : T, \Gamma') \\ \mapsto |\sigma, x \mapsto v, \sigma', v\rangle &\in \mathcal{H}(\Gamma, x : T, \Gamma') \otimes \mathcal{H}(T) \end{aligned}$$

$$\begin{array}{l} \mathcal{H}(\Gamma) \text{ --- } \mathcal{H}(\Gamma) \\ T\text{-Q}_{\text{VAR.}} \quad \mathcal{H}(x : T) \text{ --- } \mathcal{H}(T) \end{array}$$

$$\begin{aligned} |\sigma, x \mapsto v\rangle &\in \mathcal{H}(\Gamma, x : T) \\ &= |\sigma, v\rangle \in \mathcal{H}(\Gamma) \otimes \mathcal{H}(T) \end{aligned}$$



$$|\sigma, \tau, \tau_0, \tau_1\rangle \in \mathcal{H}(\Gamma, \Delta, \Delta_0, \Delta_1)$$

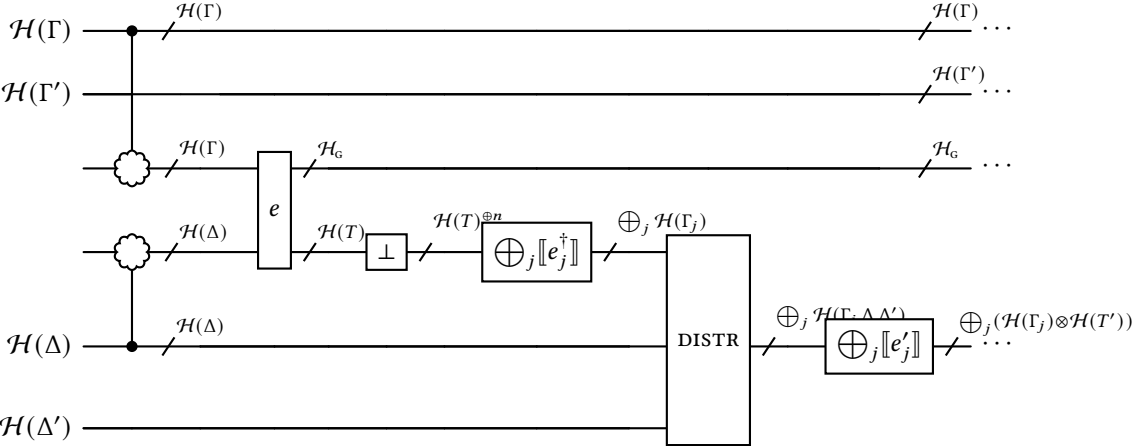
$$\mapsto |\sigma, \tau, \tau_0, \tau, \tau_1\rangle \in \mathcal{H}(\Gamma, \Delta, \Delta_0, \Delta, \Delta_1)$$

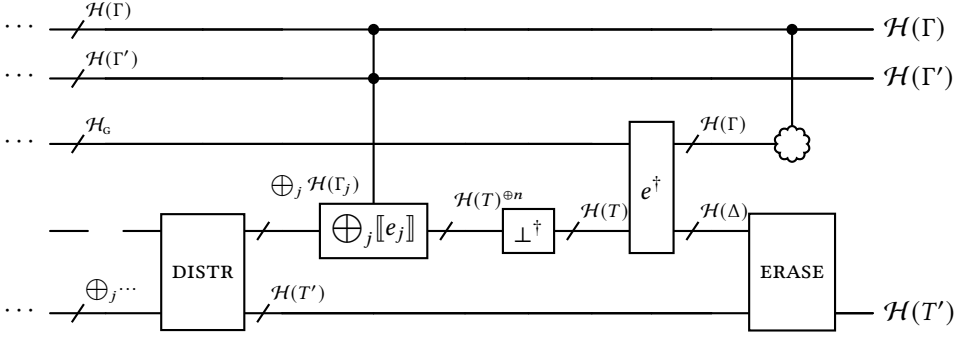
$$\mapsto |\sigma\rangle \otimes \llbracket \sigma : \Gamma \parallel \Delta, \Delta_0 \vdash e_0 : T_0 \rrbracket |\tau, \tau_0\rangle \otimes |\tau, \tau_1\rangle \in \mathcal{H}(\Gamma) \otimes \mathcal{H}(T_0) \otimes \mathcal{H}(\Delta, \Delta_1)$$

$$\mapsto |\sigma\rangle \otimes \llbracket \sigma : \Gamma \parallel \Delta, \Delta_0 \vdash e_0 : T_0 \rrbracket |\tau, \tau_0\rangle \otimes \llbracket \sigma : \Gamma \parallel \Delta, \Delta_1 \vdash e_1 : T_1 \rrbracket |\tau, \tau_1\rangle \in \mathcal{H}(\Gamma) \otimes \mathcal{H}(T_0 \otimes T_1)$$

T-CTRL. The compiled circuit for T-CTRL uses modified versions of its subcircuits. We use Lemma ?? to get a purified version of the circuit for e with semantics $\llbracket e \rrbracket : \mathcal{H}(\Gamma, \Delta) \rightarrow \mathcal{H}(T) \otimes \mathcal{H}_G$. Here, $\mathcal{H}_G$ is some “garbage” Hilbert space containing vectors $\{|g_{\sigma, \tau, v}\rangle : \sigma \in \mathbb{V}(\Gamma), \tau \in \mathbb{V}(\Delta), v \in \mathbb{V}(T)\} \subset \mathcal{H}_G$ such that $\llbracket e \rrbracket |\sigma, \tau\rangle = \sum_{v \in \mathbb{V}(T)} \langle g_{\sigma, \tau, v} | \llbracket e \rrbracket |\sigma, \tau\rangle \cdot |g_{\sigma, \tau, v}\rangle$ for all σ, τ, v . We also use Lemma ?? to get a fallible version of the circuit for e_j with isometric semantics $\llbracket e_j^\dagger \rrbracket : \mathcal{H}(T) \rightarrow \mathcal{H}_j \oplus \mathcal{H}(\Gamma_j)$ and its adjoint $\llbracket e_j \rrbracket : \mathcal{H}_j \oplus \mathcal{H}(\Gamma_j) \rightarrow \mathcal{H}(T)$, where $\mathcal{H}_j$ is some “error” Hilbert space corresponding to e_j . Lemma ?? will ensure that $\llbracket e_j^\dagger \rrbracket |v\rangle = (|a_{j,v}\rangle \oplus 0) + \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket e_j^\dagger \rrbracket |v\rangle \cdot (0 \oplus |\sigma_j\rangle)$ for some “error state” $|a_{j,v}\rangle \in \mathcal{H}_j$.

The circuit below is too large to fit on a single page, so the dots denote where the two pieces must fit together.

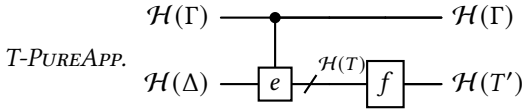




$$\begin{aligned}
& |\sigma, \sigma', \tau, \tau'\rangle \\
& \in \mathcal{H}(\Gamma, \Gamma', \Delta, \Delta') \\
& \mapsto |\sigma, \sigma', \sigma, \tau, \tau, \tau'\rangle \\
& \in \mathcal{H}(\Gamma, \Gamma', \Gamma, \Delta, \Delta, \Delta') \\
& \mapsto |\sigma, \sigma'\rangle \otimes \llbracket e \rrbracket |\sigma, \tau\rangle \otimes |\tau, \tau'\rangle \\
& = |\sigma, \sigma'\rangle \otimes \sum_{v \in \mathbb{V}(T)} \langle g_{\sigma, \tau, v}, v | \llbracket e \rrbracket |\sigma, \tau\rangle \cdot |g_{\sigma, \tau, v}\rangle \otimes |\tau, \tau'\rangle \\
& \in \mathcal{H}(\Gamma, \Gamma') \otimes \mathcal{H}_G \otimes \mathcal{H}(T) \otimes \mathcal{H}(\Delta, \Delta') \\
& \mapsto |\sigma, \sigma'\rangle \otimes \sum_{v \in \mathbb{V}(T)} \langle g_{\sigma, \tau, v}, v | \llbracket e \rrbracket |\sigma, \tau\rangle \cdot |g_{\sigma, \tau, v}\rangle \otimes \left(\bigoplus_{j=1}^n \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket e_j^\dagger \rrbracket |v\rangle \cdot |v\rangle \right) \otimes |\tau, \tau'\rangle \\
& \in \mathcal{H}(\Gamma, \Gamma') \otimes \mathcal{H}_G \otimes \mathcal{H}(T)^{\oplus n} \otimes \mathcal{H}(\Delta, \Delta') \\
& \mapsto |\sigma, \sigma'\rangle \otimes \sum_{v \in \mathbb{V}(T)} \langle g_{\sigma, \tau, v}, v | \llbracket e \rrbracket |\sigma, \tau\rangle \cdot |g_{\sigma, \tau, v}\rangle \otimes \left(\bigoplus_{j=1}^n \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket e_j^\dagger \rrbracket |v\rangle \cdot \llbracket e_j^\dagger \rrbracket |v\rangle \right) \otimes |\tau, \tau'\rangle \\
& = |\sigma, \sigma'\rangle \otimes \sum_{v \in \mathbb{V}(T)} \langle g_{\sigma, \tau, v}, v | \llbracket e \rrbracket |\sigma, \tau\rangle \cdot |g_{\sigma, \tau, v}\rangle \otimes \left(\bigoplus_{j=1}^n \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket e_j^\dagger \rrbracket |v\rangle \cdot \langle \sigma_j | \llbracket e_j^\dagger \rrbracket |v\rangle \cdot |\sigma_j\rangle \right) \otimes |\tau, \tau'\rangle \\
& = |\sigma, \sigma'\rangle \otimes \sum_{v \in \mathbb{V}(T)} \langle g_{\sigma, \tau, v}, v | \llbracket e \rrbracket |\sigma, \tau\rangle \cdot |g_{\sigma, \tau, v}\rangle \otimes \left(\bigoplus_{j=1}^n \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket e_j^\dagger \rrbracket |v\rangle \cdot |\sigma_j\rangle \right) \otimes |\tau, \tau'\rangle \\
& \in \mathcal{H}(\Gamma, \Gamma') \otimes \mathcal{H}_G \otimes \bigoplus_{j=1}^n \mathcal{H}(\Gamma_j) \otimes \mathcal{H}(\Delta, \Delta') \\
& \mapsto |\sigma, \sigma'\rangle \otimes \sum_{v \in \mathbb{V}(T)} \langle g_{\sigma, \tau, v}, v | \llbracket e \rrbracket |\sigma, \tau\rangle \cdot |g_{\sigma, \tau, v}\rangle \otimes \left(\bigoplus_{j=1}^n \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket e_j^\dagger \rrbracket |v\rangle \cdot |\sigma_j, \tau, \tau'\rangle \right) \\
& \in \mathcal{H}(\Gamma, \Gamma') \otimes \mathcal{H}_G \otimes \bigoplus_{j=1}^n (\mathcal{H}(\Gamma_j, \Delta, \Delta')) \\
& \mapsto |\sigma, \sigma'\rangle \otimes \sum_{v \in \mathbb{V}(T)} \langle g_{\sigma, \tau, v}, v | \llbracket e \rrbracket |\sigma, \tau\rangle \cdot |g_{\sigma, \tau, v}\rangle \otimes \left(\bigoplus_{j=1}^n \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket e_j^\dagger \rrbracket |v\rangle \cdot \llbracket e_j' \rrbracket |\sigma_j, \tau, \tau'\rangle \right)
\end{aligned}$$

$$\begin{aligned}
& |\sigma, \sigma'\rangle \otimes \sum_{v \in \mathbb{V}(T)} \langle g_{\sigma, \tau, v} | \llbracket e \rrbracket | \sigma, \tau \rangle \cdot |g_{\sigma, \tau, v}\rangle \\
& \otimes \left(\bigoplus_{j=1}^n \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket e_j^\dagger \rrbracket | v \rangle \cdot \sum_{v' \in \mathbb{V}(T')} \langle \sigma_j, v' | \llbracket e'_j \rrbracket | \sigma_j, \tau, \tau' \rangle \cdot | \sigma_j, v' \rangle \right) \\
& = |\sigma, \sigma'\rangle \otimes \sum_{v \in \mathbb{V}(T)} \langle g_{\sigma, \tau, v} | \llbracket e \rrbracket | \sigma, \tau \rangle \cdot |g_{\sigma, \tau, v}\rangle \\
& \otimes \left(\sum_{j=1}^n \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket e_j^\dagger \rrbracket | v \rangle \cdot \sum_{v' \in \mathbb{V}(T')} \langle \sigma_j, v' | \llbracket e'_j \rrbracket | \sigma_j, \tau, \tau' \rangle \cdot \text{inj}_j | \sigma_j, v' \rangle \right) \\
& \in \mathcal{H}(\Gamma, \Gamma') \otimes \mathcal{H}_G \otimes \bigoplus_{j=1}^n (\mathcal{H}(\Gamma_j) \otimes \mathcal{H}(T')) \\
& \mapsto |\sigma, \sigma'\rangle \otimes \sum_{v \in \mathbb{V}(T)} \langle g_{\sigma, \tau, v} | \llbracket e \rrbracket | \sigma, \tau \rangle \cdot |g_{\sigma, \tau, v}\rangle \\
& \otimes \left(\sum_{j=1}^n \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket e_j^\dagger \rrbracket | v \rangle \cdot \text{inj}_j | \sigma_j \rangle \otimes \sum_{v' \in \mathbb{V}(T')} \langle \sigma_j, v' | \llbracket e'_j \rrbracket | \sigma_j, \tau, \tau' \rangle \cdot |v'\rangle \right) \\
& \in \mathcal{H}(\Gamma, \Gamma') \otimes \mathcal{H}_G \otimes \bigoplus_{j=1}^n \mathcal{H}(\Gamma_j) \otimes \mathcal{H}(T') \\
& \mapsto |\sigma, \sigma'\rangle \otimes \sum_{v \in \mathbb{V}(T)} \langle g_{\sigma, \tau, v} | \llbracket e \rrbracket | \sigma, \tau \rangle \cdot |g_{\sigma, \tau, v}\rangle \\
& \otimes \left(\sum_{j=1}^n \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket e_j^\dagger \rrbracket | v \rangle \cdot \text{inj}_j \llbracket e_j \rrbracket | \sigma_j \rangle \otimes \sum_{v' \in \mathbb{V}(T')} \langle \sigma_j, v' | \llbracket e'_j \rrbracket | \sigma_j, \tau, \tau' \rangle \cdot |v'\rangle \right) \\
& = |\sigma, \sigma'\rangle \otimes \sum_{v \in \mathbb{V}(T)} \langle g_{\sigma, \tau, v} | \llbracket e \rrbracket | \sigma, \tau \rangle \cdot |g_{\sigma, \tau, v}\rangle \\
& \otimes \left(\sum_{j=1}^n \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket e_j^\dagger \rrbracket | v \rangle \cdot \text{inj}_j | v \rangle \otimes \sum_{v' \in \mathbb{V}(T')} \langle \sigma_j, v' | \llbracket e'_j \rrbracket | \sigma_j, \tau, \tau' \rangle \cdot |v'\rangle \right) \\
& \in \mathcal{H}(\Gamma, \Gamma') \otimes \mathcal{H}_G \otimes \bigoplus_{j=1}^n \mathcal{H}(T) \otimes \mathcal{H}(T') \\
& \mapsto |\sigma, \sigma'\rangle \otimes \sum_{v \in \mathbb{V}(T)} \langle g_{\sigma, \tau, v} | \llbracket e \rrbracket | \sigma, \tau \rangle \cdot |g_{\sigma, \tau, v}\rangle \\
& \otimes \left(\sum_{j=1}^n \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket e_j^\dagger \rrbracket | v \rangle \cdot |v\rangle \otimes \sum_{v' \in \mathbb{V}(T')} \langle \sigma_j, v' | \llbracket e'_j \rrbracket | \sigma_j, \tau, \tau' \rangle \cdot |v'\rangle \right) \\
& \in \mathcal{H}(\Gamma, \Gamma') \otimes \mathcal{H}_G \otimes \mathcal{H}(T) \otimes \mathcal{H}(T') \\
& \mapsto |\sigma, \sigma'\rangle \otimes \sum_{v \in \mathbb{V}(T)} \langle g_{\sigma, \tau, v} | \llbracket e \rrbracket | \sigma, \tau \rangle \\
& \otimes \left(\sum_{j=1}^n \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket e_j^\dagger \rrbracket | v \rangle \cdot \llbracket e^\dagger \rrbracket | g_{\sigma, \tau, v}, v \rangle \otimes \sum_{v' \in \mathbb{V}(T')} \langle \sigma_j, v' | \llbracket e'_j \rrbracket | \sigma_j, \tau, \tau' \rangle \cdot |v'\rangle \right) \\
& \in \mathcal{H}(\Gamma, \Gamma', \Gamma, \Delta) \otimes \mathcal{H}(T')
\end{aligned}$$

$$\begin{aligned}
& |\sigma, \sigma'\rangle \otimes \sum_{v \in \mathbb{V}(T)} \langle g_{\sigma, \tau, v} | \llbracket e \rrbracket | \sigma, \tau \rangle \\
& \otimes \left(\sum_{j=1}^n \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket e_j^\dagger \rrbracket | v \rangle \cdot \sum_{\sigma_\star, \tau_\star} \langle \sigma_\star, \tau_\star | \llbracket e^\dagger \rrbracket | g_{\sigma, \tau, v}, v \rangle \cdot |\sigma_\star, \tau_\star\rangle \otimes \sum_{v' \in \mathbb{V}(T')} \langle \sigma_j, v' | \llbracket e'_j \rrbracket | \sigma_j, \tau, \tau' \rangle \cdot |v'\rangle \right) \\
& \in \mathcal{H}(\Gamma, \Gamma', \Gamma, \Delta) \otimes \mathcal{H}(T') \\
& \mapsto |\sigma, \sigma'\rangle \otimes \sum_{v \in \mathbb{V}(T)} \langle g_{\sigma, \tau, v} | \llbracket e \rrbracket | \sigma, \tau \rangle \\
& \otimes \left(\sum_{j=1}^n \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket e_j^\dagger \rrbracket | v \rangle \cdot \sum_{\tau_\star} \langle \sigma, \tau_\star | \llbracket e^\dagger \rrbracket | g_{\sigma, \tau, v}, v \rangle \cdot |\tau_\star\rangle \otimes \sum_{v' \in \mathbb{V}(T')} \langle \sigma_j, v' | \llbracket e'_j \rrbracket | \sigma_j, \tau, \tau' \rangle \cdot |v'\rangle \right) \\
& \in \mathcal{H}(\Gamma, \Gamma', \Delta) \otimes \mathcal{H}(T') \\
& = |\sigma, \sigma'\rangle \otimes \sum_{v \in \mathbb{V}(T)} \langle g_{\sigma, \tau, v} | \llbracket e \rrbracket | \sigma, \tau \rangle \langle \sigma, \tau | \llbracket e^\dagger \rrbracket | g_{\sigma, \tau, v}, v \rangle \\
& \otimes \left(\sum_{j=1}^n \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket e_j^\dagger \rrbracket | v \rangle \cdot |\tau\rangle \otimes \sum_{v' \in \mathbb{V}(T')} \langle \sigma_j, v' | \llbracket e'_j \rrbracket | \sigma_j, \tau, \tau' \rangle \cdot |v'\rangle \right) \\
& \in \mathcal{H}(\Gamma, \Gamma', \Delta) \otimes \mathcal{H}(T') \\
& \mapsto |\sigma, \sigma'\rangle \otimes \sum_{v \in \mathbb{V}(T)} \langle g_{\sigma, \tau, v} | \llbracket e \rrbracket | \sigma, \tau \rangle \langle \sigma, \tau | \llbracket e^\dagger \rrbracket | g_{\sigma, \tau, v}, v \rangle \\
& \otimes \left(\sum_{j=1}^n \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket e_j^\dagger \rrbracket | v \rangle \otimes \sum_{v' \in \mathbb{V}(T')} \langle \sigma_j, v' | \llbracket e'_j \rrbracket | \sigma_j, \tau, \tau' \rangle \cdot |v'\rangle \right) \\
& \in \mathcal{H}(\Gamma, \Gamma') \otimes \mathcal{H}(T')
\end{aligned}$$

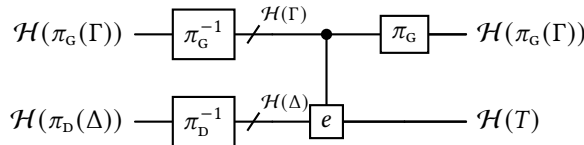


$$|\sigma, \tau\rangle \in \mathcal{H}(\Gamma, \Delta)$$

$$\mapsto |\sigma\rangle \otimes \llbracket e \rrbracket |\tau\rangle \in \mathcal{H}(\Gamma) \otimes \mathcal{H}(T)$$

$$\mapsto |\sigma\rangle \otimes \llbracket f \rrbracket \llbracket e \rrbracket |\tau\rangle \in \mathcal{H}(\Gamma) \otimes \mathcal{H}(T')$$

2688 $T\text{-PUREPERM.}$



$$\begin{aligned}
& |\pi_G(\sigma), \pi_D(\tau)\rangle \in \mathcal{H}(\pi_G(\Gamma), \pi_D(\Delta)) \\
& \mapsto |\sigma, \tau\rangle \in \mathcal{H}(\Gamma, \Delta) \\
& \mapsto |\sigma\rangle \otimes \llbracket e \rrbracket |\tau\rangle \in \mathcal{H}(\Gamma) \otimes \mathcal{H}(T) \\
& \mapsto |\pi_G(\sigma)\rangle \otimes \llbracket e \rrbracket |\tau\rangle \in \mathcal{H}(\pi_G(\Gamma)) \otimes \mathcal{H}(T)
\end{aligned}$$

T-GATE. (We assume that our low-level circuits contain these gates as primitives.)

$$\mathcal{H}(\text{Bit}) \text{ --- } \boxed{\begin{array}{cc} \cos(r_\theta/2) & -e^{ir_\lambda} \sin(r_\theta/2) \\ e^{ir_\phi} \sin(r_\theta/2) & e^{i(r_\phi+r_\lambda)} \cos(r_\theta/2) \end{array}} \text{ --- } \mathcal{H}(\text{Bit})$$

$$\begin{aligned}
|\emptyset\rangle & \in \mathcal{H}(\text{Bit}) \\
\mapsto \cos(r_\theta/2) |\emptyset\rangle + e^{ir_\phi} \sin(r_\theta/2) |1\rangle & \in \mathcal{H}(\text{Bit}) \\
|1\rangle & \in \mathcal{H}(\text{Bit}) \\
\mapsto -e^{ir_\lambda} \sin(r_\theta/2) |\emptyset\rangle + e^{i(r_\phi+r_\lambda)} \cos(r_\theta/2) |1\rangle & \in \mathcal{H}(\text{Bit})
\end{aligned}$$

T-LEFT. (This was already implemented in the previous section.)

$$\mathcal{H}(T_0) \text{ --- } \boxed{\text{left}_{T_0 \oplus T_1}} \text{ --- } \mathcal{H}(T_0 \oplus T_1)$$

$$\begin{aligned}
|v\rangle & \in \mathcal{H}(T_0) \\
\mapsto |v\rangle \oplus 0
\end{aligned}$$

T-RIGHT. (This was already implemented in the previous section.)

$$\mathcal{H}(T_1) \text{ --- } \boxed{\text{right}_{T_0 \oplus T_1}} \text{ --- } \mathcal{H}(T_0 \oplus T_1)$$

$$\begin{aligned}
|v\rangle & \in \mathcal{H}(T_1) \\
\mapsto 0 \oplus |v\rangle
\end{aligned}$$

$$\text{\textit{T-PUREABS.}} \quad \mathcal{H}(T) \text{ --- } \boxed{e^\dagger} \text{ --- } \mathcal{H}(\Delta) \text{ --- } \boxed{e'} \text{ --- } \mathcal{H}(T')$$

$$\begin{aligned}
|v\rangle & \in \mathcal{H}(T) \\
\mapsto \llbracket e \rrbracket^\dagger |v\rangle & \in \mathcal{H}(\Delta) \\
\mapsto \llbracket e' \rrbracket \llbracket e \rrbracket^\dagger |v\rangle & \in \mathcal{H}(T')
\end{aligned}$$

T-RPHASE. Let $E_F : \mathcal{H}(T) \rightarrow \mathcal{H}(\Delta) \oplus \mathcal{H}_F$ be the norm-preserving operator constructed from $\llbracket \emptyset \parallel \Delta \vdash e : T \rrbracket^\dagger$ using Lemma ?? . The compiled T-RPHASE circuit then looks like this:

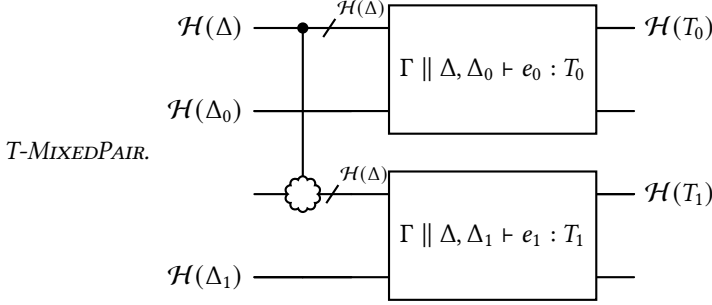
$$\mathcal{H}(T) \text{ --- } \boxed{E_F} \text{ --- } \mathcal{H}(\Delta) \oplus \mathcal{H}_F \text{ --- } \boxed{e^{ir'} \mathbb{I}_\Delta \oplus e^{ir} \mathbb{I}_F} \text{ --- } \mathcal{H}(\Delta) \oplus \mathcal{H}_F \text{ --- } \boxed{E_F^\dagger} \text{ --- } \mathcal{H}(T)$$

Using the fact that $\llbracket \emptyset \parallel \Delta \vdash e : T \rrbracket^\dagger = \llbracket \text{left}_{\Delta \oplus G} \rrbracket^\dagger E_F$,

$$\begin{aligned}
 & E_F^\dagger \left(e^{ir} \mathbb{I}_\Delta \oplus e^{ir'} \mathbb{I}_F \right) E_F \\
 &= E_F^\dagger \left(e^{ir} \llbracket \text{left}_{\Delta \oplus G} \rrbracket^\dagger \llbracket \text{left}_{\Delta \oplus G} \rrbracket^\dagger + e^{ir'} \llbracket \text{right}_{\Delta \oplus G} \rrbracket^\dagger \llbracket \text{right}_{\Delta \oplus G} \rrbracket^\dagger \right) E_F \\
 &= E_F^\dagger \left(e^{ir} \llbracket \text{left}_{\Delta \oplus G} \rrbracket^\dagger \llbracket \text{left}_{\Delta \oplus G} \rrbracket^\dagger + e^{ir'} (\mathbb{I} - \llbracket \text{left}_{\Delta \oplus G} \rrbracket^\dagger \llbracket \text{left}_{\Delta \oplus G} \rrbracket^\dagger) \right) E_F \\
 &= e^{ir} E_F^\dagger \llbracket \text{left}_{\Delta \oplus G} \rrbracket^\dagger \llbracket \text{left}_{\Delta \oplus G} \rrbracket^\dagger E_F + e^{ir'} \left(\mathbb{I} - E_F^\dagger \llbracket \text{left}_{\Delta \oplus G} \rrbracket^\dagger \llbracket \text{left}_{\Delta \oplus G} \rrbracket^\dagger E_F \right) \\
 &= e^{ir} \llbracket \emptyset \parallel \Delta \vdash e : T \rrbracket^\dagger \llbracket \emptyset \parallel \Delta \vdash e : T \rrbracket^\dagger + e^{ir'} \left(\mathbb{I} - \llbracket \emptyset \parallel \Delta \vdash e : T \rrbracket^\dagger \llbracket \emptyset \parallel \Delta \vdash e : T \rrbracket^\dagger \right)
 \end{aligned}$$

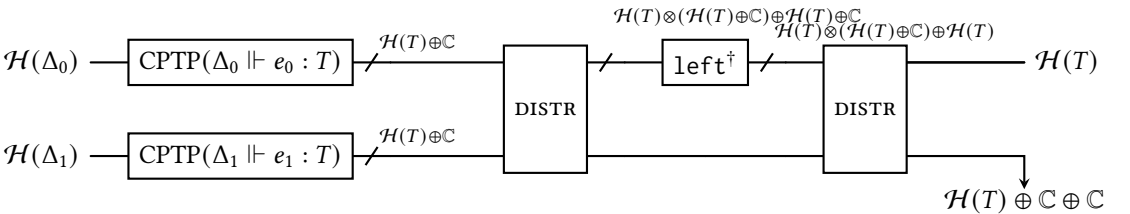
T-MIX. $\mathcal{H}(\Delta) \text{ --- } \boxed{e} \text{ --- } \mathcal{H}(T)$

$$\begin{aligned}
 & |\tau \rangle \langle \tau'| \in \mathcal{L}(\mathcal{H}(\Delta)) \\
 & \mapsto \llbracket e \rrbracket |\tau \rangle \langle \tau'| \llbracket e \rrbracket^\dagger \in \mathcal{L}(\mathcal{H}(T))
 \end{aligned}$$



$$\begin{aligned}
 & |\tau, \tau_0, \tau_1 \rangle \langle \tau', \tau'_0, \tau'_1| \in \mathcal{L}(\mathcal{H}(\Delta, \Delta_0, \Delta_1)) \\
 & \mapsto |\tau, \tau_0, \tau, \tau_1 \rangle \langle \tau', \tau'_0, \tau', \tau'_1| \\
 &= |\tau, \tau_0 \rangle \langle \tau', \tau'_0| \otimes |\tau, \tau_1 \rangle \langle \tau', \tau'_1| \in \mathcal{L}(\mathcal{H}(\Delta, \Delta_0)) \otimes \mathcal{L}(\mathcal{H}(\Delta, \Delta_1)) \\
 & \mapsto \llbracket e_0 \rrbracket (|\tau, \tau_0 \rangle \langle \tau', \tau'_0|) \otimes \llbracket e_1 \rrbracket (|\tau, \tau_1 \rangle \langle \tau', \tau'_1|) \in \mathcal{L}(\mathcal{H}(T_0 \otimes T_1))
 \end{aligned}$$

T-TRY.



$$\begin{aligned}
& |\tau_0, \tau_1 \rangle \langle \tau'_0, \tau'_1| \\
&= |\tau_0 \rangle \langle \tau'_0| \otimes |\tau_1 \rangle \langle \tau'_1| \\
&\mapsto (\llbracket \Delta_0 \Vdash e_0 : T \rrbracket (|\tau_0 \rangle \langle \tau'_0|) \oplus (1 - \text{tr}(\llbracket \Delta_0 \Vdash e_0 : T \rrbracket (|\tau_0 \rangle \langle \tau'_0|)))) \\
&\quad \otimes (\llbracket \Delta_1 \Vdash e_1 : T \rrbracket (|\tau_1 \rangle \langle \tau'_1|) \oplus (1 - \text{tr}(\llbracket \Delta_1 \Vdash e_1 : T \rrbracket (|\tau_1 \rangle \langle \tau'_1|)))) \\
&\mapsto \llbracket \Delta_0 \Vdash e_0 : T \rrbracket (|\tau_0 \rangle \langle \tau'_0|) \otimes (\llbracket \Delta_1 \Vdash e_1 : T \rrbracket (|\tau_1 \rangle \langle \tau_1|) \oplus (1 - \text{tr}(\llbracket \Delta_1 \Vdash e_1 : T \rrbracket (|\tau_1 \rangle \langle \tau_1|)))) \\
&\quad \oplus (1 - \text{tr}(\llbracket \Delta_0 \Vdash e_0 : T \rrbracket (|\tau_0 \rangle \langle \tau'_0|))) (\llbracket \Delta_1 \Vdash e_1 : T \rrbracket (|\tau_1 \rangle \langle \tau_1|) \oplus (1 - \text{tr}(\llbracket \Delta_1 \Vdash e_1 : T \rrbracket (|\tau_1 \rangle \langle \tau_1|)))) \\
&\mapsto (\llbracket \Delta_0 \Vdash e_0 : T \rrbracket (|\tau_0 \rangle \langle \tau'_0|) + (1 - \text{tr}(\llbracket \Delta_0 \Vdash e_0 : T \rrbracket (|\tau_0 \rangle \langle \tau'_0|))) \llbracket \Delta_1 \Vdash e_1 : T \rrbracket (|\tau_1 \rangle \langle \tau_1|)) \otimes (\cdots) \\
&\mapsto \llbracket \Delta_0 \Vdash e_0 : T \rrbracket (|\tau_0 \rangle \langle \tau'_0|) + (1 - \text{tr}(\llbracket \Delta_0 \Vdash e_0 : T \rrbracket (|\tau_0 \rangle \langle \tau'_0|))) \llbracket \Delta_1 \Vdash e_1 : T \rrbracket (|\tau_1 \rangle \langle \tau_1|)
\end{aligned}$$

$$T\text{-MIXEDAPP. } \mathcal{H}(\Delta) \text{ --- } \boxed{e} \text{ --- } \boxed{f} \text{ --- } \mathcal{H}(T')$$

$$\begin{aligned}
& |\tau \rangle \langle \tau'| \in \mathcal{L}(\mathcal{H}(\Delta)) \\
&\mapsto \llbracket e \rrbracket (|\tau \rangle \langle \tau'|) \in \mathcal{L}(\mathcal{H}(T)) \\
&\mapsto \llbracket f \rrbracket (\llbracket e \rrbracket (|\tau \rangle \langle \tau'|)) \in \mathcal{L}(\mathcal{H}(T'))
\end{aligned}$$

T-MIXEDPERM.

$$\mathcal{H}(\pi(\Delta)) \text{ --- } \boxed{\pi^{-1}} \text{ --- } \boxed{e} \text{ --- } \mathcal{H}(T)$$

$$\begin{aligned}
& |\pi(\tau) \rangle \langle \pi(\tau')| \in \mathcal{L}(\mathcal{H}(\pi(\Delta))) \\
&\mapsto |\tau \rangle \langle \tau'| \in \mathcal{L}(\mathcal{H}(\Delta)) \\
&\mapsto \llbracket e \rrbracket (|\tau \rangle \langle \tau'|) \in \mathcal{L}(\mathcal{H}(T))
\end{aligned}$$

$$T\text{-CHANNEL. } \mathcal{H}(T) \text{ --- } \boxed{f} \text{ --- } \mathcal{H}(T')$$

$$\begin{aligned}
& |v \rangle \langle v'| \in \mathcal{L}(\mathcal{H}(T)) \\
&\mapsto \llbracket f \rrbracket |v \rangle \langle v'| \llbracket f \rrbracket^\dagger \in \mathcal{L}(\mathcal{H}(T'))
\end{aligned}$$

$$\begin{array}{c}
\mathcal{H}(T) \text{ --- } \boxed{e^\dagger} \text{ --- } \boxed{e'} \text{ --- } \mathcal{H}(T') \\
\downarrow \\
\mathcal{H}(\Delta_0)
\end{array}$$

T-MIXEDABS.

$$\begin{aligned}
& |v \rangle \langle v'| \in \mathcal{L}(\mathcal{H}(T)) \\
&\mapsto \llbracket e \rrbracket^\dagger |v \rangle \langle v'| \llbracket e \rrbracket \in \mathcal{L}(\mathcal{H}(\Delta, \Delta_0)) \\
&\mapsto \text{tr}_{\Delta_0} (\llbracket e \rrbracket^\dagger |v \rangle \langle v'| \llbracket e \rrbracket) \in \mathcal{L}(\mathcal{H}(\Delta)) \\
&\mapsto \llbracket e' \rrbracket (\text{tr}_{\Delta_0} (\llbracket e \rrbracket^\dagger |v \rangle \langle v'| \llbracket e \rrbracket)) \in \mathcal{L}(\mathcal{H}(T'))
\end{aligned}$$

LEMMA K.1 (ORTHOGONALITY COMPILATION). *Suppose $(\emptyset \parallel \Gamma_j \vdash e_j : T)$ for all $j \in \{1, \dots, n\}$ and $\text{ortho}_T(e_1, \dots, e_n)$. Then, one can implement a Kraus operator $\llbracket \text{ortho}_T(e_1, \dots, e_n) \rrbracket : \mathcal{H}(T) \rightarrow$*

$\mathcal{H}(T)^{\oplus n}$ such that for all $v \in \mathbb{V}(T)$:

$$\begin{aligned} \llbracket \text{ortho}_T(e_1, \dots, e_n) \rrbracket |v\rangle &= \bigoplus_{j=1}^n \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket \emptyset : \emptyset \parallel \Gamma_j \vdash e_j : T \rrbracket^\dagger |v\rangle \cdot |v\rangle \\ \llbracket \text{ortho}_T(e_1, \dots, e_n) \rrbracket^\dagger (\text{inj}_j |v\rangle) &= \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket \emptyset : \emptyset \parallel \Gamma_j \vdash e_j : T \rrbracket^\dagger |v\rangle \cdot |v\rangle \end{aligned}$$

PROOF. We will first prove this for the case where $\text{spanning}_T(e_1, \dots, e_n)$ holds, and then generalize. We construct the required circuit recursively based on specific rule used to prove the spanning judgment.

S-UNIT. In this case, $n = 1$, $e_1 = ()$, $T = \text{Unit}$, and $\Gamma_1 = \emptyset$. The output Hilbert space $\mathcal{H}(\text{Unit})^{\oplus 1}$ is isomorphic to the input Hilbert space $\mathcal{H}(\text{Unit})$, and the required behavior reduces to the identity function because $\langle \emptyset | \llbracket \emptyset : \emptyset \parallel \emptyset \vdash () : \text{Unit} \rrbracket^\dagger |()\rangle = 1$, so an empty circuit suffices:

$$\mathcal{H}(\text{Unit}) \longrightarrow \mathcal{H}(\text{Unit})^{\oplus 1}$$

$$\begin{aligned} \llbracket \text{spanning}_{\text{Unit}}(()) \rrbracket |()\rangle &= |()\rangle \\ &= \langle \emptyset | \llbracket \emptyset : \emptyset \parallel \emptyset \vdash () : \text{Unit} \rrbracket^\dagger |()\rangle \cdot |()\rangle \\ &= \sum_{\sigma_1 \in \mathbb{V}(\emptyset)} \langle \sigma_1 | \llbracket \emptyset : \emptyset \parallel \emptyset \vdash () : \text{Unit} \rrbracket^\dagger |()\rangle \cdot |()\rangle \\ &= \bigoplus_{j=1}^1 \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket \emptyset : \emptyset \parallel \Gamma_j \vdash e_j : \text{Unit} \rrbracket^\dagger |()\rangle \cdot |()\rangle \\ \llbracket \text{spanning}_{\text{Unit}}(()) \rrbracket^\dagger (\text{inj}_1 |()\rangle) &= |()\rangle \\ &= \sum_{\sigma_1 \in \mathbb{V}(\emptyset)} \langle \sigma_1 | \llbracket \emptyset : \emptyset \parallel \emptyset \vdash () : \text{Unit} \rrbracket^\dagger |()\rangle \cdot |()\rangle \end{aligned}$$

S-VAR. In this case, $n = 1$, $e_1 = x$, and $\Gamma_1 = (x : T)$. Again, an identity circuit suffices:

$$\mathcal{H}(T) \longrightarrow \mathcal{H}(T)^{\oplus 1}$$

$$\begin{aligned} \llbracket \text{spanning}_T(x) \rrbracket |v\rangle &= |v\rangle \\ &= \langle x \mapsto v | \llbracket \emptyset : \emptyset \parallel x : T \vdash x : T \rrbracket^\dagger |v\rangle \cdot |v\rangle \\ &= \sum_{\sigma_1 \in \mathbb{V}(x:T)} \langle \sigma_1 | \llbracket \emptyset : \emptyset \parallel x : T \vdash x : T \rrbracket^\dagger |v\rangle \cdot |v\rangle \\ &= \bigoplus_{j=1}^1 \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket \emptyset : \emptyset \parallel \Gamma_j \vdash e_j : T \rrbracket^\dagger |v\rangle \cdot |v\rangle \\ \llbracket \text{spanning}_T(x) \rrbracket^\dagger (\text{inj}_1 |v\rangle) &= |v\rangle \\ &= \sum_{\sigma_1 \in \mathbb{V}(x:T)} \langle \sigma_1 | \llbracket \emptyset : \emptyset \parallel x : T \vdash x : T \rrbracket^\dagger |v\rangle \cdot |v\rangle \end{aligned}$$

S-SUM. In this case, our list of expressions is:

$$[\text{left}_{T \oplus T'} e_1, \dots, \text{left}_{T \oplus T'} e_n, \text{right}_{T \oplus T'} e'_1, \dots, \text{right}_{T \oplus T'} e'_{n'}]$$

$$\mathcal{H}(T)^{\oplus n} \oplus \mathcal{H}(T')^{\oplus n'}$$

$$\mathcal{H}(T \oplus T') \text{ --- } \boxed{\text{spanning} \oplus \text{spanning}'} \text{ --- } \boxed{\llbracket \text{left}_{T \oplus T'} \rrbracket \oplus \llbracket \text{right}_{T \oplus T'} \rrbracket} \text{ --- } \mathcal{H}(T \oplus T')^{\oplus (n+n')}$$

$$\llbracket \text{spanning}_{T \oplus T'} (\dots) \rrbracket (|v\rangle \oplus 0)$$

$$= \left(\bigoplus_{j=1}^n \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket \emptyset : \emptyset \parallel \Gamma_j \vdash e_j : T \rrbracket^\dagger |v\rangle \cdot (|v\rangle \oplus 0) \right) \oplus 0$$

$$= \left(\bigoplus_{j=1}^n \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket \emptyset : \emptyset \parallel \Gamma_j \vdash \text{left}_{T \oplus T'} e_j : T \oplus T' \rrbracket^\dagger (|v\rangle \oplus 0) \cdot (|v\rangle \oplus 0) \right) \oplus 0$$

$$\llbracket \text{spanning}_{T \oplus T'} (\dots) \rrbracket (0 \oplus |v\rangle)$$

$$= 0 \oplus \left(\bigoplus_{j=1}^{n'} \sum_{\sigma_j \in \mathbb{V}(\Gamma'_j)} \langle \sigma_j | \llbracket \emptyset : \emptyset \parallel \Gamma'_j \vdash e'_j : T' \rrbracket^\dagger |v\rangle \cdot (0 \oplus |v\rangle) \right)$$

$$= 0 \oplus \left(\bigoplus_{j=1}^{n'} \sum_{\sigma_j \in \mathbb{V}(\Gamma'_j)} \langle \sigma_j | \llbracket \emptyset : \emptyset \parallel \Gamma'_j \vdash \text{right}_{T \oplus T'} e'_j : T \oplus T' \rrbracket^\dagger (0 \oplus |v\rangle) \cdot (0 \oplus |v\rangle) \right)$$

$$\llbracket \text{spanning}_{T \oplus T'} (\dots) \rrbracket^\dagger (\text{inj}_j(|v\rangle \oplus 0) \oplus 0)$$

$$= \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket \emptyset : \emptyset \parallel \Gamma_j \vdash e_j : T \rrbracket^\dagger |v\rangle \cdot |v\rangle \oplus 0$$

$$= \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket \emptyset : \emptyset \parallel \Gamma_j \vdash \text{left}_{T \oplus T'} e_j : T \oplus T' \rrbracket^\dagger (|v\rangle \oplus 0) \cdot (|v\rangle \oplus 0)$$

$$\llbracket \text{spanning}_{T \oplus T'} (\dots) \rrbracket^\dagger (\text{inj}_j(0 \oplus |v\rangle) \oplus 0)$$

$$= 0$$

$$= \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \langle \sigma_j | \llbracket \emptyset : \emptyset \parallel \Gamma_j \vdash \text{left}_{T \oplus T'} e_j : T \oplus T' \rrbracket^\dagger (0 \oplus |v\rangle) \cdot (0 \oplus |v\rangle)$$

$$\llbracket \text{spanning}_{T \oplus T'} (\dots) \rrbracket^\dagger (0 \oplus \text{inj}_j(|v\rangle \oplus 0))$$

$$= 0$$

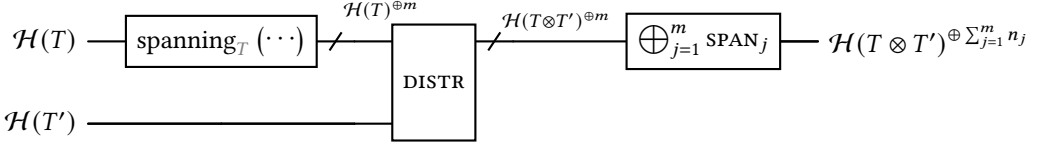
$$= \sum_{\sigma_j \in \mathbb{V}(\Gamma'_j)} \langle \sigma_j | \llbracket \emptyset : \emptyset \parallel \Gamma'_j \vdash \text{right}_{T \oplus T'} e'_j : T \oplus T' \rrbracket^\dagger (|v\rangle \oplus 0) \cdot (|v\rangle \oplus 0)$$

$$\llbracket \text{spanning}_{T \oplus T'} (\dots) \rrbracket^\dagger (0 \oplus \text{inj}_j(0 \oplus |v\rangle))$$

$$= \sum_{\sigma_j \in \mathbb{V}(\Gamma'_j)} \langle \sigma_j | \llbracket \emptyset : \emptyset \parallel \Gamma'_j \vdash e'_j : T' \rrbracket^\dagger |v\rangle \cdot (0 \oplus |v\rangle)$$

$$= \sum_{\sigma_j \in \mathbb{V}(\Gamma'_j)} \langle \sigma_j | \llbracket \emptyset : \emptyset \parallel \Gamma'_j \vdash \text{right}_{T \oplus T'} e'_j : T \oplus T' \rrbracket^\dagger (0 \oplus |v\rangle) \cdot (0 \oplus |v\rangle)$$

S-PAIR. We can compile this judgment for $T \otimes T'$ in terms of the compiled judgments for $\text{spanning}_T(e_1, \dots, e_m)$ and $\text{spanning}_{T'}(e'_{j,1}, \dots, e'_{j,n_j})$ for $j = 1, \dots, m$. The output from the T compilation is used to control which of the T' judgments is used. With abbreviation $\text{SPAN}_j := \text{spanning}_{T'}(e'_{j,1}, \dots, e'_{j,n_j})$:



Assume that each pattern is well-typed as $\emptyset \parallel \Gamma_{j,k} \vdash (e_j, e'_{j,k}) : T \otimes T'$. The spanning judgment ensures that the free variables in e_j are distinct from those of $e'_{j,k}$, so the context can be partitioned as $\Gamma_{j,k} = \Gamma_j, \Gamma'_{j,k}$, with $\emptyset \parallel \Gamma_j \vdash e_j : T$ and $\emptyset \parallel \Gamma'_{j,k} \vdash e'_{j,k} : T'$.

$$\begin{aligned}
& \llbracket \text{spanning}_{T \otimes T'}(\dots) \rrbracket |v, v'\rangle \\
&= \bigoplus_{j=1}^m \bigoplus_{k=1}^{n_j} \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \sum_{\sigma'_{j,k} \in \mathbb{V}(\Gamma'_{j,k})} \langle \sigma_j | \llbracket \emptyset : \emptyset \parallel \Gamma_j \vdash e_j : T \rrbracket^\dagger |v\rangle \cdot |v\rangle \otimes \langle \sigma'_{j,k} | \llbracket \emptyset : \emptyset \parallel \Gamma'_{j,k} \vdash e'_{j,k} : T' \rrbracket^\dagger |v'\rangle \cdot |v'\rangle \\
&= \bigoplus_{j=1}^m \bigoplus_{k=1}^{n_j} \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \sum_{\sigma'_{j,k} \in \mathbb{V}(\Gamma'_{j,k})} \langle \sigma_j, \sigma'_{j,k} | \llbracket \emptyset : \emptyset \parallel \Gamma_j, \Gamma'_{j,k} \vdash (e_j, e'_{j,k}) : T \otimes T' \rrbracket^\dagger |v, v'\rangle \cdot |v, v'\rangle \\
&= \llbracket \text{spanning}_{T \otimes T'}(\dots) \rrbracket^\dagger \text{inj}_j \text{inj}_k |v, v'\rangle \\
&= \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \sum_{\sigma'_{j,k} \in \mathbb{V}(\Gamma'_{j,k})} \langle \sigma_j | \llbracket \emptyset : \emptyset \parallel \Gamma_j \vdash e_j : T \rrbracket^\dagger |v\rangle \cdot |v\rangle \otimes \langle \sigma'_{j,k} | \llbracket \emptyset : \emptyset \parallel \Gamma'_{j,k} \vdash e'_{j,k} : T' \rrbracket^\dagger |v'\rangle \cdot |v'\rangle \\
&= \sum_{\sigma_j \in \mathbb{V}(\Gamma_j)} \sum_{\sigma'_{j,k} \in \mathbb{V}(\Gamma'_{j,k})} \langle \sigma_j, \sigma'_{j,k} | \llbracket \emptyset : \emptyset \parallel \Gamma_j, \Gamma'_{j,k} \vdash (e_j, e'_{j,k}) : T \otimes T' \rrbracket^\dagger |v, v'\rangle
\end{aligned}$$

$$S\text{-FLIP. } \mathcal{H}(T) \text{ --- } \boxed{\text{spanning}_T(\cdots)} \text{ --- } \boxed{\mathcal{H}(T)^{\oplus(n_1+n_2+n_3+n_4)} \quad \mathbb{I} \oplus \text{FLIP} \oplus \mathbb{I}} \text{ --- } \mathcal{H}(T)^{\oplus(n_1+n_3+n_2+n_4)}$$

$$\begin{aligned} & \llbracket \text{spanning}_T(\cdots) \rrbracket \\ & : \quad |v\rangle \\ & \mapsto \bigoplus_{j=1}^{n_1} \sum_{\sigma \in \mathbb{V}(\Gamma_{1,j})} \langle \sigma | \llbracket \emptyset : \emptyset \parallel \Gamma_{1,j} \vdash e_{1,j} : T \rrbracket^\dagger |v\rangle \cdot |v\rangle \\ & \quad \oplus \bigoplus_{j=1}^{n_2} \sum_{\sigma \in \mathbb{V}(\Gamma_{2,j})} \langle \sigma | \llbracket \emptyset : \emptyset \parallel \Gamma_{2,j} \vdash e_{2,j} : T \rrbracket^\dagger |v\rangle \cdot |v\rangle \\ & \quad \oplus \bigoplus_{j=1}^{n_3} \sum_{\sigma \in \mathbb{V}(\Gamma_{3,j})} \langle \sigma | \llbracket \emptyset : \emptyset \parallel \Gamma_{3,j} \vdash e_{3,j} : T \rrbracket^\dagger |v\rangle \cdot |v\rangle \\ & \quad \oplus \bigoplus_{j=1}^{n_4} \sum_{\sigma \in \mathbb{V}(\Gamma_{4,j})} \langle \sigma | \llbracket \emptyset : \emptyset \parallel \Gamma_{4,j} \vdash e_{4,j} : T \rrbracket^\dagger |v\rangle \cdot |v\rangle \\ & \mapsto \bigoplus_{j=1}^{n_1} \sum_{\sigma \in \mathbb{V}(\Gamma_{1,j})} \langle \sigma | \llbracket \emptyset : \emptyset \parallel \Gamma_{1,j} \vdash e_{1,j} : T \rrbracket^\dagger |v\rangle \cdot |v\rangle \\ & \quad \oplus \bigoplus_{j=1}^{n_3} \sum_{\sigma \in \mathbb{V}(\Gamma_{3,j})} \langle \sigma | \llbracket \emptyset : \emptyset \parallel \Gamma_{3,j} \vdash e_{3,j} : T \rrbracket^\dagger |v\rangle \cdot |v\rangle \\ & \quad \oplus \bigoplus_{j=1}^{n_2} \sum_{\sigma \in \mathbb{V}(\Gamma_{2,j})} \langle \sigma | \llbracket \emptyset : \emptyset \parallel \Gamma_{2,j} \vdash e_{2,j} : T \rrbracket^\dagger |v\rangle \cdot |v\rangle \\ & \quad \oplus \bigoplus_{j=1}^{n_4} \sum_{\sigma \in \mathbb{V}(\Gamma_{4,j})} \langle \sigma | \llbracket \emptyset : \emptyset \parallel \Gamma_{4,j} \vdash e_{4,j} : T \rrbracket^\dagger |v\rangle \cdot |v\rangle \end{aligned}$$

Ortho. We have now shown that the needed Kraus operator can be implemented whenever $\text{spanning}_T(e_1, \dots, e_n)$ is true. In the general case, $\text{ortho}_T(e_1, \dots, e_n)$ is true instead, where $[e_1, \dots, e_n]$ is a subsequence of some larger sequence $[e'_1, \dots, e'_{n'}]$ such that $\text{spanning}_T(e'_1, \dots, e'_{n'})$ is true. The circuit for the orthogonality judgment can be constructed from the circuit for the spanning judgment. Given an expression e'_j in the spanning list, define V_j to be $\mathbb{I} \in \mathcal{L}(\mathcal{H}(T))$ whenever e'_j is in the ortho list, and $0 \in \mathcal{L}(\mathcal{H}(T), \mathcal{H}(\text{Void}))$ whenever it is not. The ortho circuit is then constructed as follows:

$$\begin{aligned}
& \mathcal{H}(T) \longrightarrow \boxed{\text{spanning}_T(e'_1, \dots, e'_{n'})} \xrightarrow{\mathcal{H}(T)^{\oplus n'}} \boxed{\bigoplus_{j=1}^{n'} V_j} \longrightarrow \mathcal{H}(T)^{\oplus n} \\
& \quad \llbracket \text{ortho}_T(e_1, \dots, e_n) \rrbracket \\
& : \quad |v\rangle \\
& \mapsto \bigoplus_{j=1}^{n'} \sum_{\sigma \in \mathbb{V}(\Gamma_j)} \langle \sigma | \llbracket \emptyset : \emptyset \parallel \Gamma_j \vdash e'_j : T \rrbracket^\dagger |v\rangle \cdot |v\rangle \in \mathcal{H}(T)^{\oplus n'} \\
& \mapsto \bigoplus_{j=1}^{n'} \sum_{\sigma \in \mathbb{V}(\Gamma_j)} \langle \sigma | \llbracket \emptyset : \emptyset \parallel \Gamma_j \vdash e'_j : T \rrbracket^\dagger |v\rangle \cdot V_j |v\rangle \\
& = \bigoplus_{j=1}^n \sum_{\sigma \in \mathbb{V}(\Gamma_j)} \langle \sigma | \llbracket \emptyset : \emptyset \parallel \Gamma_j \vdash e_j : T \rrbracket^\dagger |v\rangle \cdot |v\rangle \in \mathcal{H}(T)^{\oplus n} \\
& \quad \llbracket \text{ortho}_T(e_1, \dots, e_n) \rrbracket^\dagger \\
& : \quad \text{inj}_j |v\rangle \\
& \mapsto \sum_{\sigma \in \mathbb{V}(\Gamma_j)} \langle \sigma | \llbracket \emptyset : \emptyset \parallel \Gamma_j \vdash e'_{j'} : T \rrbracket^\dagger |v\rangle \cdot |v\rangle \\
& \mapsto \sum_{\sigma \in \mathbb{V}(\Gamma_j)} \langle \sigma | \llbracket \emptyset : \emptyset \parallel \Gamma_j \vdash e_j : T \rrbracket^\dagger |v\rangle \cdot |v\rangle
\end{aligned}$$

□

From the operators constructed here, it is straightforward to prove Lemmas ?? and ??, by noting that the operators in those lemmas are $\llbracket \text{ortho}_T(e_1, \dots, e_n) \rrbracket^\dagger \llbracket \text{ortho}_T(e_1, \dots, e_n) \rrbracket$ and $\llbracket \text{spanning}_T(e_1, \dots, e_n) \rrbracket^\dagger \llbracket \text{spanning}_T(e_1, \dots, e_n) \rrbracket$. In the spanning case, this construction is norm-preserving, and in the ortho case, it is norm-non-increasing.

LEMMA K.2 (ERASURE COMPILATION). *Suppose $(\Gamma, \Gamma_j \parallel \Delta, \Delta' \vdash e'_j : T')$ for all $j \in \{1, \dots, n\}$ and $\text{erases}_{T'}(x; e'_1, \dots, e'_n)$ is true for all $x \in \text{dom}(\Delta)$. Then, one can implement a Kraus operator $\llbracket \text{erases}_{T'}(\Delta; e'_1, \dots, e'_n) \rrbracket : \mathcal{H}(\Delta) \otimes \mathcal{H}(T') \rightarrow \mathcal{H}(T')$ with the following behavior for all $\sigma \in \mathbb{V}(\Gamma), \sigma_j \in \mathbb{V}(\Gamma_j), \tau \in \mathbb{V}(\Delta), \tau' \in \mathbb{V}(\Delta')$:*

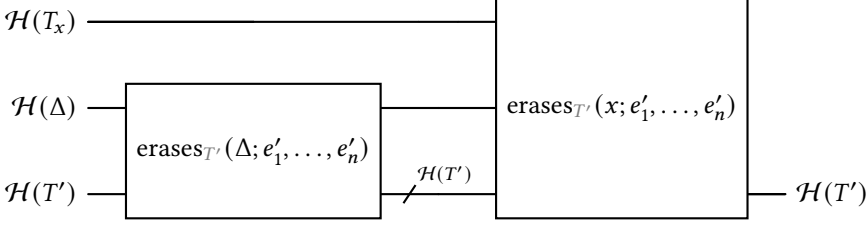
$$|\tau\rangle \otimes \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel \Delta, \Delta' \vdash e'_j : T' \rrbracket |\tau, \tau'\rangle \mapsto \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel \Delta, \Delta' \vdash e'_j : T' \rrbracket |\tau, \tau'\rangle$$

PROOF. We construct the circuit by recursing on Δ . In the base case, $\Delta = \emptyset$ and an identity operator (empty circuit) suffices. Thus, we focus on the inductive case where our context is $(x : T_x, \Delta)$, assuming the inductive hypothesis that $\llbracket \text{erases}_{T'}(\Delta; e'_1, \dots, e'_n) \rrbracket$ is implementable with the behavior:

$$|\tau\rangle \otimes \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta, \Delta' \vdash e'_j : T' \rrbracket |x \mapsto v, \tau, \tau'\rangle \mapsto \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta, \Delta' \vdash e'_j : T' \rrbracket |x \mapsto v, \tau, \tau'\rangle$$

(Note that x appears here even though this is the inductive hypothesis, which would normally be free of x . This is still a valid induction principle; we are effectively inducting on the number of variables that must be erased.) The problem is then reduced to implementing a Kraus operator $\llbracket \text{erases}_{T'}(x; e'_1, \dots, e'_n) \rrbracket : \mathcal{H}(T_x) \otimes \mathcal{H}(T') \rightarrow \mathcal{H}(T')$ with the following behavior:

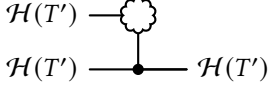
$$|v\rangle \otimes \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta, \Delta' \vdash e'_j : T' \rrbracket |x \mapsto v, \tau, \tau'\rangle \mapsto \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta, \Delta' \vdash e'_j : T' \rrbracket |x \mapsto v, \tau, \tau'\rangle$$



$$\begin{aligned}
 & \llbracket \text{erases}_{T'}(x : T_x, \Delta; e'_1, \dots, e'_n) \rrbracket \\
 & : \quad |x \mapsto v, \tau\rangle \otimes \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta, \Delta' \vdash e'_j : T' \rrbracket |x \mapsto v, \tau, \tau'\rangle \\
 & \mapsto |v\rangle \otimes \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta, \Delta' \vdash e'_j : T' \rrbracket |x \mapsto v, \tau, \tau'\rangle \\
 & \mapsto \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta, \Delta' \vdash e'_j : T' \rrbracket |x \mapsto v, \tau, \tau'\rangle
 \end{aligned}$$

The rest of this proof constructs this gate $\llbracket \text{erases}_{T'}(x; e'_1, \dots, e'_n) \rrbracket$ by induction on the rule used to prove the erasure judgment.

E-VAR. In this case, $e'_1 = \dots = e'_n = x$ and $T' = T_x$. We know that $\Delta = \Delta' = \emptyset$ because these contexts must be relevant.



$$\begin{aligned}
 & \llbracket \text{erases}_{T'}(x; e'_1, \dots, e'_n) \rrbracket \\
 & : \quad |v\rangle \otimes \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T' \vdash x : T' \rrbracket |x \mapsto v\rangle \\
 & = \quad |v\rangle \otimes |v\rangle \\
 & \mapsto |v\rangle \\
 & = \quad \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T' \vdash x : T' \rrbracket |x \mapsto v\rangle
 \end{aligned}$$

E-GPHASE. In this case, the circuit produced by the inductive hypothesis already has the needed behavior.

$$\begin{aligned}
 & \llbracket \text{erases}_{T'}(x; e'_1, \dots, e'_{j-1}, e'_j \triangleright \text{gphase}_T(r), e'_{j+1}, \dots, e'_n) \rrbracket \\
 & : \quad |v\rangle \otimes \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta, \Delta' \vdash e'_j \triangleright \text{gphase}_T(r) : T' \rrbracket |x \mapsto v, \tau, \tau'\rangle \\
 & = \quad |v\rangle \otimes e^{ir} \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta, \Delta' \vdash e'_j : T' \rrbracket |x \mapsto v, \tau, \tau'\rangle \\
 & \mapsto e^{ir} \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta, \Delta' \vdash e'_j : T' \rrbracket |x \mapsto v, \tau, \tau'\rangle \\
 & = \quad \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta, \Delta' \vdash e'_j \triangleright \text{gphase}_T(r) : T' \rrbracket |x \mapsto v, \tau, \tau'\rangle
 \end{aligned}$$

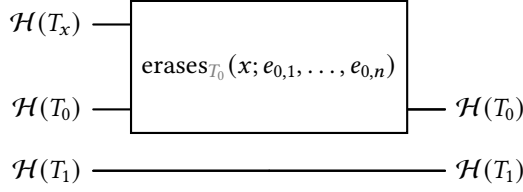
E-CTRL. This rule allows us effectively to “inline” the right side of the ctrl expressions for the purpose of the erases judgment. Assume one of the expressions is of the following form:

$$\text{ctrl } e \left\{ \begin{array}{c} e_{j,1} \mapsto e'_{j,1} \\ \dots \\ e_{j,m} \mapsto e'_{j,m} \end{array} \right\}_{T'}$$

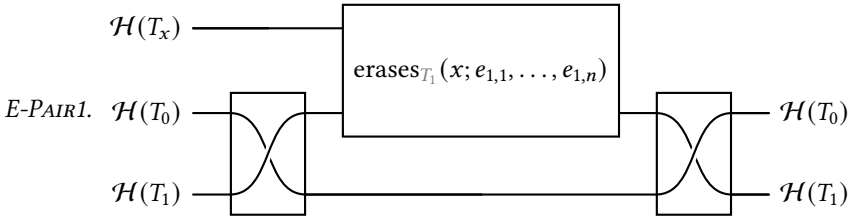
Then, we can use the fact that the semantics of `ctrl` is a linear combination of the semantics of its subexpressions:

$$\begin{aligned}
 & \llbracket \text{erases}_{T'}(x; e'_1, \dots, e'_{j-1}, \text{ctrl } e_T \{ \dots \}_{T'}, e'_{j+1}, \dots, e'_n) \rrbracket \\
 & : |v\rangle \otimes \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta, \Delta' \vdash \text{ctrl } e_T \{ \dots \}_{T'} : T' \rrbracket |x \mapsto v, \tau, \tau'\rangle \\
 & = \sum \dots |v\rangle \otimes \llbracket \sigma, \sigma_{j,k} : \Gamma, \Gamma_{j,k} \parallel x : T_x, \Delta, \Delta' \vdash e'_{j,k} : T' \rrbracket |x \mapsto v, \tau, \tau'\rangle \\
 & \mapsto \sum \dots \llbracket \sigma, \sigma_{j,k} : \Gamma, \Gamma_{j,k} \parallel x : T_x, \Delta, \Delta' \vdash e'_{j,k} : T' \rrbracket |x \mapsto v, \tau, \tau'\rangle \\
 & = \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta, \Delta' \vdash \text{ctrl } e_T \{ \dots \}_{T'} : T' \rrbracket |x \mapsto v, \tau, \tau'\rangle
 \end{aligned}$$

E-PAIR0.



$$\begin{aligned}
 & \llbracket \text{erases}_{T_0 \otimes T_1}(x; (e_{0,1}, e_{1,1}), \dots, (e_{0,n}, e_{1,n})) \rrbracket \\
 & : |v\rangle \otimes \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta, \Delta' \vdash (e_{0,j}, e_{1,j}) : T_0 \otimes T_1 \rrbracket |x \mapsto v, \tau, \tau'\rangle \\
 & = |v\rangle \otimes \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta_*, \Delta_0, \Delta_1, \Delta'_*, \Delta'_0, \Delta'_1 \vdash (e_{0,j}, e_{1,j}) : T_0 \otimes T_1 \rrbracket |x \mapsto v, \tau_*, \tau_0, \tau_1, \tau'_*, \tau'_0, \tau'_1\rangle \\
 & = |v\rangle \otimes \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta_*, \Delta_0, \Delta'_*, \Delta'_0 \vdash e_{0,j} : T_0 \rrbracket |x \mapsto v, \tau_*, \tau_0, \tau'_*, \tau'_0\rangle \\
 & \quad \otimes \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta_*, \Delta_1, \Delta'_*, \Delta'_1 \vdash e_{1,j} : T_1 \rrbracket |x \mapsto v, \tau_*, \tau_1, \tau'_*, \tau'_1\rangle \\
 & \mapsto \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta_*, \Delta_0, \Delta'_*, \Delta'_0 \vdash e_{0,j} : T_0 \rrbracket |x \mapsto v, \tau_*, \tau_0, \tau'_*, \tau'_0\rangle \\
 & \quad \otimes \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta_*, \Delta_1, \Delta'_*, \Delta'_1 \vdash e_{1,j} : T_1 \rrbracket |x \mapsto v, \tau_*, \tau_1, \tau'_*, \tau'_1\rangle \\
 & = \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta, \Delta' \vdash (e_{0,j}, e_{1,j}) : T_0 \otimes T_1 \rrbracket |x \mapsto v, \tau, \tau'\rangle
 \end{aligned}$$



$$\begin{aligned}
 & \llbracket \text{erases}_{T_0 \otimes T_1}(x; (e_{0,1}, e_{1,1}), \dots, (e_{0,n}, e_{1,n})) \rrbracket \\
 & : |v\rangle \otimes \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta, \Delta' \vdash (e_{0,j}, e_{1,j}) : T_0 \otimes T_1 \rrbracket |x \mapsto v, \tau, \tau'\rangle \\
 & = |v\rangle \otimes \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta_*, \Delta_0, \Delta_1, \Delta'_*, \Delta'_0, \Delta'_1 \vdash (e_{0,j}, e_{1,j}) : T_0 \otimes T_1 \rrbracket |x \mapsto v, \tau_*, \tau_0, \tau_1, \tau'_*, \tau'_0, \tau'_1\rangle \\
 & = |v\rangle \otimes \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta_*, \Delta_0, \Delta'_*, \Delta'_0 \vdash e_{0,j} : T_0 \rrbracket |x \mapsto v, \tau_*, \tau_0, \tau'_*, \tau'_0\rangle \\
 & \quad \otimes \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta_*, \Delta_1, \Delta'_*, \Delta'_1 \vdash e_{1,j} : T_1 \rrbracket |x \mapsto v, \tau_*, \tau_1, \tau'_*, \tau'_1\rangle \\
 & \mapsto \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta_*, \Delta_0, \Delta'_*, \Delta'_0 \vdash e_{0,j} : T_0 \rrbracket |x \mapsto v, \tau_*, \tau_0, \tau'_*, \tau'_0\rangle \\
 & \quad \otimes \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta_*, \Delta_1, \Delta'_*, \Delta'_1 \vdash e_{1,j} : T_1 \rrbracket |x \mapsto v, \tau_*, \tau_1, \tau'_*, \tau'_1\rangle \\
 & = \llbracket \sigma, \sigma_j : \Gamma, \Gamma_j \parallel x : T_x, \Delta, \Delta' \vdash (e_{0,j}, e_{1,j}) : T_0 \otimes T_1 \rrbracket |x \mapsto v, \tau, \tau'\rangle
 \end{aligned}$$

3186 We have thus demonstrated that a circuit with this semantics can always be constructed. $\square$

3187

3188

3189

3190

3191

3192

3193

3194

3195

3196

3197

3198

3199

3200

3201

3202

3203

3204

3205

3206

3207

3208

3209

3210

3211

3212

3213

3214

3215

3216

3217

3218

3219

3220

3221

3222

3223

3224

3225

3226

3227

3228

3229

3230

3231

3232

3233

3234